AETO TO-DO PROJECT · LEARNING MANUAL

Learn Hono — The Complete Beginner's Guide

Building, understanding and extending a full-stack To-Do application with Hono, TypeScript, SQLite and Drizzle ORM

Hono · TypeScript · Node.js · SQLite · Drizzle ORM · Zod · HTML · CSS · JavaScript · Vitest

Application version 1.0.0
Generated 2026-11-01 · Build 60981f4d1c
Source: docs/DOCUMENTATION-SOURCE.md + docs/CHANGELOG.md

Table of Contents

Preface — How to Use This Manual

- Who this manual is for
- How the book is organised
- How to read it
- Conventions used in this book
- Quick start

Chapter 1 — Introduction

- What are we building?
- What is a To-Do application?
- What is frontend development?
- What is backend development?
- What is full-stack development?
- What happens when someone opens a website?
- How does a browser communicate with a server?
- Try it yourself

Chapter 2 — Understanding Hono

- What is Hono?
- Why does Hono exist?
- What problem does Hono solve?
- What makes Hono different from Express?
- How does Hono process requests?
- What is a Hono application?
- What is a Hono context?
- What are routes?
- What are handlers?
- What is middleware?
- How does Hono organise applications?
- Try it yourself

Chapter 3 — Understanding Node.js

- What is Node.js?
- Why do we need it?
- How does JavaScript run outside the browser?
- What is npm? What is pnpm?
- What is a package?
- What is package.json?
- What are dependencies?
- What are development dependencies?
- The lockfile and pnpm-workspace.yaml
- How does the Node.js server start?

Try it yourself

Chapter 4 — Understanding TypeScript

What is TypeScript?
Variables and constants
Strings, numbers and booleans
Arrays
Objects
Functions and arrow functions
Types and interfaces
Generics
Type inference
Imports and exports
Promises, async and await
Error handling
Try it yourself

Chapter 5 — Understanding HTTP and APIs

HTTP
Requests
Responses
Methods: GET, POST, PUT, PATCH, DELETE
Status codes
Headers
URL parameters and query parameters
Request bodies and JSON
REST APIs
How the browser communicates with Hono
Try it yourself

Chapter 6 — Understanding Middleware

What middleware is
Why middleware exists
How middleware executes: the onion
Middleware execution order
next()
Logging middleware
Database middleware (a custom middleware that stores a value)
Validation middleware
Authentication and authorization middleware
Built-in middleware we use
Error middleware
Try it yourself

Chapter 7 — Understanding the Database

What is a database?
Why do we need one?
What is SQLite?
What is SQL?
Tables, rows and columns
What is a primary key?
What is a foreign key? What are relationships?
The actual database schema
What is Drizzle ORM?
What are migrations?
How does database persistence work?
Try it yourself

Chapter 8 — Understanding the Frontend

HTML
Views and the address bar (hash routing)
CSS
JavaScript
The DOM
Events and event listeners
The Fetch API
JSON parsing
Async requests and updating the interface
Logging in from the frontend
Loading states, error states and empty states
How the frontend communicates with Hono
Try it yourself

Chapter 9 — Complete Application Architecture

The big picture
The layers and their single responsibilities
Every folder and file
Dependency direction
Important architectural decisions
Try it yourself

Chapter 10 — Complete Code Flow

Flow 1: Opening the page (initial load)
Flow 2: Creating a task
Flow 3: A validation error
Flow 4: Listing, searching, filtering and sorting
Flow 5: Editing a task
Flow 6: Completing and reopening
Flow 7: Deleting a task
Flow 8: Managing categories

Flow 9: The dashboard

Flow 10: Errors that are not your fault

Chapter 11 — Understanding Every Source File

src/index.ts — the entry point

src/app.ts — builds the Hono application

src/config.ts — settings from environment variables

src/types.ts — the shape of the Hono environment

src/db/schema.ts — the tables

src/db/index.ts — connecting to the database

src/lib/errors.ts — our error classes

src/lib/dates.ts — date helpers

src/lib/password.ts and src/lib/login-throttle.ts

src/middleware/auth.ts

src/middleware/request-logger.ts, database.ts, validate.ts, error-handler.ts

src/validators/common.ts

src/validators/task.validator.ts

src/validators/category.validator.ts and dashboard.validator.ts

src/validators/auth.validator.ts

src/services/auth.service.ts

src/services/task.service.ts

src/services/subtask.service.ts and src/validators/subtask.validator.ts

src/services/category.service.ts

src/services/dashboard.service.ts

src/routes/*.routes.ts

public/index.html, public/css/styles.css, public/js/*.js

Configuration files

Scripts

Chapter 12 — API Reference

GET /api/health

POST /api/auth/register

POST /api/auth/login

POST /api/auth/logout

GET /api/auth/me

GET /api/admin/users

GET /api/admin/tasks

GET /api/tasks

GET /api/tasks/:id

POST /api/tasks

PATCH /api/tasks/:id

POST /api/tasks/:id/complete

POST /api/tasks/:id/reopen

DELETE /api/tasks/:id

POST /api/tasks/:id/subtasks
PATCH /api/subtasks/:id
DELETE /api/subtasks/:id
GET /api/categories
POST /api/categories
PATCH /api/categories/:id
DELETE /api/categories/:id
GET /api/dashboard

Chapter 13 — Testing

What is automated testing?
Unit tests and integration tests
Vitest
The test helper
What is tested
How to run the tests
Writing your own test

Chapter 14 — Security

Input validation
SQL injection
Safe database queries
XSS (Cross-Site Scripting)
Error handling
Limiting request size
Environment variables
Dependency security
Data protection
Authentication and authorization (summary)

Chapter 15 — Troubleshooting

How to troubleshoot anything
Common problems
Reading a stack trace
Starting fresh

Chapter 16 — Modifying the Application

The golden workflow
Recipe: add a new database field (example: notes)
Recipe: add a new filter (example: "created this week")
Recipe: add a new API endpoint (example: delete all completed tasks)
Recipe: add a new route group / new database table (example: comments on tasks)
Recipe: add a new page
Recipe: add a new frontend interaction (example: a "Start" button that sets status to in_progress)
Ideas for bigger features

Chapter 17 — Deployment

- What production needs
- Build process
- Database handling
- Server configuration example (Linux VPS)
- Your code on GitHub (Git basics)
- Deployment checklist

Chapter 18 — Learning Roadmap

- Stage 1 — Read and run (week 1–2)
- Stage 2 — Small changes (week 3–4)
- Stage 3 — Full features (month 2)
- Stage 4 — Deeper understanding (month 3)
- Stage 5 — Build your own app (month 4+)
- Further topics to explore

Chapter 19 — How This Manual Is Built

- The files
- The command
- What the generator does
- Design decisions
- Updating the manual

Chapter 20 — Authentication and Authorization

- Two different questions
- What we built
- The users and sessions tables
- Passwords: never store them
- Sessions and cookies: staying logged in
- The authentication middleware
- Flow A: Registering
- Flow B: Logging in
- Flow C: Every protected request
- Flow D: Logging out, and Flow E: an expired session
- Authorization: only your own data
- Roles: user and admin
- CSRF: requests your browser sends for someone else
- Upgrading an existing database
- Limitations and next steps
- Try it yourself

Chapter 21 — Teamwork: Assigning Tasks, Group Tasks, Subtasks and Views

- From "owner" to "assignees": a many-to-many relationship
- Who may do what with a task
- Assigning: who decides?

"Created by" — only for admins

Loading related data efficiently (avoiding "N+1 queries")

Subtasks: tasks inside a task

The four views

Upgrading existing data

Try it yourself

Development Log

Development Entry: #1 — Build the Hono To-Do Application and Its Learning Manual

Development Entry: #2 — Install the Project With pnpm Instead of npm

Development Entry: #3 — Add Authentication and Authorization

Development Entry: #4 — Question: What Is the Admin's ID and Password?

Development Entry: #5 — Bug Report: "Route POST /api/auth/register was not found."

Development Entry: #6 — Admin Assigns Tasks, Group & Individual Tasks, Subtasks, Separate Dashboard and Calendar Views, "Created By" for Admins

Development Entry: #7 — Different View Types in the Calendar

Development Entry: #8 — Rename to "Aeto To-Do", Logo Theme, and Publish on GitHub

Glossary

Appendix — Change History

2026-11-01 — Entry #8: Rename to "Aeto To-Do", logo theme, publish on GitHub

2026-11-01 — Entry #7: Different view types in the calendar

2026-11-01 — Entry #6: Admin assigns tasks, group and individual tasks, subtasks, separate views, "created by" for admins

2026-11-01 — Entry #5: Bug report "Route POST /api/auth/register was not found."

2026-11-01 — Entry #4: Question about the admin ID and password

2026-11-01 — Entry #3: Authentication and authorization

2026-11-01 — Entry #2: Install the project with pnpm

2026-11-01 — Entry #1: Initial build of the Hono To-Do application and learning manual

Preface — How to Use This Manual

Welcome! This manual teaches you how a real web application works, from the moment you click a button in the browser to the moment a row is saved in a database, using a working To-Do application as the example.

You do **not** need any previous programming knowledge. Every new word is explained the first time it appears, and there is a **Glossary** near the end of the book you can return to at any time.

Who this manual is for

This manual is written for an absolute beginner who wants to *understand* programming, not just copy code. By the end you should be able to:

- open this project and explain what every folder and file does;
- follow a request from the browser, through the Hono server, into the database and back;
- add new features (fields, filters, pages, API endpoints) on your own;
- start a brand-new Hono application without depending entirely on AI.

How the book is organised

Part	Chapters	What you learn
Foundations	1–5	What the web is, what Hono, Node.js, TypeScript and HTTP are
Building blocks	6–8	Middleware, databases and the frontend
The whole system	9–12	Architecture, request flows, every source file, the API
Quality	13–15	Testing, security, troubleshooting
Going further	16–19	Changing the app, deploying it, a learning roadmap, how this manual is built
Accounts	20	Authentication (who are you?) and authorization (what may you do?)
Teamwork	21	Assigning tasks, group tasks, subtasks, the Dashboard/Tasks/Calendar/Admin views
Development Log	—	A permanent record of every change ever requested, with explanations
Reference	—	Glossary and the change history

How to read it

1. Read chapters 1–5 in order. They build on each other.

2. Keep the project open in your editor (for example VS Code) while reading. Whenever the manual shows code, find the same code in the real file.
3. Do the **Try it yourself** exercises. Typing code and seeing what happens is how programming is learned.
4. When something breaks, go to **Chapter 15 — Troubleshooting**.

> **TIP**
> Code shown in this manual is copied from the real project files. If you ever find a difference between the manual and the code, the code is the truth — and the manual should be fixed.

Conventions used in this book

- `Text like this` is code, a file name, or something you type.
- Boxes labelled **Note**, **Tip**, **Warning** and **Important** contain extra information.
- Diagrams are drawn with text characters so they work everywhere, even offline and when printed. Read them from top to bottom, following the arrows (`↓`, `→`).
- Commands are shown for a terminal (PowerShell on Windows, Terminal on macOS/Linux). The commands are the same on every system unless stated otherwise.

Quick start

If you only want to run the application right now:

BASH

```
# 0. Download the project from GitHub (only needed once)
git clone https://github.com/minal-aetonet/aeto-to-do.git

# 1. Go into the project folder
cd aeto-to-do

# 2. Install the libraries the project needs (only needed once)
pnpm install

# 3. (Optional) Create a demo account with example tasks
pnpm db:seed

# 4. Start the server
pnpm start
```

Then open **http://localhost:3000** in your browser and click **Create one** to register an account. The **first account** ever created becomes the administrator. If you ran `pnpm db:seed`, you can instead log in as `demo@example.com` with the password `demo-password-123`. Press `Ctrl+C` in the terminal to stop the server.

Chapter 1 — Introduction

What are we building?

We are building a **To-Do application**: a website where you can write down tasks you need to do, organise them, and tick them off when they are done.

Our application has these features:

- **Dashboard** — numbers at the top of the page: total tasks, pending, in progress, completed, overdue, due today, and a completion percentage bar.
- **Task management** — create, view, edit, delete, complete and reopen tasks.
- **Task details** — every task has a title, description, status, priority, category, due date, created date and updated date.
- **Organisation** — categories (with colours), search, filters, sorting.
- **A friendly interface** — notifications ("Task created"), loading spinners, error messages and helpful "empty" messages.
- **Accounts** — register, log in and log out. Every user sees only the tasks assigned to them and their own categories. The first account is an administrator (Chapter 20).
- **Teamwork** — administrators assign tasks to one person (**individual task**) or several people (**group task**); every task can contain **subtasks** (a checklist); separate **Dashboard**, **Tasks**, **Calendar** and **Admin** views; only administrators see **who created** a task (Chapter 21).

Everything you do in the interface is stored in a real database file on your computer, so your tasks are still there after you close the browser or restart the server.

What is a To-Do application?

A To-Do application is a digital version of a paper checklist. It is one of the most popular first projects for learning web development because it contains every important idea in a small package:

Idea	Where it appears in a To-Do app
Showing data	The list of tasks
Creating data	The "New task" form
Changing data	Editing a task, ticking it as completed
Deleting data	The "Delete" button
Searching and filtering	The search box and filter dropdowns
Calculating summaries	The dashboard numbers

Idea	Where it appears in a To-Do app
Remembering data	The database

These four basic operations — **C**reate, **R**ead, **U**pdate, **D**elete — are so common that programmers call them **CRUD**. Almost every application you use (email, shopping, social media) is CRUD at its heart.

What is frontend development?

The **frontend** is the part of an application you can *see* and *touch*: buttons, text, colours, forms. It runs inside your web browser (Chrome, Edge, Firefox, Safari).

The frontend is built with three languages:

- **HTML** — the *structure*: "here is a heading, here is a button, here is a list".
- **CSS** — the *appearance*: colours, sizes, spacing, layout.
- **JavaScript** — the *behaviour*: "when the button is clicked, send the new task to the server and update the list".

In this project the frontend lives in the `public/` folder.

What is backend development?

The **backend** is the part of an application that runs on a **server** — a computer whose job is to answer requests from browsers. You cannot see the backend; you only see its results.

The backend is responsible for:

- receiving requests ("please create this task");
- checking that the data is valid ("a task must have a title");
- storing and reading data in a **database**;
- sending back a **response** ("here is the task you created").

In this project the backend lives in the `src/` folder and is built with **Hono**, running on **Node.js**, written in **TypeScript**.

What is full-stack development?

A **full-stack** developer works on both the frontend and the backend. The "stack" is the pile of technologies an application uses. Ours is:

```
┌──────────────────────────────────────────┐
│  Browser: HTML + CSS + JavaScript (public/) │   ← frontend
├──────────────────────────────────────────┤
│  Hono web framework (src/)               │
│  Zod validation                          │   ← backend
│  Drizzle ORM                             │
├──────────────────────────────────────────┤
│  SQLite database (data/todo.db)          │   ← storage
├──────────────────────────────────────────┤
│  Node.js (runs our TypeScript via tsx)   │   ← runtime
└──────────────────────────────────────────┘
```

What happens when someone opens a website?

When you type `http://localhost:3000` into your browser and press Enter, a surprising amount happens in a fraction of a second:

```
You type http://localhost:3000 and press Enter
                    ↓
1. The browser works out WHICH computer to talk to.
   "localhost" means "this same computer". Port 3000 is the
   "door" our server listens on.
                    ↓
2. The browser sends an HTTP REQUEST:  GET /
                    ↓
3. Our Hono server receives it. No API route matches "/",
   so the static-file middleware sends back public/index.html.
                    ↓
4. The browser reads the HTML and sees it needs more files:
     /css/styles.css   /js/app.js   /favicon.svg
   It sends one request for each.
                    ↓
5. The browser builds the page and runs app.js.
                    ↓
6. app.js asks "am I logged in?":  GET /api/auth/me
   The browser automatically sends the session cookie (if any).
     401 → show the login screen and stop here
     200 → continue
                    ↓
7. app.js asks the server for DATA (the cookie goes along again):
     GET /api/categories
     GET /api/tasks?...
     GET /api/dashboard?...
                    ↓
8. The server checks the session, reads only THIS user's rows
   from the database and replies with JSON.
                    ↓
9. app.js turns the JSON into visible task cards and numbers.
```

How does a browser communicate with a server?

They talk using **HTTP** (HyperText Transfer Protocol). A protocol is simply an agreed set of rules for a conversation — like the rules of a phone call: one person dials, the other answers, they take turns talking,

one hangs up.

HTTP conversations always have two halves:

1. The browser sends a **request** ("GET me the list of tasks").
2. The server sends back a **response** ("200 OK — here are the tasks").

The server never starts a conversation by itself; it only answers. Chapter 5 explains HTTP in detail.

Try it yourself

1. Start the app (`pnpm start`) and open http://localhost:3000.
2. In your browser, press `F12` to open the **Developer Tools** and click the **Network** tab.
3. Reload the page. You will see every request from the diagram above: `localhost` , `styles.css` , `app.js` , `tasks` , `dashboard` and so on.
4. Click on the `dashboard` request and then on **Response**. That is the raw data the server sent.

Chapter 2 — Understanding Hono

What is Hono?

Hono (Japanese for "flame" 🔥) is a **web framework** for JavaScript and TypeScript. A web framework is a library that does the repetitive work of a web server for you, so you only write the parts that are specific to your application.

Without a framework, you would have to read raw network data, work out which URL was requested, parse the request body, and build the response text by hand. With Hono you write:

```ts
app.get('/api/health', (c) => c.json({ data: { status: 'ok' } }));
```

This single line means: *"When someone sends a GET request to `/api/health`, answer with this JSON."* That line is real code from `src/app.ts`.

Why does Hono exist?

JavaScript used to run in only two kinds of places: browsers and Node.js servers. Today it also runs on many other platforms — Cloudflare Workers, Deno, Bun, AWS Lambda, Vercel and more. Each platform has slightly different ways of handling requests.

Hono was created to be:

- **Small and fast** — it has no dependencies and starts instantly.
- **Built on Web Standards** — it uses the standard `Request` and `Response` objects that browsers also use, so the same code runs on every platform.
- **TypeScript-first** — it gives you excellent auto-completion and catches mistakes before you run the code.

What problem does Hono solve?

Hono solves three everyday problems:

Problem	Hono's solution
"Which code should handle this URL?"	**Routing**: `app.get('/api/tasks', handler)`
"I need to run the same code for many requests (logging, security)"	**Middleware**: `app.use('*', requestLogger(...))`

Problem	Hono's solution
"Reading requests and writing responses is tedious"	**Context helpers**: `c.req.param()`, `c.json()`, `c.body()`

What makes Hono different from Express?

Express is the oldest and most famous Node.js framework (released in 2010). You will see it in many tutorials. Here is how the two compare:

	Express	Hono
Runs on	Node.js only	Node.js, Bun, Deno, Cloudflare, Vercel, AWS…
Request/response objects	Node-specific `req`, `res`	Web-standard `Request`, `Response` via the context `c`
Sending a response	`res.json(data)` (side effect)	`return c.json(data)` (you *return* the response)
TypeScript	Added later through separate type packages	Written in TypeScript; types are built in
Async errors	Older versions needed extra care	`async` handlers and thrown errors just work
Size	Larger, many dependencies	Tiny, zero dependencies

The biggest mental difference: **in Hono, a handler returns a Response**. In Express, a handler calls methods on `res` to send something.

How does Hono process requests?

```
HTTP request arrives at Node.js (port 3000)
                ↓
@hono/node-server converts it into a standard Request
                ↓
app.fetch(request)  ← the entry point of every Hono app
                ↓
Hono creates a Context object "c" for this request
                ↓
Hono finds all middleware and the route matching the
method + path, and runs them in order (an "onion")
                ↓
The route handler returns a Response (e.g. c.json(...))
                ↓
Middleware gets a chance to act on the way back out
                ↓
@hono/node-server writes the Response to the network
```

If anything throws an error along the way, Hono stops and calls the error handler registered with `app.onError(...)`. If no route matches, Hono calls the handler registered with `app.notFound(...)`.

What is a Hono application?

A Hono application is an object created with `new Hono()`. You register routes and middleware on it. In our project, `src/app.ts` builds it inside a function called `createApp`:

```ts
export function createApp({ db, logRequests = true, serveFrontend = true, secureCookies = false }:
CreateAppOptions) {
  const app = new Hono<AppEnv>();
  // ... middleware and routes are registered here ...
  return app;
}
```

The `<AppEnv>` part is a TypeScript **generic** (Chapter 4). It tells Hono what extra values we store on the context — in our case the database (`db`), the authentication settings (`auth`) and the logged-in user (`user`).

Notice that `createApp` does **not** start a server. It only *builds* the application object. `src/index.ts` starts the server. This separation matters: the automated tests can create an app with a temporary database and call it directly, without opening any network port.

What is a Hono context?

Every time a request arrives, Hono creates a fresh **context** object, always called `c` by convention. It is your toolbox for that one request:

You write	What it does
`c.req`	The incoming request
`c.req.method`	`"GET"`, `"POST"`, …
`c.req.path`	The path, e.g. `"/api/tasks/7"`
`c.req.param('id')`	A URL parameter (raw text)
`c.req.query('status')`	A query-string value (raw text)
`c.req.valid('json')`	The *validated* JSON body (after our validation middleware)
`c.json(data, status)`	Build a JSON response
`c.body(null, 204)`	Build a response with no body
`c.set('db', db)`	Store a value for later middleware/handlers

You write	What it does
`c.get('db')`	Read a stored value
`c.get('user')`	The logged-in user (stored by `requireAuth`)
`c.res`	The response (available in middleware after `await next()`)

What are routes?

A **route** connects an HTTP *method* and a *path* to some code. Here are real routes from `src/routes/task.routes.ts`:

```ts
taskRoutes.get('/', ...);              // GET    /api/tasks
taskRoutes.get('/:id', ...);           // GET    /api/tasks/7
taskRoutes.post('/', ...);             // POST   /api/tasks
taskRoutes.patch('/:id', ...);         // PATCH  /api/tasks/7
taskRoutes.post('/:id/complete', ...); // POST   /api/tasks/7/complete
taskRoutes.post('/:id/reopen', ...);   // POST   /api/tasks/7/reopen
taskRoutes.delete('/:id', ...);        // DELETE /api/tasks/7
```

`:id` is a **path parameter** — a placeholder. When someone requests `/api/tasks/7`, Hono matches the `/:id` route and remembers that `id` is `"7"`.

Grouping routes with `app.route()`

Instead of writing the full path every time, we create a small Hono app for each area and **mount** it under a prefix in `src/app.ts`:

```ts
app.route('/api/auth', authRoutes);
app.route('/api/tasks', taskRoutes);
app.route('/api/categories', categoryRoutes);
app.route('/api/dashboard', dashboardRoutes);
app.route('/api/admin', adminRoutes);
```

Now `taskRoutes.get('/:id')` answers `GET /api/tasks/:id`. A route group can also have its own middleware: the first line in `task.routes.ts` is `taskRoutes.use('*', requireAuth);`, which makes every task route require a login (Chapter 20).

What are handlers?

A **handler** is the function that runs when a route matches. It receives the context `c` and must return a response:

```ts
// GET /api/tasks/:id
taskRoutes.get('/:id', validate('param', idParamSchema), (c) => {
  const { id } = c.req.valid('param');
  const task = taskService.getTaskById(c.get('db'), c.get('user'), id);
  return c.json({ data: task });
});
```

Read it like a sentence:

0. (Before this route, `requireAuth` has already checked the login and stored the user on the context.)
1. `validate('param', idParamSchema)` — first, check that `:id` is a positive whole number (this is middleware; Chapter 6).
2. `(c) => { ... }` — then run this handler.
3. `c.req.valid('param')` — get the *already-validated* `id` (a real number, not text).
4. `taskService.getTaskById(c.get('db'), c.get('user'), id)` — ask the service layer to load task `id` from the database **if the logged-in user (the "viewer") may see it** — it is assigned to them, or they are an admin (Chapter 21).
5. `return c.json({ data: task })` — reply with the task as JSON (status 200 by default).

Notice the handler is short. It does not contain SQL or business rules. Its only job is to translate between **HTTP** and the **service** layer. This keeps code easy to read and test.

What is middleware?

Middleware is code that runs *before and/or after* route handlers, usually for many routes at once. Examples in our app: logging every request, adding security headers, limiting body size, putting the database on the context, validating input. Chapter 6 is entirely about middleware.

How does Hono organise applications?

Hono does not force a folder structure. We chose a common, beginner-friendly layered structure:

```
src/
├── index.ts        starts the server
├── app.ts          builds the Hono app (middleware + routes)
├── routes/         HTTP layer: which URL runs which code
├── middleware/     code that runs around routes
├── validators/     Zod schemas: what valid input looks like
├── services/       business logic + database queries
├── db/             database connection, schema, migrations
└── lib/            small shared helpers (errors, dates)
```

Each layer only talks to the layer below it: **routes → services → database**. Chapter 9 explains why.

Try it yourself

1. Open `src/app.ts`. Add a new route below the health check:

```ts
app.get('/api/hello', (c) => c.json({ message: 'Hello from Hono!' }));
```

2. Restart the server and open http://localhost:3000/api/hello.
3. Change it to read a query parameter: `c.req.query('name')`, and open `/api/hello?name=Sam`.
4. Remove the route again when you are finished (or keep it as a souvenir!).

Chapter 3 — Understanding Node.js

What is Node.js?

JavaScript was invented to run inside web browsers. **Node.js** is a program that lets JavaScript run *outside* the browser — on your computer or on a server — with extra abilities browsers do not give it, such as reading files, opening network ports and talking to databases.

Node.js is built on **V8**, the same JavaScript engine that powers Google Chrome.

Why do we need it?

Our backend is written in TypeScript/JavaScript. Something has to run it and keep it listening for requests. That something is Node.js. When you run `pnpm start`, Node.js starts, loads our code, and keeps running until you press `Ctrl+C`.

How does JavaScript run outside the browser?

```
Browser                          Node.js
────────                         ────────
JavaScript code                  JavaScript code
      ↓                                ↓
V8 engine                        V8 engine
      ↓                                ↓
Browser APIs:                    Node APIs:
 document, window,                fs (files), http (servers),
 fetch, localStorage              path, process, crypto
```

The language is the same; the available tools are different. In the browser there is a `document` (the web page) but no file system. In Node.js there is a file system (`node:fs`) but no `document`.

You can see Node APIs in our code. For example, `src/db/index.ts` uses the file-system module to create the `data` folder:

```ts
import { mkdirSync } from 'node:fs';
// ...
mkdirSync(dirname(filePath), { recursive: true });
```

What is npm? What is pnpm?

Programmers share reusable code as **packages**. **npm** (Node Package Manager) is both:

1. the huge public **registry** (online library) of packages at npmjs.com, and
2. the command-line tool that comes with Node.js for downloading them.

pnpm ("performant npm") is an alternative command-line tool that downloads from the *same* registry. **This project uses pnpm.** Why?

- **Faster and smaller** — pnpm stores each package version once on your computer (in a shared "store") and links it into projects, instead of copying it into every project.
- **Stricter** — your code can only use packages that are listed in `package.json`, which prevents "it works on my computer" surprises.
- **Safer by default** — pnpm does not run packages' install scripts unless you allow them (see `pnpm-workspace.yaml` below).

Task	npm	pnpm
Install everything in package.json	`npm install`	`pnpm install`
Add a library	`npm install zod`	`pnpm add zod`
Add a development tool	`npm install -D vitest`	`pnpm add -D vitest`
Remove a library	`npm uninstall zod`	`pnpm remove zod`
Run a script	`npm run test`	`pnpm test` (or `pnpm run test`)
Run a tool from node_modules	`npx drizzle-kit`	`pnpm exec drizzle-kit`

> **NOTE**
> If pnpm is not installed, install it once with `npm install -g pnpm` or with Node's built-in Corepack: `corepack enable pnpm`.

What is a package?

A **package** is a folder of code with a `package.json` file describing it, published so others can use it. When you install packages, they are placed in the `node_modules/` folder. You never edit `node_modules/` by hand, and it is never committed to git — anyone can recreate it with `pnpm install`.

What is package.json?

`package.json` is the identity card and instruction sheet of a Node.js project. Here is ours:

```json
{
  "name": "aeto-to-do",
  "version": "1.0.0",
  "type": "module",
  "private": true,
  "packageManager": "pnpm@12.4.1",
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "start": "tsx src/index.ts",
    "typecheck": "tsc --noEmit",
    "test": "vitest run",
    "test:watch": "vitest",
    "test:e2e": "tsx scripts/e2e-smoke.ts",
    "db:generate": "drizzle-kit generate",
    "db:migrate": "tsx scripts/migrate.ts",
    "db:seed": "tsx scripts/seed.ts",
    "docs:build": "tsx scripts/build-docs.ts"
  },
  "engines": { "node": ">=22" },
  "dependencies": { "...": "see below" },
  "devDependencies": { "...": "see below" }
}
```

Field	Meaning
`name`, `version`	The project's name and version number
`type: "module"`	Use modern JavaScript modules (`import` / `export`)
`private: true`	Prevents accidentally publishing this project to the npm registry
`packageManager`	Tells tools this project uses pnpm version 12.4.1
`scripts`	Named commands you run with `pnpm <name>`
`engines`	The Node.js version required (22 or newer, because we use `process.loadEnvFile`)
`dependencies`	Packages needed to *run* the app
`devDependencies`	Packages needed only while *developing* (tests, type checking, docs)

The scripts

Command	What it does
`pnpm dev`	Starts the server and **restarts it automatically** when you save a file
`pnpm start`	Starts the server normally
`pnpm typecheck`	Asks TypeScript to check all code for type mistakes
`pnpm test`	Runs the automated tests once
`pnpm test:watch`	Re-runs tests whenever you save a file

Command	What it does
`pnpm test:e2e`	Opens the real app in a headless browser and clicks through it
`pnpm db:generate`	Creates a new SQL migration after you change `src/db/schema.ts`
`pnpm db:migrate`	Applies migrations to the database file
`pnpm db:seed`	Fills an empty database with example data
`pnpm docs:build`	Regenerates this manual (HTML + PDF)

What are dependencies?

Dependencies are packages your application needs while running:

Package	Why we need it
`hono`	The web framework
`@hono/node-server`	Connects Hono to Node.js's HTTP server and serves static files
`@hono/zod-validator`	Connects Zod validation to Hono routes
`zod`	Describes and checks the shape of data
`drizzle-orm`	Lets us write database queries in TypeScript
`better-sqlite3`	The driver that actually talks to the SQLite database file
`tsx`	Runs TypeScript files directly in Node.js

What are development dependencies?

Development dependencies (`devDependencies`) are tools for programmers. The application does not need them to serve users:

Package	Why we need it
`typescript`	The TypeScript type checker (`tsc`)
`@types/node`, `@types/better-sqlite3`	Type descriptions for libraries written in plain JavaScript
`vitest`	The testing framework
`drizzle-kit`	Generates SQL migration files from our schema
`marked`, `highlight.js`	Turn this manual's Markdown into HTML with coloured code
`puppeteer-core`	Controls a real browser to print the PDF and run the browser smoke test

The lockfile and pnpm-workspace.yaml

- `pnpm-lock.yaml` records the *exact* version of every package (and every package *they* depend on) that was installed. `package.json` says `"hono": "^4.13.16"` ("4.13.16 or a compatible newer 4.x"); the lockfile pins the precise version so everyone gets identical code. Commit it to git; never edit it by hand.
- `pnpm-workspace.yaml` holds pnpm settings. Ours decides which packages may run install ("build") scripts:

YAML

```yaml
allowBuilds:
  better-sqlite3: false
  esbuild: true
minimumReleaseAgeExclude:
  - hono@4.13.16
```

`esbuild` (used internally by `tsx` and `vitest`) needs its script to install its program. `better-sqlite3` already contains ready-made binaries for Windows, macOS and Linux in its `prebuilds` folder, so compiling it from C++ source is unnecessary — and would fail on computers without C++ build tools — so we set it to `false`. `minimumReleaseAgeExclude` was added by pnpm itself: pnpm normally refuses package versions that are only a few hours old (a supply-chain safety feature), and this line allows that one specific Hono version.

How does the Node.js server start?

When you run `pnpm start`:

```
pnpm start
    ↓  pnpm reads package.json → "start": "tsx src/index.ts"
tsx src/index.ts
    ↓  tsx converts TypeScript to JavaScript in memory, Node runs it
src/index.ts
    ↓  imports src/config.ts → reads .env, validates settings
    ↓  createDatabase(...)   → opens data/todo.db
    ↓  runMigrations(db)     → creates/updates tables
    ↓  createApp({ db })     → builds the Hono app
    ↓  serve({ fetch: app.fetch, port: 3000 })
"To-Do app running at http://localhost:3000"
    ↓  Node keeps running, waiting for requests…
```

The full file is explained in Chapter 11.

Try it yourself

1. Run `node --version` and `pnpm --version` in a terminal.
2. Run `pnpm list` in the project folder to see the installed packages.

3. Create a file `hello.js` anywhere containing `console.log('Hello from Node', process.version)` and run `node hello.js`. You just ran JavaScript outside a browser.

Chapter 4 — Understanding TypeScript

What is TypeScript?

TypeScript is JavaScript with **types** added. A *type* describes what kind of value something is: a piece of text, a number, a list, and so on. TypeScript checks your code *before* it runs and warns you about mistakes like "you are trying to use a number as if it were text".

```
You write  .ts files (TypeScript)
    ↓
tsc checks types        (pnpm typecheck)  → shows mistakes, changes nothing
tsx removes the types   (pnpm start)      → runs the remaining JavaScript
    ↓
Node.js runs plain JavaScript
```

Types exist only while you write code. They are removed before the code runs, so they cost nothing at run time. The browser files in `public/js/` are plain JavaScript (`.js`) because browsers cannot run TypeScript directly and we chose not to add a build step for the frontend.

Variables and constants

A **variable** is a named box that holds a value.

- `const` creates a box whose value can never be replaced.
- `let` creates a box whose value can be replaced later.

```ts
const timestamp = nowIso();      // from src/services/task.service.ts — never reassigned
let orderBy: SQL[];              // from the same file — assigned later in a switch
```

Prefer `const`. Use `let` only when you really need to change the value. (You may see the old keyword `var` in tutorials; we do not use it.)

Strings, numbers and booleans

These are the basic **primitive types**:

Type	Examples	Used in our app for
`string`	`'Buy milk'`, `"pending"`, `` `Task ${id}` ``	titles, statuses, dates
`number`	`7`, `3.5`, `100 * 1024`	IDs, counts, percentages

Type	Examples	Used in our app for
`boolean`	`true`, `false`	settings like `logRequests`
`null`	`null`	"no value", e.g. a task without a due date
`undefined`	`undefined`	"not provided", e.g. an optional field left out

Strings written with back-ticks (`` ` ``) are **template literals**: `${...}` inserts a value.

```ts
throw new NotFoundError(`Task ${id} was not found.`);  // src/services/task.service.ts
```

Arrays

An **array** is an ordered list. `string[]` means "a list of strings".

```ts
export const TASK_STATUSES = ['pending', 'in_progress', 'completed'] as const;  // src/db/schema.ts
const conditions: SQL[] = [];          // an empty list of SQL conditions
conditions.push(eq(tasks.status, query.status));  // add one item to the end
```

Arrays have useful methods such as `.map()` (transform every item), `.filter()` (keep some items), `.some()` (is at least one item true?) and `.join()` (glue into a string):

```ts
// src/middleware/validate.ts — turn Zod's issues into simple objects
const details = result.error.issues.map((issue) => ({
  field: issue.path.join('.') || target,
  message: issue.message,
}));
```

Objects

An **object** groups named values (called *properties*):

```ts
export const config = {
  port: parsed.data.PORT,
  databasePath: parsed.data.DATABASE_PATH,
  logRequests: parsed.data.LOG_REQUESTS,
};
// config.port → 3000
```

Two very common object tricks:

```ts
// Destructuring: pull properties out into variables
const { id } = c.req.valid('param');

// Spread: copy all properties of one object into another
return { ...row, completionPercentage, today };
```

Functions and arrow functions

A **function** is a reusable piece of code that receives inputs (**parameters**) and can give back an output (**return value**).

```ts
// A normal function declaration, with parameter and return types
export function todayLocal(): string {
  const now = new Date();
  // ...
  return `${year}-${month}-${day}`;
}
```

An **arrow function** is a shorter way to write a function, very common for small callbacks:

```ts
app.get('/api/health', (c) => c.json({ data: { status: 'ok' } }));
//                      ^^^^^^ arrow function: receives c, returns c.json(...)
```

Types and interfaces

You can give a name to the shape of an object using `type` (or `interface`, which is very similar). We use `type` throughout this project for consistency.

```ts
// src/services/dashboard.service.ts
export type DashboardStats = {
  total: number;
  completed: number;
  pending: number;
  inProgress: number;
  overdue: number;
  dueToday: number;
  completionPercentage: number;
  today: string;
};
```

Now if a function says it returns `DashboardStats`, TypeScript makes sure every property is present and has the right type.

Union types say "one of these":

```ts
category: Pick<Category, 'id' | 'name' | 'color'> | null;   // an object OR null
```

`Pick<Category, 'id' | 'name' | 'color'>` means "the `Category` type, but only those three properties".

Generics

A **generic** is a type with a "blank" you fill in, written with angle brackets `< >`. Think of a box labelled "a list of ____".

```ts
const conditions: SQL[] = [];                  // Array of SQL (short form of Array<SQL>)
const app = new Hono<AppEnv>();                // a Hono app whose context stores AppEnv values
sql<number>`COUNT(*)`                          // a SQL expression that produces a number
```

You will mostly *use* generics from libraries rather than write your own. One we wrote is in `src/middleware/validate.ts`:

```ts
export function validate<Target extends keyof ValidationTargets, Schema extends ZodType>(target:
Target, schema: Schema)
```

It means: "`validate` works for any `Target` (json, query, param, ...) and any Zod `Schema`, and remembers which ones you passed so `c.req.valid(...)` has the right type."

Type inference

TypeScript can often work out types by itself. This is **type inference**:

```ts
const timestamp = nowIso();   // TypeScript knows this is a string because nowIso returns string
```

Libraries take inference much further. Drizzle infers TypeScript types from our database tables, and Zod infers types from our validation schemas:

```ts
// src/db/schema.ts
export type Task = typeof tasks.$inferSelect;

// src/validators/task.validator.ts
export type CreateTaskInput = z.infer<typeof createTaskSchema>;
```

This means we describe each shape **once**, and the types follow automatically. If you add a column to the table, the `Task` type updates by itself.

Imports and exports

Each file is a **module**. A module shares things with `export` and uses things from other modules with `import`.

```ts
// src/lib/dates.ts
export function nowIso(): string { ... }

// src/services/task.service.ts
import { nowIso, todayLocal } from '../lib/dates';
```

- `'../lib/dates'` is a **relative path**: go up one folder (`..`), then into `lib/`, file `dates.ts`.
- `'hono'` (no dot) means a package from `node_modules`.
- `'node:fs'` means a module built into Node.js.
- `import type { ... }` imports *only types* — it disappears completely when the code runs.
- `import * as taskService from '...'` imports everything a module exports, grouped under one name: `taskService.listTasks(...)`.

Promises, async and await

Some operations take time: a network request, reading a file, waiting for a timer. JavaScript does not stop and wait for them; instead it gives you a **Promise** — an object that says "I will have the answer later".

`async` and `await` let you write code that waits for a Promise as if it were normal step-by-step code:

```js
// public/js/app.js
async function loadDashboard() {
  try {
    renderDashboard(await api.getDashboard());   // wait for the server's answer
  } catch (error) {
    showToast(`Could not load dashboard: ${error.message}`, 'error');
  }
}
```

- A function marked `async` always returns a Promise.
- `await` pauses *this function only* until the Promise settles. The rest of the page keeps working.
- `Promise.all([a, b])` waits for several Promises at the same time:

```js
function refresh() {
  return Promise.all([loadTasks(), loadDashboard()]);
}
```

In Hono middleware, `await next()` waits for the rest of the request (later middleware and the handler) to finish.

> **NOTE**
>
> Our *database* calls are **synchronous** (no `await`): the `better-sqlite3` driver reads the local file so quickly that it simply returns the result directly. This keeps the service code simple. If you later switch to a remote database, those calls would become `async` and need `await`.
>
> **Password hashing is different:** it is *deliberately* slow (Chapter 20), so `hashPassword` and `verifyPassword` are `async`, and so are the services and route handlers that use them (`registerUser`, `authenticate`, the `/register` and `/login` handlers). Hono handlers may be `async` — Hono simply waits for the returned Promise.

Error handling

When something goes wrong, code can **throw** an error. Throwing immediately stops the current function and jumps to the nearest `catch`.

TS

```ts
// src/services/task.service.ts — throw when the task is missing
if (!row) {
  throw new NotFoundError(`Task ${id} was not found.`);
}
```

JS

```js
// public/js/app.js — catch errors and show them to the user
try {
  await api.deleteTask(task.id);
  showToast('Task deleted', 'success');
} catch (error) {
  showToast(error.message, 'error');
}
```

`finally { ... }` runs whether or not an error happened — we use it to re-enable the Save button.

We also create our own error **classes** in `src/lib/errors.ts`. A class is a blueprint for objects; `extends` builds a new blueprint on top of an existing one:

TS

```ts
export class NotFoundError extends AppError {
  constructor(message: string) {
    super(message, 404, 'NOT_FOUND');   // super(...) calls the parent class's constructor
  }
}
```

On the backend, we never catch errors inside routes. They travel up to the **error handler middleware** (Chapter 6), which turns them into proper HTTP responses in one central place.

Try it yourself

1. In `src/lib/dates.ts`, change the return type of `nowIso` from `string` to `number` and run `pnpm typecheck`. Read the error, then undo the change.
2. In `src/services/dashboard.service.ts`, delete the line `today: string;` from `DashboardStats` and run `pnpm typecheck`. What does TypeScript say? Undo it.
3. Hover your mouse over `CreateTaskInput` in VS Code to see the type that Zod inferred.

Chapter 5 — Understanding HTTP and APIs

HTTP

HTTP is the language browsers and servers use to talk. Every HTTP exchange is one **request** followed by one **response**.

Requests

A request has four parts:

```
POST /api/tasks HTTP/1.1                    ← 1. method + 2. URL (path + query)
Host: localhost:3000                        ┐
Content-Type: application/json              ├ 3. headers (extra information)
Content-Length: 41                          ┘

{"title":"Buy milk","priority":"high"}      ← 4. body (optional data)
```

Responses

A response has a **status code**, headers and an optional body:

```
HTTP/1.1 201 Created                        ← status code + reason
Content-Type: application/json              ┐
Content-Security-Policy: default-src 'self' ├ headers
X-Content-Type-Options: nosniff             ┘

{"data":{"id":7,"title":"Buy milk",...}}    ← body
```

Methods: GET, POST, PUT, PATCH, DELETE

The **method** says what kind of action the request wants:

Method	Meaning	Has a body?	Used in our API
`GET`	Read something. Must not change anything.	No	List tasks, get one task, categories, dashboard
`POST`	Create something, or perform an action	Usually	Create a task/category; complete/reopen a task
`PUT`	*Replace* something completely	Yes	Not used (see below)
`PATCH`	Change *part* of something	Yes	Edit a task or category

Method	Meaning	Has a body?	Used in our API
`DELETE`	Remove something	No	Delete a task or category

> **NOTE**
>
> **Why PATCH and not PUT?** With `PUT`, the client must send *every* field, and missing fields would be erased. With `PATCH`, the client sends only what changed (`{"title":"New"}`) and everything else stays the same. That is what our edit form and tests need, so we use `PATCH`.

> **NOTE**
>
> **Why are complete/reopen `POST`?** They are *actions* ("complete this task") rather than plain field edits. A separate URL makes the intention obvious in logs and code. They could also have been done with `PATCH /api/tasks/7` and `{"status":"completed"}` — both work, and our API supports both.

Status codes

The **status code** is a three-digit number summarising the result. The first digit is the category:

Range	Meaning
2xx	Success
3xx	Redirection (go somewhere else)
4xx	The **client** made a mistake
5xx	The **server** made a mistake

Codes used by our application:

Code	Name	When our app sends it
200	OK	Successful GET, PATCH, complete, reopen
201	Created	A task or category was created
204	No Content	A task or category was deleted (nothing to send back)
400	Bad Request	Validation failed, invalid JSON, or unknown category
401	Unauthorized	Not logged in, session expired, or wrong email/password ("who are you?")
403	Forbidden	Logged in, but not allowed — e.g. a normal user opening an admin route, or a cross-site request blocked by CSRF protection ("you may not")

Code	Name	When our app sends it
404	Not Found	The task/category/route does not exist — **or belongs to another user**
409	Conflict	A category with that name already exists, or an email is already registered
413	Payload Too Large	The request body is bigger than 100 KB
429	Too Many Requests	Too many failed logins for one email; wait 15 minutes
500	Internal Server Error	An unexpected bug on the server

Headers

Headers are `Name: value` lines carrying extra information. Important ones in our app:

Header	Direction	Purpose
`Content-Type: application/json`	both	"The body is JSON"
`Content-Security-Policy`	response	Tells the browser which sources of scripts/styles are allowed (security)
`X-Content-Type-Options: nosniff`	response	Stops the browser from guessing file types (security)
`Set-Cookie: session=…; HttpOnly; SameSite=Lax`	response	Asks the browser to store the login cookie (after register/login)
`Cookie: session=…`	request	The browser sends the stored cookie back with every request to our server — this is how the server knows who you are
`Origin` / `Sec-Fetch-Site`	request	Tell the server which website started the request; used by CSRF protection

Cookies

A **cookie** is a small named value that a server asks the browser to remember (`Set-Cookie`). From then on, the browser automatically attaches it to every request to that same server (`Cookie`). HTTP itself has no memory — each request is independent — so cookies are how a website "remembers" that you logged in. Chapter 20 explains our session cookie in detail.

URL parameters and query parameters

A URL can carry information in two ways:

```
http://localhost:3000/api/tasks/7/complete
                      └──────┘ └┘
                        path    path parameter  (:id = "7")

http://localhost:3000/api/tasks?status=pending&sort=dueDate&order=asc
                                └──────────────────────────────────┘
                                  query string: key=value pairs joined by &
```

- **Path parameters** identify *which* resource (task 7).
- **Query parameters** describe *how* you want it (filtered, sorted).

Both always arrive as **text**. That is why our validators use `z.coerce.number()` to turn `"7"` into `7`.

Request bodies and JSON

The **body** carries data, usually for POST/PATCH. Our API uses **JSON** (JavaScript Object Notation), a text format that looks like JavaScript objects:

JSON

```
{
  "title": "Write report",
  "description": "Quarterly numbers",
  "status": "in_progress",
  "priority": "high",
  "categoryId": 1,
  "dueDate": "2026-12-01"
}
```

JSON rules: keys in double quotes; values can be strings, numbers, `true` / `false`, `null`, arrays `[...]` or objects `{...}`.

- In the browser, `JSON.stringify(object)` turns an object into JSON text, and `response.json()` turns JSON text back into an object.
- In Hono, `c.json(object)` turns an object into a JSON response, and the validator reads JSON bodies for us.

REST APIs

An **API** (Application Programming Interface) is a set of URLs a program can call to get or change data. Our frontend is the program that calls our API.

REST is a popular style for designing web APIs:

- URLs name **resources** (nouns): `/api/tasks`, `/api/tasks/7`, `/api/categories`.
- HTTP **methods** say what to do (verbs): GET, POST, PATCH, DELETE.
- Responses use **status codes** honestly.
- Each request contains everything needed to handle it (the server does not remember previous requests — which is why the browser sends the session cookie with *every* request).

Our responses always have the same "envelope", which makes the frontend simpler:

```json
{ "data": { "...": "the result" } }
```

```json
{ "error": { "code": "VALIDATION_ERROR", "message": "The request contains invalid data.", "details": [
{ "field": "title", "message": "Title is required" } ] } }
```

How the browser communicates with Hono

```
public/js/api.js                          src/ (Hono)
────────────────                          ──────────
fetch('/api/tasks', {
  method: 'POST',
  headers: {Content-Type: json},  ──►  app.route('/api/tasks', taskRoutes)
  body: JSON.stringify(task)           taskRoutes.post('/', validate(...), handler)
})                                              ↓
                                       return c.json({ data: task }, 201)
const payload = await response.json() ◄──
return payload.data
```

Try it yourself

With the server running, try these in PowerShell (Windows). Because the task API requires a login, we first log in and keep the cookie in a "web session" variable `$s` (use your own email and password):

```powershell
# Not logged in yet: 401
Invoke-WebRequest http://localhost:3000/api/tasks
# Log in; -SessionVariable stores the cookie in $s
Invoke-RestMethod http://localhost:3000/api/auth/login -Method Post -ContentType 'application/json' -
Body '{"email":"you@example.com","password":"your-password"}' -SessionVariable s
# Read (the cookie in $s is sent along)
Invoke-RestMethod http://localhost:3000/api/tasks -WebSession $s
# Create
Invoke-RestMethod http://localhost:3000/api/tasks -Method Post -ContentType 'application/json' -Body
'{"title":"From PowerShell"}' -WebSession $s
# Trigger a validation error (see the 400 response)
Invoke-WebRequest http://localhost:3000/api/tasks -Method Post -ContentType 'application/json' -Body
'{"title":""}' -WebSession $s
```

Or with `curl` (macOS, Linux and modern Windows). `-c jar.txt` saves cookies to a file, `-b jar.txt` sends them:

```bash
curl -c jar.txt -X POST http://localhost:3000/api/auth/login -H "Content-Type: application/json" -d
'{"email":"you@example.com","password":"your-password"}'
curl -b jar.txt http://localhost:3000/api/dashboard
curl -b jar.txt -X POST http://localhost:3000/api/tasks -H "Content-Type: application/json" -d
'{"title":"From curl"}'
# Requests without a JSON body must say where they come from (CSRF protection, Chapter 20):
curl -b jar.txt -X DELETE -i http://localhost:3000/api/tasks/1 -H "Origin: http://localhost:3000"
```

Chapter 6 — Understanding Middleware

What middleware is

Middleware is a function that runs *around* your route handlers. It can:

- do something **before** the handler (log the start time, check input, store the database on the context);
- **stop** the request early (reject invalid data);
- do something **after** the handler (log how long it took, add headers to the response).

Why middleware exists

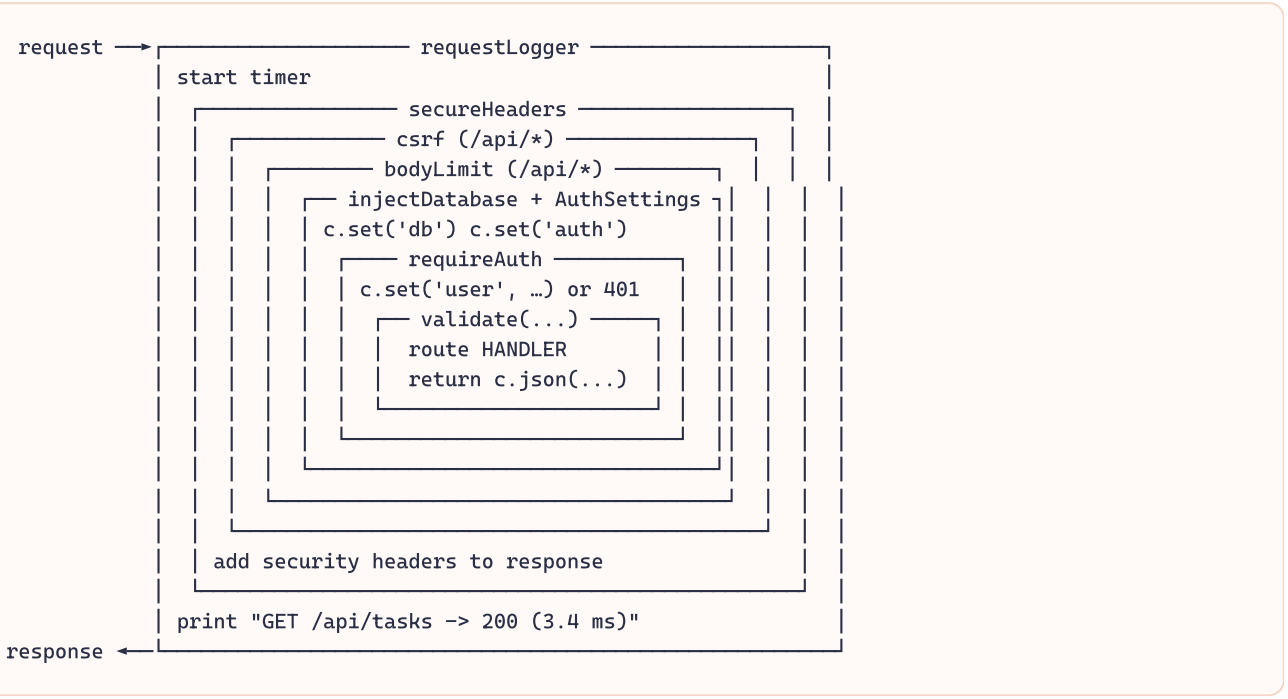
Imagine you want to log every request. Without middleware you would copy logging code into all eighteen API routes — and forget it in the nineteenth. Middleware lets you write it **once** and apply it everywhere:

```ts
app.use('*', requestLogger({ enabled: logRequests }));
```

`app.use(path, middleware)` registers middleware. `'*'` means "every path"; `'/api/*'` means "every path starting with /api/".

How middleware executes: the onion

Hono runs middleware like the layers of an onion. A request travels *inward* through each layer to the handler, and the response travels back *outward* through the same layers in reverse order.



Middleware execution order

Middleware runs **in the order it is registered**. Here is the real order from `src/app.ts`:

#	Registered as	Applies to	Purpose
1	`requestLogger`	every request	Log method, path, status, duration
2	`secureHeaders`	every request	Add security headers
3	`csrf`	`/api/*`	Block state-changing requests started by other websites
4	`bodyLimit`	`/api/*`	Reject bodies over 100 KB
5	`injectDatabase`	`/api/*`	Put the database on the context
6	`injectAuthSettings`	`/api/*`	Put the cookie settings and login throttle on the context
7	`requireAuth`	task, category, dashboard, admin routes and `/api/auth/me`	Check the session cookie; store the user or answer 401
8	`requireRole('admin')`	admin routes	Answer 403 unless the user is an admin
9	`validate(...)`	specific routes	Check params/query/body
10	the route handler	its route	Do the actual work
—	`serveStatic`	other paths	Send files from `public/`
—	`app.notFound`	no match	JSON 404

#	Registered as	Applies to	Purpose
—	`app.onError`	any thrown error	JSON error response

The logger is first on purpose: it wraps everything, so its timer measures the whole request, including all other middleware.

next()

Every middleware receives `next`, a function that means "continue to the next layer". Code before `await next()` runs on the way **in**; code after it runs on the way **out**.

If a middleware does **not** call `next()` (or throws an error), the request stops there and the inner layers never run.

Logging middleware

`src/middleware/request-logger.ts`:

```ts
import { createMiddleware } from 'hono/factory';

export function requestLogger(options: { enabled: boolean }) {
  return createMiddleware(async (c, next) => {
    if (!options.enabled) {
      await next();
      return;
    }

    const start = performance.now();
    await next();
    const durationMs = (performance.now() - start).toFixed(1);
    console.log(`${c.req.method} ${c.req.path} -> ${c.res.status} (${durationMs} ms)`);
  });
}
```

Line by line:

1. `requestLogger(options)` is a **factory**: a function that *creates* the middleware. This lets us switch logging off (the tests pass `enabled: false` to keep their output clean).
2. `createMiddleware(...)` is a Hono helper that gives the function the correct TypeScript types.
3. If logging is disabled, just call `next()` and stop.
4. Otherwise remember the start time, `await next()` — the whole rest of the request happens here — then read `c.res.status` (the response now exists) and print one line.

Example output in the terminal:

```
GET /api/tasks -> 200 (4.0 ms)
POST /api/tasks -> 400 (11.0 ms)
```

> **TIP**
> Hono has a built-in logger, `import { logger } from 'hono/logger'`. We wrote our own because it is the best way to learn how middleware works.

Database middleware (a custom middleware that stores a value)

`src/middleware/database.ts`:

```ts
export function injectDatabase(db: AppDatabase) {
  return createMiddleware<AppEnv>(async (c, next) => {
    c.set('db', db);
    await next();
  });
}
```

It stores the database on the context; handlers read it with `c.get('db')`. The type `AppEnv` (in `src/types.ts`) tells TypeScript that `c.get('db')` returns an `AppDatabase`.

Why not simply import a global database in every file? Because then the tests could not give the app a different, temporary database. Passing it in is called **dependency injection**. It is a fancy name for a simple idea: *give a piece of code the things it needs, instead of letting it grab them itself*.

Validation middleware

`src/middleware/validate.ts` wraps `zValidator` from `@hono/zod-validator`:

```ts
export function validate<Target extends keyof ValidationTargets, Schema extends ZodType>(
  target: Target,
  schema: Schema,
) {
  return zValidator(target, schema, (result) => {
    if (!result.success) {
      const details = result.error.issues.map((issue) => ({
        field: issue.path.join('.') || target,
        message: issue.message,
      }));
      throw new ValidationError('The request contains invalid data.', details);
    }
  });
}
```

- `target` is which part of the request to check: `'json'` (body), `'query'` (query string) or `'param'` (path parameters).
- `schema` is a Zod schema that describes valid data (Chapter 7 and Chapter 11 show them).
- `zValidator` reads that part of the request and checks it with the schema.
- The third argument is a **hook** that runs after checking. If the data is invalid, we throw a `ValidationError` containing a friendly list of problems. The handler never runs.
- If the data is valid, the *cleaned* data (trimmed strings, numbers converted, defaults applied) is available in the handler through `c.req.valid(target)`.

You can stack several validators on one route; they run left to right:

```ts
taskRoutes.patch('/:id', validate('param', idParamSchema), validate('json', updateTaskSchema), (c) =>
{ ... });
```

Authentication and authorization middleware

Three more of our own middleware functions live in `src/middleware/auth.ts`:

- `requireAuth` reads the `session` cookie, looks it up in the database and, if valid, stores the user with `c.set('user', user)`. Otherwise it throws a `401` error and the handler never runs.
- `requireRole('admin')` is a *factory* (like `requestLogger`) that creates middleware checking `c.get('user').role`, answering `403` otherwise.
- `injectAuthSettings(settings)` stores the cookie settings and login throttle on the context.

Route groups apply them with their own `use`:

```ts
taskRoutes.use('*', requireAuth);                          // src/routes/task.routes.ts
adminRoutes.use('*', requireAuth, requireRole('admin'));   // src/routes/admin.routes.ts
authRoutes.get('/me', requireAuth, (c) => ...);            // on a single route
```

The order matters: `requireRole` reads `c.get('user')`, which only exists after `requireAuth` ran. Chapter 20 explains all of this in depth.

Built-in middleware we use

```ts
app.use('*', secureHeaders({ contentSecurityPolicy: { defaultSrc: ["'self'"], ... } }));
app.use('/api/*', csrf());
app.use('/api/*', bodyLimit({ maxSize: 100 * 1024, onError: () => { throw new AppError(...413...) }
}));
app.use('/*', serveStatic({ root: './public' }));
```

- `secureHeaders` (from `hono/secure-headers`) adds headers that make browsers stricter. Chapter 14 explains them.
- `csrf` (from `hono/csrf`) refuses POST/PATCH/DELETE requests that have no body or a form body *and* were started by another website (checked with the `Origin` and `Sec-Fetch-Site` headers). Chapter 20 explains why.
- `bodyLimit` (from `hono/body-limit`) rejects request bodies larger than 100 KB, so nobody can send a gigantic body to exhaust the server's memory.
- `serveStatic` (from `@hono/node-server/serve-static`) sends files from `public/`. A request for `/css/styles.css` returns `public/css/styles.css`, and `/` returns `public/index.html`. It is registered *after* the API routes, so API paths are handled first.

Error middleware

Hono has a special place for error handling: `app.onError(handler)`. Any error thrown anywhere — in middleware, validators, handlers or services — ends up there. Ours is in `src/middleware/error-handler.ts`:

TS

```ts
export const errorHandler: ErrorHandler = (err, c) => {
  // Errors we threw on purpose (validation, not found, conflict, ...).
  if (err instanceof AppError) {
    return c.json(
      { error: { code: err.code, message: err.message, details: err.details } },
      err.status,
    );
  }

  // Errors raised by Hono itself, e.g. a request body that is not valid JSON.
  if (err instanceof HTTPException) {
    return c.json(
      { error: { code: 'HTTP_ERROR', message: err.message } },
      err.status as ContentfulStatusCode,
    );
  }

  // Anything else is a bug. Log the full details on the server, but never
  // send internal details (like stack traces) to the browser.
  console.error('Unexpected error:', err);
  return c.json(
    { error: { code: 'INTERNAL_ERROR', message: 'Something went wrong on the server.' } },
    500,
  );
};
```

`instanceof` asks "was this object made from this class (or a class that extends it)?" Because `NotFoundError`, `ValidationError` and `ConflictError` all extend `AppError`, the first `if` handles all of them using the status code stored inside the error.

`app.notFound(notFoundHandler)` handles requests that matched nothing and returns a JSON 404.

Why one central error handler?

- Every error response has the **same shape**, so the frontend handles them all the same way.
- Services can simply `throw new NotFoundError(...)` without knowing anything about HTTP responses.
- Unexpected errors are logged in **one** place and never leak internal details to users.

Try it yourself

1. Write a middleware in `src/app.ts` that adds a header to every response:

   ```ts
   app.use('*', async (c, next) => {
     await next();
     c.header('X-Powered-By-Student', 'yes');
   });
   ```

   Restart, open DevTools → Network, click any request and look for your header.
2. Move your middleware *above* `requestLogger`. Does the logged time change? Why?
3. Temporarily add `throw new Error('boom')` at the top of the `/api/health` handler. Call it. What does the browser receive? What does the terminal print? Remove it afterwards.

Chapter 7 — Understanding the Database

What is a database?

A **database** is an organised place to store data so it survives when programs stop, and so it can be searched quickly. Without a database, every task would vanish the moment the server restarts, because normal variables live only in memory.

Why do we need one?

- **Persistence** — data is saved on disk.
- **Querying** — "give me all pending high-priority tasks sorted by due date" in one instruction.
- **Rules** — the database itself can refuse bad data (a missing title, a category that does not exist).
- **Safety** — databases make sure a change is either fully saved or not saved at all.

What is SQLite?

SQLite is a database that stores everything in a **single file** — ours is `data/todo.db`. There is no separate database server to install or start. It is the most widely used database in the world (it is inside every phone and web browser).

SQLite is perfect for learning and for small-to-medium applications. Chapter 17 discusses when you might move to a database server such as PostgreSQL.

We talk to SQLite through the **better-sqlite3** driver, a package that knows how to read and write SQLite files from Node.js.

What is SQL?

SQL (Structured Query Language, pronounced "S-Q-L" or "sequel") is the language databases understand:

```sql
SELECT title, due_date FROM tasks WHERE status = 'pending' ORDER BY due_date ASC;
INSERT INTO tasks (title, created_at, updated_at) VALUES ('Buy milk', '2026-11-01T09:00:00Z', '2026-11-01T09:00:00Z');
UPDATE tasks SET status = 'completed' WHERE id = 7;
DELETE FROM tasks WHERE id = 7;
```

The four main commands — `SELECT`, `INSERT`, `UPDATE`, `DELETE` — match the CRUD operations from Chapter 1.

Tables, rows and columns

A database contains **tables**. A table is like a spreadsheet:

- each **column** is one piece of information with a fixed type (title, status, due date);
- each **row** is one record (one task).

Our `tasks` table might look like this:

id	title	status	priority	category_id	due_date	created_at
1	Read the Hono manual	in_progress	high	3	2026-11-01	2026-11-01T09:36:23Z
2	Prepare weekly report	pending	high	1	2026-10-30	2026-11-01T09:36:23Z
6	Water the plants	pending	low	NULL	NULL	2026-11-01T09:36:23Z

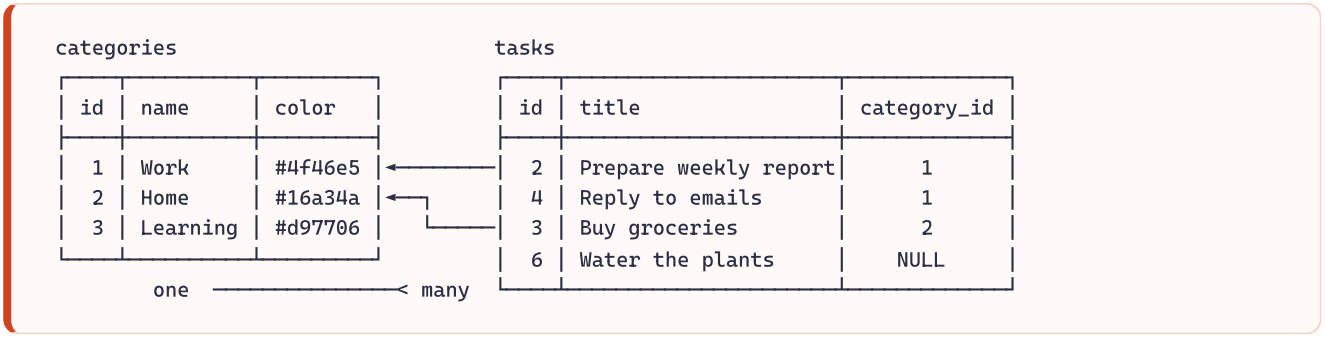
`NULL` means "no value".

What is a primary key?

A **primary key** is a column whose value is unique for every row, used to identify that row. Ours is `id`, an integer that SQLite assigns automatically (1, 2, 3, …) thanks to `AUTOINCREMENT`. That is the number in URLs like `/api/tasks/7`.

What is a foreign key? What are relationships?

A **foreign key** is a column that points at the primary key of a row in *another* table. `tasks.category_id` points at `categories.id`. This creates a **relationship**: *one category has many tasks; each task belongs to zero or one category*.

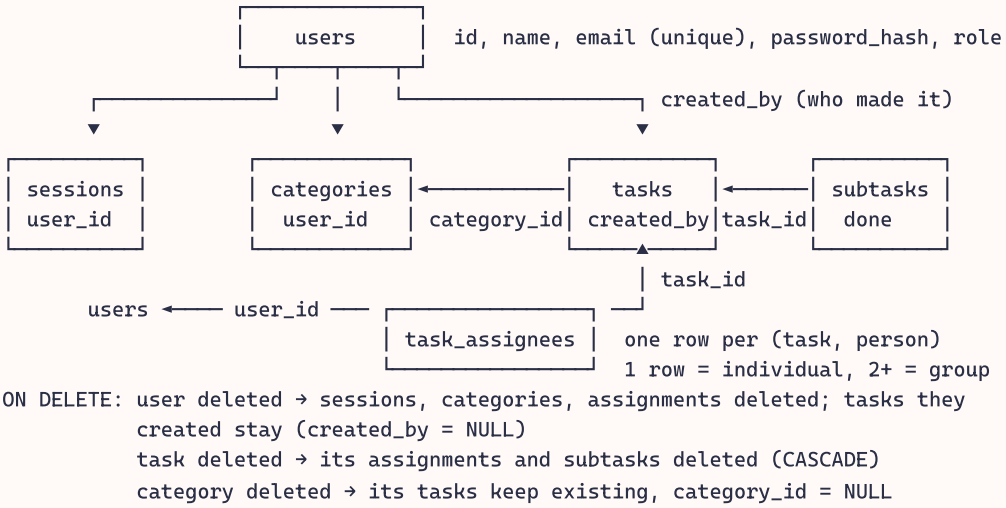


The foreign key has a rule: `ON DELETE SET NULL`. When a category is deleted, SQLite automatically sets `category_id` to `NULL` in its tasks. The tasks are kept, just uncategorised. The database also *refuses* a `category_id` that does not exist.

> **WARNING**
> SQLite only enforces foreign keys when you switch them on for each connection. That is why `src/db/index.ts` runs `sqlite.pragma('foreign_keys = ON')`. Forgetting this is a classic SQLite mistake.

All six tables

Since accounts (Development Entry #3) and teamwork (Entry #6) were added, the database has six tables:



`task_assignees` is a **link table** (also called a *join table*). It creates a **many-to-many relationship**: a task can have many people, and a person can have many tasks. Each row simply says "task 7 is assigned to user 3". Chapter 21 explains it in detail.

The actual database schema

The **schema** is the description of all tables and columns. We write it in TypeScript in `src/db/schema.ts`. Here are the task-related tables (the `users` and `sessions` tables are shown and explained in Chapter 20):

```ts
export const TASK_STATUSES = ['pending', 'in_progress', 'completed'] as const;
export const TASK_PRIORITIES = ['low', 'medium', 'high'] as const;

export const categories = sqliteTable(
  'categories',
  {
    id: integer('id').primaryKey({ autoIncrement: true }),
    userId: integer('user_id').references(() => users.id, { onDelete: 'cascade' }),
    name: text('name').notNull(),
    color: text('color').notNull().default('#6366f1'),
    createdAt: text('created_at').notNull(),
  },
```

```
  // Category names are unique per user: two users may both have "Work".
  (table) => [uniqueIndex('categories_user_name_unique').on(table.userId, table.name)],
);

export const tasks = sqliteTable(
  'tasks',
  {
    id: integer('id').primaryKey({ autoIncrement: true }),
    // The user who created the task. Only administrators are shown this.
    createdById: integer('created_by').references(() => users.id, { onDelete: 'set null' }),
    title: text('title').notNull(),
    description: text('description').notNull().default(''),
    status: text('status', { enum: TASK_STATUSES }).notNull().default('pending'),
    priority: text('priority', { enum: TASK_PRIORITIES }).notNull().default('medium'),
    categoryId: integer('category_id').references(() => categories.id, { onDelete: 'set null' }),
    dueDate: text('due_date'),
    createdAt: text('created_at').notNull(),
    updatedAt: text('updated_at').notNull(),
  },
  (table) => [
    index('tasks_created_by_idx').on(table.createdById),
    index('tasks_status_idx').on(table.status),
    index('tasks_due_date_idx').on(table.dueDate),
    index('tasks_category_idx').on(table.categoryId),
  ],
);

export const taskAssignees = sqliteTable(
  'task_assignees',
  {
    taskId: integer('task_id').notNull().references(() => tasks.id, { onDelete: 'cascade' }),
    userId: integer('user_id').notNull().references(() => users.id, { onDelete: 'cascade' }),
    assignedAt: text('assigned_at').notNull(),
  },
  (table) => [
    // The pair (task, user) is the primary key: nobody is assigned twice.
    primaryKey({ columns: [table.taskId, table.userId] }),
    index('task_assignees_user_idx').on(table.userId),
  ],
);

export const subtasks = sqliteTable(
  'subtasks',
  {
    id: integer('id').primaryKey({ autoIncrement: true }),
    taskId: integer('task_id').notNull().references(() => tasks.id, { onDelete: 'cascade' }),
    title: text('title').notNull(),
    // SQLite has no true/false type; Drizzle stores booleans as 0/1.
    done: integer('done', { mode: 'boolean' }).notNull().default(false),
    createdAt: text('created_at').notNull(),
  },
  (table) => [index('subtasks_task_idx').on(table.taskId)],
);
```

A **composite primary key** — `primaryKey({ columns: [taskId, userId] })` — uses *two* columns together as the unique identity of a row. The same person can never be assigned to the same task twice.

> **NOTE**
>
> **Why is `categories.user_id` allowed to be `NULL`?** Categories created *before* accounts existed have no owner. Adding a `NOT NULL` column to a table that already has rows would fail. So the column allows `NULL`, the **first account to register claims** all ownerless rows, and every query filters by owner — so ownerless rows are invisible to everybody. The same happens to tasks that have no row in `task_assignees`.

A **unique index on two columns** (`user_id`, `name`) means the *combination* must be unique: Alice and Bob can each have a "Work" category, but Alice cannot have two.

How to read a column definition, for example `title: text('title').notNull()`:

- `title:` — the name we use in **TypeScript** (`task.title`).
- `text('title')` — the column's type and its name in the **database**.
- `.notNull()` — the column must always have a value.
- `.default('')` — the value used when none is given.
- `.unique()` — no two rows may have the same value.
- `.references(() => categories.id, ...)` — this is a foreign key.

Notice `categoryId` in TypeScript becomes `category_id` in SQL. JavaScript usually uses *camelCase*; SQL usually uses *snake_case*. Drizzle translates for us.

Indexes are like the index at the back of a book: they let the database find rows by status, due date or category without reading every row.

Design decisions

Decision	Reason
Dates stored as text `YYYY-MM-DD`	SQLite has no date type. This format sorts correctly as text, and compares correctly with `<` and `>`.
Timestamps stored as ISO 8601 text (`2026-11-01T09:30:00.000Z`)	Readable, sortable, includes time zone (`Z` = UTC).
Status and priority stored as text with a fixed list	Readable in the database; Zod and TypeScript enforce the allowed values.
Category name is unique **per user** (`user_id` + `name`)	Two "Work" categories for one person would be confusing, but different people may use the same names. The service also checks ignoring letter case.
Deleting a category keeps its tasks	Losing tasks by accident would be worse than leaving them uncategorised.
Categories have a `user_id`; tasks have assignees in `task_assignees`	Authorization: each query can filter by owner / assignee (Chapters 20, 21).
Subtasks are a separate table, not tasks inside tasks	A simple checklist is what's needed; a separate table keeps `tasks` simple and lets subtasks be deleted with their task (`ON DELETE CASCADE`).

What is Drizzle ORM?

An **ORM** (Object-Relational Mapper) is a library that lets you work with database tables using your programming language instead of writing raw SQL strings. **Drizzle** is a modern, lightweight TypeScript ORM. Its queries look very similar to SQL, so what you learn transfers directly:

```ts
// Drizzle (src/services/category.service.ts)
db.select().from(categories).where(and(eq(categories.id, id), eq(categories.userId, userId))).get();
```

```sql
-- The SQL that Drizzle generates and sends to SQLite
SELECT id, user_id, name, color, created_at FROM categories WHERE (id = ? AND user_id = ?);
```

The `?` is a **placeholder**. The real value (`id`) is sent separately from the SQL text. This is called a *parameterised query* and it is what makes the app safe from SQL injection (Chapter 14).

Benefits of Drizzle:

- **Type safety** — misspell a column and TypeScript tells you immediately.
- **Inferred types** — `Task` and `Category` types come straight from the schema.
- **Safe by default** — values are always sent as parameters.
- **Escape hatch** — when needed, the `sql` template lets you write raw SQL safely, as we do for the dashboard counts.

Common Drizzle building blocks used in this project:

Drizzle	SQL	Meaning
`eq(a, b)`	`a = b`	equal
`ne(a, b)`	`a != b`	not equal
`lt(a, b)` / `gt(a, b)`	`a < b` / `a > b`	less / greater than
`like(a, '%x%')`	`a LIKE '%x%'`	contains text
`isNull(a)` / `isNotNull(a)`	`a IS NULL`	has no value / has a value
`and(...)` / `or(...)`	`AND` / `OR`	combine conditions
`asc(a)` / `desc(a)`	`ORDER BY a ASC/DESC`	sort direction
`count(a)`	`COUNT(a)`	count rows
`.leftJoin(t, cond)`	`LEFT JOIN t ON cond`	combine rows from two tables
`.innerJoin(t, cond)`	`INNER JOIN t ON cond`	combine rows, keeping only rows that match in both tables
`db.transaction((tx) => …)`	`BEGIN … COMMIT`	several changes that succeed or fail together
`inArray(col, [1, 2])`	`col IN (1, 2)`	value is one of a list

Drizzle	SQL	Meaning
`inArray(col, db.select(...))`	`col IN (SELECT …)`	value is in the result of a **subquery** (a query inside a query)
`gte(a, b)` / `lte(a, b)`	`a >= b` / `a <= b`	at least / at most (used for date ranges)
`.get()` / `.all()` / `.run()`	—	return one row / all rows / nothing
`.returning()`	`RETURNING *`	give back the inserted/updated row

What are migrations?

A **migration** is a file of SQL instructions that changes the database structure from one version to the next ("create the tasks table", later "add a `notes` column"). Migrations are numbered and applied in order, so every copy of the database — yours, a teammate's, the production server's — ends up with the same structure.

Our workflow:

```
1. You edit src/db/schema.ts (e.g. add a column)
        ↓
2. pnpm db:generate
   drizzle-kit compares the schema with the last snapshot
   and writes  src/db/migrations/0001_<name>.sql
        ↓
3. Restart the server (or pnpm db:migrate)
   runMigrations(db) applies every migration not yet applied
        ↓
4. The database file now has the new structure
```

Our first migration, `src/db/migrations/0000_init.sql`, was generated from the schema above:

SQL

```
CREATE TABLE `categories` (
	`id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
	`name` text NOT NULL,
	`color` text DEFAULT '#6366f1' NOT NULL,
	`created_at` text NOT NULL
);
--> statement-breakpoint
CREATE UNIQUE INDEX `categories_name_unique` ON `categories` (`name`);--> statement-breakpoint
CREATE TABLE `tasks` (
	`id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
	`title` text NOT NULL,
	`description` text DEFAULT '' NOT NULL,
	`status` text DEFAULT 'pending' NOT NULL,
	`priority` text DEFAULT 'medium' NOT NULL,
	`category_id` integer,
	`due_date` text,
	`created_at` text NOT NULL,
	`updated_at` text NOT NULL,
	FOREIGN KEY (`category_id`) REFERENCES `categories`(`id`) ON UPDATE no action ON DELETE set
null
```

```
);
--> statement-breakpoint
CREATE INDEX `tasks_status_idx` ON `tasks` (`status`);--> statement-breakpoint
CREATE INDEX `tasks_due_date_idx` ON `tasks` (`due_date`);--> statement-breakpoint
CREATE INDEX `tasks_category_idx` ON `tasks` (`category_id`);
```

The `meta/` folder next to it contains `_journal.json` (the list of migrations) and a snapshot that drizzle-kit uses to work out what changed next time. Drizzle records which migrations have been applied in a special table inside the database (`__drizzle_migrations`), so each migration runs only once.

> **IMPORTANT**
> Never edit a migration that has already been applied to a database. Always change `schema.ts` and generate a **new** migration.

The second migration: 0001_add_users_and_sessions.sql

When accounts were added, `pnpm exec drizzle-kit generate --name add_users_and_sessions` created this migration:

SQL

```
CREATE TABLE `sessions` ( `id` text PRIMARY KEY NOT NULL, `user_id` integer NOT NULL,
  `expires_at` text NOT NULL, `created_at` text NOT NULL,
  FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE cascade );
CREATE INDEX `sessions_user_idx` ON `sessions` (`user_id`);
CREATE TABLE `users` ( `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL, `name` text NOT NULL,
  `email` text NOT NULL, `password_hash` text NOT NULL, `role` text DEFAULT 'user' NOT NULL,
  `created_at` text NOT NULL );
CREATE UNIQUE INDEX `users_email_unique` ON `users` (`email`);
DROP INDEX `categories_name_unique`;
ALTER TABLE `categories` ADD `user_id` integer REFERENCES users(id) ON DELETE cascade;
CREATE UNIQUE INDEX `categories_user_name_unique` ON `categories` (`user_id`,`name`);
ALTER TABLE `tasks` ADD `user_id` integer REFERENCES users(id) ON DELETE cascade;
CREATE INDEX `tasks_user_idx` ON `tasks` (`user_id`);
```

(Shown slightly condensed; the real file has one statement per `--> statement-breakpoint`.)

> **NOTE**
> **One hand edit was made.** drizzle-kit wrote the two `ALTER TABLE … ADD user_id … REFERENCES users(id)` lines *without* `ON DELETE cascade` (a limitation when SQLite adds a column to an existing table). Because this migration had **not yet been applied anywhere**, it was safe to add `ON DELETE cascade` by hand so the database matches the schema. The rule "never edit a migration" applies to migrations that have already run.

`ALTER TABLE … ADD` keeps all existing rows; the new `user_id` column is `NULL` in them until the first account claims them.

Migrations 0002 and 0003: from one owner to many assignees

Development Entry #6 replaced `tasks.user_id` (one owner) with the `task_assignees` table (any number of people) plus `tasks.created_by`. Existing data had to be **moved**, not lost. It was done in two migrations:

0002_assignees_and_subtasks.sql — creates `subtasks` and `task_assignees`, adds `tasks.created_by`, and then (hand-written at the end) copies the data:

```sql
-- Data migration (hand-written): every existing owner becomes the task's creator and only assignee.
INSERT INTO `task_assignees` (`task_id`, `user_id`, `assigned_at`) SELECT `id`, `user_id`,
`created_at` FROM `tasks` WHERE `user_id` IS NOT NULL;
UPDATE `tasks` SET `created_by` = `user_id` WHERE `user_id` IS NOT NULL;
```

`INSERT INTO … SELECT …` inserts one new row for every row the `SELECT` returns — here, one assignment per owned task.

0003_drop_task_owner_column.sql — removes the old `user_id` column. SQLite cannot drop a column that has a foreign key, so drizzle-kit **rebuilds** the table: create `__new_tasks` without the column, copy every row, `DROP TABLE tasks`, rename `__new_tasks` to `tasks`.

> **NOTE**
>
> **Why two migrations?** When one schema change both removes `user_id` and adds `created_by`, drizzle-kit stops and asks "is this a rename?" — an interactive question that cannot be answered in a non-interactive shell. Doing "add" and "remove" in separate steps avoids the question, and also makes the order explicit: copy the data first, delete the column second.

WARNING

A data-loss bug that was caught before it reached your data. When migration 0003 was first tested on a *copy* of the real database, all 7 assignments silently disappeared. The cause: `DROP TABLE tasks` while foreign keys are switched on performs an implicit `DELETE FROM tasks`, which fires every `ON DELETE CASCADE` — wiping `task_assignees` and `subtasks`. The migration file *does* start with `PRAGMA foreign_keys=OFF`, but Drizzle runs all migrations inside a **transaction**, and SQLite **ignores** that PRAGMA inside a transaction.

The fix is in `runMigrations` (`src/db/index.ts`): switch foreign keys off *before* Drizzle opens its transaction, run SQLite's `PRAGMA foreign_key_check` afterwards to prove every reference is still valid, and switch them back on:

```ts
export function runMigrations(db: AppDatabase): void {
  const sqlite = db.$client;
  sqlite.pragma('foreign_keys = OFF');
  try {
    migrate(db, { migrationsFolder: MIGRATIONS_FOLDER });
    const problems = sqlite.pragma('foreign_key_check') as unknown[];
    if (problems.length > 0) {
      throw new Error(`Migration left ${problems.length} broken foreign key reference(s).`);
    }
  } finally {
    sqlite.pragma('foreign_keys = ON');
  }
}
```

`tests/migrations.test.ts` rebuilds a database at the *old* version, runs the new migrations and checks nothing was lost. It was verified to **fail** when the fix is removed. Lessons: always test migrations on a copy of real data, back up before migrating (a backup of your database was saved in `data/backups/`), and write a regression test for every bug.

Both migrations also needed the same hand edit as 0001: drizzle-kit leaves out the `ON DELETE` rule when it adds a foreign-key column with `ALTER TABLE … ADD`, so `ON DELETE set null` was added to `created_by`.

How does database persistence work?

```
createTask(db, viewer, input)                (src/services/task.service.ts)
       ↓
db.insert(tasks).values({...}).returning(...).get()      (Drizzle)
db.insert(taskAssignees).values([...]).run()             (same transaction)
       ↓
INSERT INTO tasks (...) VALUES (?, ?, ...) RETURNING id   (SQL + parameters)
       ↓
better-sqlite3 → SQLite engine
       ↓
data/todo.db  (+ todo.db-wal while the server runs)
```

We enable **WAL mode** (`journal_mode = WAL`, "Write-Ahead Log"). New changes are first appended to a side file, `todo.db-wal`, and merged into `todo.db` later. This makes writing faster and lets reads happen while writing. Never delete the `-wal` file while the server is running.

Try it yourself

1. Install the free "SQLite Viewer" extension in VS Code (or "DB Browser for SQLite") and open `data/todo.db`. Look at both tables.
2. Create a task in the app and refresh the viewer. Find the new row.
3. Delete a category in the app and check that its tasks now have `category_id = NULL`.

Chapter 8 — Understanding the Frontend

HTML

HTML describes the *structure* of a page using **elements** written as tags:

```HTML
<button type="button" class="btn btn-primary" id="open-new-task">+ New task</button>
```

- `<button>` … `</button>` — the element (a button).
- `type`, `class`, `id` — **attributes** giving extra information.
- `id="open-new-task"` — a unique name so JavaScript can find this element.
- `class="btn btn-primary"` — names used by CSS for styling.

Our page `public/index.html` has two "screens", and JavaScript shows exactly one of them by toggling the `hidden` attribute:

- `#auth-screen` — the **login** form and the **register** form (only one visible at a time);
- `#app-screen` — the top bar (tabs, the user's name, **Log out**) and four **views**, of which one is visible at a time: `#view-dashboard` (cards, progress, "Overdue" and "Due in the next 7 days" lists), `#view-tasks` (filters and the task list with its loading/error/empty states), `#view-calendar` (month grid) and `#view-admin` (users and all tasks — administrators only).

Outside both screens are three `<dialog>` pop-ups (task form — with an "Assign to" section only admins see —, category manager, confirmation) and a container for notifications ("toasts").

Views and the address bar (hash routing)

The tabs are ordinary links: `<a href="#/calendar">`. The part of an address after `#` is called the **hash** (or fragment). Changing it does **not** reload the page or contact the server; the browser just fires a `hashchange` event. `app.js` listens for it and shows the matching view:

```JS
function viewFromHash() {
  const name = location.hash.replace(/^#\/?/, '');
  if (!VIEWS.includes(name)) return 'dashboard';
  if (name === 'admin' && !isAdmin()) return 'dashboard'; // only admins may open it
  return name;
}

function showView(name) {
  state.view = name;
  for (const view of VIEWS) $(`view-${view}`).hidden = view !== name;
  renderActiveTab(name);
```

```
  loadCurrentView();
}
```

This technique is called **hash routing**. Its advantages: each view has its own address (you can bookmark `http://localhost:3000/#/calendar` or press the browser's Back button), and no server changes are needed. After every change (create, edit, tick…), `loadCurrentView()` reloads only the data of the visible view. Chapter 21 explains each view.

The `<dialog>` element is a built-in HTML pop-up. JavaScript opens it with `dialog.showModal()` and closes it with `dialog.close()`.

CSS

CSS describes the *appearance*. A CSS **rule** has a **selector** (which elements) and **declarations** (how they look):

```css
.btn-primary { background: var(--primary); color: #fff; }
```

Techniques used in `public/css/styles.css`:

- **CSS variables** — `--primary: #4f46e5;` defined once in `:root`, used everywhere with `var(--primary)`. Change the colour in one place and the whole app updates.
- **Flexbox** (`display: flex`) — lines things up in a row or column (the top bar, the task card).
- **Grid** (`display: grid; grid-template-columns: repeat(6, 1fr)`) — the six dashboard cards.
- **Media queries** — `@media (max-width: 560px) { ... }` changes the layout on phones (two dashboard cards per row, stacked form fields).
- **State classes** — `.task.is-completed .task-title { text-decoration: line-through; }` styles completed tasks.

The Aeto theme: colours from the logo

Since Development Entry #8 the app is called **Aeto To-Do** and its colours come from the Aeto logo. Thanks to CSS variables, the whole theme is defined in one block at the top of `styles.css`:

```css
:root {
  --brand-ink: #383745;          /* charcoal letters of the logo  */
  --brand-orange: #ef5a35;       /* end of the orange swoosh      */
  --brand-orange-light: #f58c5f; /* start of the orange swoosh    */
  --brand-cream: #f7f3ef;        /* letters of the logo on dark   */
  --brand-gradient: linear-gradient(90deg, var(--brand-orange-light), var(--brand-orange));
  --text: var(--brand-ink);
  --primary: #cf4520;            /* buttons: deeper orange, readable white text */
  ...
}
```

- The **top bar** and the **login screen** are charcoal with cream text — like the logo's version for dark backgrounds; a 3 px orange gradient line (`border-image: var(--brand-gradient) 1`) runs under the top bar.
- `public/img/aeto-mark.svg` is a simplified orange **swoosh** from the logo, drawn as an SVG path with a gradient; the favicon uses the same swoosh on a charcoal square.
- **Colour contrast:** text must be readable for people with weaker eyesight. The guideline (WCAG AA) asks for a contrast ratio of at least **4.5 : 1** for normal text. White on the logo orange `#ef5a35` reaches only about 3.2 : 1, so buttons use the slightly deeper `#cf4520` (about 4.6 : 1). The bright logo orange is used for decoration (lines, the swoosh, the progress bar) where no text sits on top of it.
- Status colours (red overdue, green done, …) keep their usual meaning, because changing them would confuse users.

JavaScript

JavaScript gives the page *behaviour*. Our frontend JavaScript is split into three **modules**:

File	Responsibility
`public/js/api.js`	Talks to the server. Every `fetch()` lives here.
`public/js/ui.js`	Draws things on the screen (task cards, dashboard numbers, toasts).
`public/js/app.js`	The "brain": remembers state, reacts to clicks, calls `api.js`, then `ui.js`.

The page loads them with `<script type="module" src="/js/app.js"></script>` . `type="module"` allows `import` / `export` between the files and automatically waits until the HTML is ready.

The DOM

When the browser reads HTML, it builds a tree of objects in memory called the **DOM** (Document Object Model). JavaScript changes the page by changing the DOM:

```JS
const list = document.getElementById('task-list');          // find an element
const item = document.createElement('li');                  // create a new element
item.textContent = 'Buy milk';                              // set its text
list.append(item);                                          // put it on the page
element.hidden = true;                                      // hide it
```

Our small helper `el()` in `ui.js` creates an element, sets its class and text in one line:

```JS
export function el(tag, className, text) {
  const element = document.createElement(tag);
  if (className) element.className = className;
  if (text !== undefined) element.textContent = text;
```

```
  return element;
}
```

Events and event listeners

An **event** is something that happens: a click, a key press, a form submission, a change in a dropdown. An **event listener** is a function that runs when the event happens:

```JS
// public/js/app.js — inside bindEvents()
$('open-new-task').addEventListener('click', openNewTask);
taskForm.addEventListener('submit', submitTaskForm);
$('filter-status').addEventListener('change', (e) => setFilter('status', e.target.value));
```

When a form is submitted, the browser's default behaviour is to reload the page. We stop that with `event.preventDefault()` because we want to send the data with JavaScript instead.

Debouncing: the search box listens to every key press, but we wait until the user has stopped typing for 300 ms before searching, so typing "groceries" sends one request instead of nine:

```JS
let searchTimer;
function onSearchInput(event) {
  clearTimeout(searchTimer);
  searchTimer = setTimeout(() => setFilter('search', event.target.value.trim()), 300);
}
```

The Fetch API

`fetch()` is the browser's built-in function for sending HTTP requests. Our wrapper in `public/js/api.js`:

```JS
async function request(path, { method = 'GET', body } = {}) {
  let response;
  try {
    response = await fetch(path, {
      method,
      headers: body === undefined ? {} : { 'Content-Type': 'application/json' },
      body: body === undefined ? undefined : JSON.stringify(body),
    });
  } catch {
    // fetch() only throws when the server cannot be reached at all.
    throw new ApiError('Cannot reach the server. Is it running?', 0);
  }

  // 204 No Content has no body to read.
  if (response.status === 204) return null;

  const payload = await response.json().catch(() => null);
```

```
  if (!response.ok) {
    const error = payload?.error;
    // A 401 from a normal API call means our session is gone. (Auth routes are
    // excluded: a 401 from /login just means "wrong password".)
    if (response.status === 401 && !path.startsWith('/api/auth/') && unauthorizedHandler) {
      unauthorizedHandler();
    }
    throw new ApiError(error?.message ?? `Request failed (${response.status})`, response.status,
error?.details);
  }
  return payload.data;
}
```

Step by step:

1. Send the request. If there is a body, convert it to JSON text and set the `Content-Type` header.
2. If `fetch` itself fails, the server is unreachable → throw a friendly error.
3. A `204` response has no body → return `null`.
4. Read and parse the JSON body.
5. `response.ok` is `true` for status 200–299. Otherwise, throw an `ApiError` carrying the server's message and validation details.
6. On success, return just the `data` part of the envelope.

Important: `fetch` does **not** throw for 400 or 500 responses — it only throws for network failures. That is why we check `response.ok` ourselves.

`?.` (optional chaining) means "if the left side is null/undefined, stop and give undefined instead of crashing". `??` (nullish coalescing) means "use the right side if the left side is null/undefined".

Every endpoint then gets a one-line function (the auth ones are `getMe`, `login`, `register`, `logout`, plus `getUsers` for admins):

JS

```
export const login = (email, password) => request('/api/auth/login', { method: 'POST', body: { email,
password } });
export const getTasks = (filters) => request(`/api/tasks${toQueryString({ ...filters, today:
localToday() })}`);
export const createTask = (task) => request('/api/tasks', { method: 'POST', body: task });
export const deleteTask = (id) => request(`/api/tasks/${id}`, { method: 'DELETE' });
```

JSON parsing

`response.json()` reads the response body and turns the JSON text into a JavaScript object. It returns a Promise, so we `await` it. If the body is not valid JSON, it throws — we catch that with `.catch(() => null)`.

Async requests and updating the interface

The pattern used for every user action:

```
user clicks / types
       ↓
app.js handler (async)
       ↓
await api.someCall(...)        ← waits for the server; page stays responsive
       ↓ success                    ↓ failure (ApiError)
showToast('…', 'success')       showToast(error.message, 'error')
await refresh()                 or showFormErrors(...)
       ↓
ui.js redraws list + dashboard
```

We redraw the list by *replacing* all its children with freshly built elements:

```JS
list.replaceChildren(...tasks.map((task) => createTaskItem(task, handlers)));
```

Logging in from the frontend

Notice that **JavaScript never touches the login token**. The server puts it in an `HttpOnly` cookie, and the browser attaches it to every `fetch` to our own server automatically. The frontend only needs to know *whether* it is logged in:

```JS
// public/js/app.js
async function start() {
  bindEvents();
  // If any later request says "401", the session has ended: go back to login.
  api.onUnauthorized(() => {
    if (state.user) showToast('Your session has ended. Please log in again.', 'error');
    showAuthScreen('login');
  });

  try {
    showApp(await api.getMe());
  } catch (error) {
    showAuthScreen('login');
    if (error.status !== 401) showToast(error.message, 'error');
  }
}
```

- On page load, `GET /api/auth/me` asks "who am I?". `200` → `showApp(user)`; `401` → `showAuthScreen('login')`.
- `onUnauthorized(handler)` in `api.js` registers a function that `request()` calls whenever a normal API call answers `401` — for example when the session expired while the page was open.

- `submitLogin` / `submitRegister` call `api.login(...)` / `api.register(...)`, then `showApp(user)`. Errors (wrong password, too short password, email taken) are shown inside the form with `showFormErrors`.
- `showApp(user)` resets the dashboard and the filters before loading, so nothing from a previous user is ever shown.
- `logout()` calls `POST /api/auth/logout` and shows the login screen.
- The **Admin** tab is only shown to admins (`renderCurrentUser` in `ui.js`), and `viewFromHash` sends non-admins who type `#/admin` back to the dashboard. Hiding a tab is *not* security — the server's `requireRole('admin')` is what actually protects the data.

Loading states, error states and empty states

The task list area can be in exactly one of four states, controlled by `showListState()` in `ui.js`:

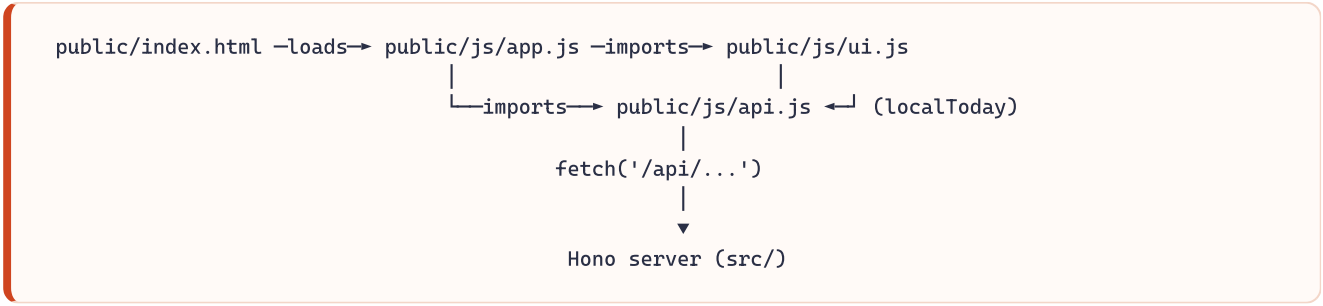
State	When	What the user sees
`loading`	A request is in progress	A spinner and "Loading tasks…"
`error`	The request failed	The error message and a **Try again** button
`empty`	Success, but zero tasks	"No tasks yet" or, if filters are active, "No matching tasks"
`list`	Success with tasks	The task cards

JS

```
export function showListState(state, message) {
  document.getElementById('state-loading').hidden = state !== 'loading';
  document.getElementById('state-error').hidden = state !== 'error';
  document.getElementById('state-empty').hidden = state !== 'empty';
  document.getElementById('task-list').hidden = state !== 'list';
  ...
}
```

Also: the form's Save button is **disabled while saving** (so a double click does not create two tasks), and **notifications** confirm every action.

How the frontend communicates with Hono



The browser loads the frontend files from the **same server** as the API (same "origin": `http://localhost:3000`). That is why the frontend can use short paths like `/api/tasks`, and why no CORS configuration is needed.

> **NOTE**
>
> **Why send `today` from the browser?** "Overdue" and "due today" depend on *your* calendar day. If the server were in a different time zone, its "today" might differ from yours. So the browser sends its local date (`?today=2026-11-01`), and the server uses it. If `today` is missing, the server falls back to its own local date.

Try it yourself

1. In `public/css/styles.css`, change `--primary: #4f46e5;` to `--primary: #e11d48;` and reload the page.
2. Open DevTools → Console and type `document.querySelectorAll('.task').length`. That is the number of task cards in the DOM.
3. Stop the server, then click + **New task** and try to save. Which message do you see, and which line of `api.js` produced it?

Chapter 9 — Complete Application Architecture

The big picture



The layers and their single responsibilities

Layer	Folder	Knows about HTTP?	Knows about SQL?	Responsibility
Frontend	`public/`	Yes (fetch)	No	Show data, collect input
Entry point	`src/index.ts`	Starts server	Opens DB	Wire everything together
App	`src/app.ts`	Yes	No	Register middleware and routes
Middleware	`src/middleware/`	Yes	No	Cross-cutting concerns
Routes	`src/routes/`	Yes	No	Translate HTTP ↔ service calls
Validators	`src/validators/`	No	No	Describe valid input
Services	`src/services/`	No	Yes (via Drizzle)	Business rules + queries

Layer	Folder	Knows about HTTP?	Knows about SQL?	Responsibility
Database	`src/db/`	No	Yes	Schema, connection, migrations
Lib	`src/lib/`	No	No	Shared helpers (errors, dates)

Why separate layers?

- **Readability** — when you look for "how is the overdue count calculated?", you know it is in a service, not in a route.
- **Testing** — services can be tested directly (our dashboard test does this), and routes can be tested without a browser.
- **Changing things safely** — switching from SQLite to PostgreSQL would mainly affect `src/db` and services, not routes or the frontend.

Every folder and file

```
aeto-to-do/
├── src/                          BACKEND SOURCE CODE
│   ├── index.ts                  Starts the server (entry point)
│   ├── app.ts                    Builds the Hono application
│   ├── config.ts                 Reads + validates environment variables
│   ├── types.ts                  AppEnv + AuthSettings: types for context variables
│   ├── db/
│   │   ├── schema.ts             Tables: users, sessions, categories, tasks, task_assignees,
subtasks
│   │   ├── index.ts              Open DB, run migrations (foreign keys off while migrating), close
DB
│   │   └── migrations/           Generated SQL migration files
│   │       ├── 0000_init.sql
│   │       ├── 0001_add_users_and_sessions.sql
│   │       ├── 0002_assignees_and_subtasks.sql
│   │       ├── 0003_drop_task_owner_column.sql
│   │       └── meta/             drizzle-kit bookkeeping
│   ├── lib/
│   │   ├── errors.ts             AppError, ValidationError, NotFoundError, ConflictError
│   │   ├── dates.ts              nowIso(), todayLocal()
│   │   ├── password.ts           hashPassword(), verifyPassword() (scrypt)
│   │   └── login-throttle.ts     LoginThrottle: blocks repeated failed logins
│   ├── middleware/
│   │   ├── request-logger.ts     Logs each request
│   │   ├── database.ts           Puts db on the context
│   │   ├── auth.ts               requireAuth, requireRole, session cookie helpers
│   │   ├── validate.ts           Zod validation wrapper
│   │   └── error-handler.ts      onError + notFound handlers
│   ├── validators/
│   │   ├── common.ts             dateString, idParamSchema
│   │   ├── auth.validator.ts     register/login schemas
│   │   ├── task.validator.ts     create/update/list task schemas (+ assignees, admin list)
│   │   ├── subtask.validator.ts  create/update subtask schemas
│   │   ├── category.validator.ts create/update category schemas
│   │   └── dashboard.validator.ts dashboard query schema
│   ├── services/
│   │   ├── auth.service.ts       Accounts, passwords, sessions, user list
│   │   ├── task.service.ts       Tasks: access rules, assignees, search, filter, sort
│   │   ├── subtask.service.ts    Subtasks (checklists inside tasks)
│   │   ├── category.service.ts   Category CRUD + task counts (per user)
│   │   └── dashboard.service.ts  Dashboard statistics (tasks assigned to the user)
│   └── routes/
│       ├── auth.routes.ts        /api/auth endpoints (register, login, logout, me)
│       ├── task.routes.ts        /api/tasks endpoints (+ POST /:id/subtasks)
│       ├── subtask.routes.ts     /api/subtasks endpoints
│       ├── category.routes.ts    /api/categories endpoints
│       ├── dashboard.routes.ts   /api/dashboard endpoint
│       └── admin.routes.ts       /api/admin endpoints: users, all tasks (admins only)
├── public/                       FRONTEND (sent to the browser as-is)
│   ├── index.html
│   ├── favicon.svg
│   ├── css/styles.css
│   └── js/ api.js  ui.js  app.js
├── tests/                        AUTOMATED TESTS (Vitest)
│   ├── helpers.ts                createTestApp(): in-memory DB + cookie-aware clients
│   ├── auth.test.ts  authorization.test.ts  assignment.test.ts  subtasks.test.ts
│   ├── tasks.test.ts  categories.test.ts  dashboard.test.ts  app.test.ts  migrations.test.ts
├── scripts/                      COMMAND-LINE SCRIPTS
│   ├── migrate.ts                pnpm db:migrate
│   ├── seed.ts                   pnpm db:seed
```

```
│   ├── e2e-smoke.ts              pnpm test:e2e (browser test)
│   ├── build-docs.ts             pnpm docs:build
│   └── lib/browser.ts            Finds Chrome/Edge on this computer
├── docs/                         THIS MANUAL
│   ├── DOCUMENTATION-SOURCE.md   Source of truth (edit this)
│   ├── CHANGELOG.md              Every change, newest first
│   ├── LEARN-HONO-COMPLETE.html  Generated
│   └── LEARN-HONO-COMPLETE.pdf   Generated
├── data/                         SQLite database (created at runtime, not in git)
├── package.json  pnpm-lock.yaml  pnpm-workspace.yaml
├── tsconfig.json                 TypeScript settings
├── drizzle.config.ts             drizzle-kit settings
├── vitest.config.ts              Vitest settings
├── .env.example                  Example environment variables
├── .gitignore                    Files git must ignore
└── README.md                     Short project overview
```

Dependency direction

Arrows mean "imports from". Notice that nothing points *up*: services never import routes, the database never imports services.



Important architectural decisions

Decision	Why
`createApp()` separate from `index.ts`	Tests build the app with an in-memory DB and call `app.request()` without a network port.
Database passed in via middleware (`c.set('db')`)	Dependency injection: same code, different databases for dev and tests.
Services are plain functions taking `db` first	Simple to read and to test; no classes or containers needed.
Errors thrown as classes, handled centrally	Consistent JSON errors; services stay free of HTTP code.
Zod schemas in their own folder	One place describes valid input; types are inferred from it.
Vanilla JavaScript frontend, no build step	Fewer tools to learn; the browser runs the files directly.
Frontend served by the same Hono server	One process, one port, no CORS configuration.
Synchronous SQLite driver	Simplest code; extremely fast for a local file.

Decision	Why
Sessions in the database + HttpOnly cookie (not JWT)	Logout and expiry take effect immediately; JavaScript cannot read the token (Chapter 20).
Authorization checks in the **services** (`user_id` in every query)	Impossible to forget in one route: a service cannot even be called without a `userId`.
`requireAuth` applied per route group	Public routes (health, login, register) stay public; everything else is protected right where it is defined.
Assignees in a link table (`task_assignees`), not a list inside `tasks`	A database column holds one value; a many-to-many relationship needs its own table (Chapter 21).
Services receive the whole **viewer** (`{ id, role }`), not just an ID	Access rules depend on the role too (admins see everything).
Views switched with the URL hash, no frontend framework	Bookmarkable views and a working Back button with a few lines of plain JavaScript.

Try it yourself

Draw the architecture diagram on paper from memory. Then compare it with the one above. Which layer did you forget?

Chapter 10 — Complete Code Flow

This chapter follows real requests through real code, feature by feature.

Flow 1: Opening the page (initial load)

```
Browser requests GET /
       ↓
requestLogger → secureHeaders  (bodyLimit/injectDatabase skip: not /api/*)
       ↓
No API route matches → serveStatic finds public/index.html → 200
       ↓
Browser loads /css/styles.css, /js/app.js (+ ui.js, api.js), /favicon.svg
       ↓
app.js runs start():  bindEvents();  GET /api/auth/me  (cookie sent automatically)
       ↓ 401: no valid session              ↓ 200: { data: user }
showAuthScreen('login')               showApp(user): name in top bar,
(see Chapter 20, Flow A)              loadCategories();  refresh();
                                             ↓                 ↓
                                  GET /api/categories    GET /api/tasks?
sort=createdAt&order=desc&today=…
                                                        GET /api/dashboard?today=…
       ↓
ui.js fills dropdowns, task list and dashboard with THIS user's data
```

Flow 2: Creating a task

1. User action. The user clicks **+ New task**. `openNewTask()` resets the form and calls `taskDialog.showModal()`. The user fills in the form and clicks **Create task**.

2. Frontend JavaScript. The form's `submit` event runs `submitTaskForm` in `public/js/app.js`:

JS

```
async function submitTaskForm(event) {
  event.preventDefault(); // stop the browser from reloading the page
  const submitButton = $('task-submit');
  submitButton.disabled = true;
  taskFormErrors.hidden = true;

  try {
    const data = readTaskForm();
    if (state.editingTaskId === null) {
      await api.createTask(data);
      showToast('Task created', 'success');
    } else {
      await api.updateTask(state.editingTaskId, data);
      showToast('Task updated', 'success');
    }
    taskDialog.close();
    await Promise.all([refresh(), loadCategories()]);
  } catch (error) {
```

```
    showFormErrors(taskFormErrors, error);
  } finally {
    submitButton.disabled = false;
  }
}
```

`readTaskForm()` converts the form into the API's shape — for example an empty category becomes `null` and the category ID text becomes a number.

3. HTTP request. `api.createTask(data)` → `request('/api/tasks', { method: 'POST', body: data })` sends:

```
POST /api/tasks
Content-Type: application/json

{"title":"Write
report","description":"","status":"pending","priority":"high","categoryId":1,"dueDate":"2026-12-01"}
```

The browser also adds `Cookie: session=…` and `Origin: http://localhost:3000` by itself.

4. Middleware. `requestLogger` starts its timer → `secureHeaders` → `csrf` (a JSON body from our own page is fine) → `bodyLimit` checks the size → `injectDatabase` sets `c.set('db', db)` → `injectAuthSettings` sets `c.set('auth', …)`.

5. Hono route + authentication. `app.route('/api/tasks', taskRoutes)` forwards to the task routes. Their first middleware, `requireAuth`, finds the session for the cookie and stores the user (`c.set('user', user)`); without a valid session the request ends here with `401`.

6. Validation. `validate('json', createTaskSchema)` parses the JSON and checks it. It trims the title, applies defaults, and checks the date format. If invalid → `ValidationError` → error handler → `400`.

7. Handler.

TS
```ts
taskRoutes.post('/', validate('json', createTaskSchema), (c) => {
  const input = c.req.valid('json');
  const task = taskService.createTask(c.get('db'), c.get('user'), input);
  return c.json({ data: task }, 201); // 201 Created
});
```

8. Service. `createTask` in `src/services/task.service.ts`:

TS
```ts
export function createTask(db: AppDatabase, viewer: Viewer, input: CreateTaskInput): TaskView {
  const { assigneeIds, ...fields } = input;
  const assignees = resolveAssignees(db, viewer, assigneeIds);
  ensureCategoryExists(db, viewer.id, fields.categoryId);
  const timestamp = nowIso();

  // Task and assignees are saved together, or not at all.
  const id = db.transaction((tx) => {
```

```
    const created = tx
      .insert(tasks)
      .values({ ...fields, createdById: viewer.id, createdAt: timestamp, updatedAt: timestamp })
      .returning({ id: tasks.id })
      .get();
    tx.insert(taskAssignees)
      .values(assignees.map((userId) => ({ taskId: created.id, userId, assignedAt: timestamp })))
      .run();
    return created.id;
  });
  return getTaskById(db, viewer, id);
}
```

It works out who the task is for (`resolveAssignees`: nobody given → the viewer; other people → only allowed for admins, `403` otherwise; unknown users → `400 INVALID_ASSIGNEE`), checks the category belongs to the viewer (otherwise `400 INVALID_CATEGORY`), and then — in one **transaction** — inserts the task with the viewer as **creator** and one `task_assignees` row per person. Finally it loads the full task (category, assignees, subtasks) for the response.

9. Database. Drizzle sends `INSERT INTO tasks (...) VALUES (?, ?, ...) RETURNING id` with the values as parameters. SQLite assigns the new `id`.

10. HTTP response.

```
HTTP/1.1 201 Created
Content-Type: application/json

{"data":{"id":7,"title":"Write report",...,"category":{"id":1,"name":"Work","color":"#4f46e5"}}}
```

On the way out, `secureHeaders` adds its headers and `requestLogger` prints `POST /api/tasks -> 201 (2.1 ms)`.

11. Frontend update. `request()` returns `payload.data`. `submitTaskForm` shows "Task created", closes the dialog, and calls `refresh()` (task list + dashboard) and `loadCategories()` (to update the task counts).

Flow 3: A validation error

```
User clicks "Create task" with an empty title
       ↓
POST /api/tasks  {"title":"", ...}
       ↓
validate('json', createTaskSchema): title.trim().min(1) fails
       ↓
hook throws ValidationError('The request contains invalid data.',
                            [{ field: 'title', message: 'Title is required' }])
       ↓   (the handler never runs)
app.onError(errorHandler): err instanceof AppError → status 400
       ↓
400 {"error":{"code":"VALIDATION_ERROR","message":"…","details":[…]}}
       ↓
api.js: response.ok is false → throw ApiError(message, 400, details)
       ↓
submitTaskForm catch → showFormErrors(...) lists "Title is required"
```

Flow 4: Listing, searching, filtering and sorting

```
User types "report" (debounced 300 ms) or changes a dropdown
       ↓
setFilter(key, value) → state.filters updated → loadTasks()
       ↓
showListState('loading')
       ↓
GET /api/tasks?search=report&status=pending&sort=dueDate&order=asc&today=2026-11-01
  (toQueryString skips empty values)
       ↓
validate('query', listTasksQuerySchema): checks every value is allowed,
  converts categoryId text → number, applies defaults sort=createdAt order=desc
       ↓
listTasks(db, viewer, query, 'mine') builds a list of SQL conditions:
  always   → id IN (SELECT task_id FROM task_assignees WHERE user_id = ?)
             (only tasks assigned to the logged-in user)
  kind     → (SELECT COUNT(*) FROM task_assignees WHERE task_id = tasks.id) > 1   (group)
  dueFrom/dueTo → due_date >= ? / due_date <= ?        (used by the calendar)
  search   → (title LIKE ? OR description LIKE ?)
  status   → status = ?
  due=overdue → due_date IS NOT NULL AND due_date < today AND status != 'completed'
  …and an ORDER BY for the chosen sort
       ↓
SELECT … FROM tasks LEFT JOIN categories … WHERE … ORDER BY …
       ↓
200 {"data":[…]}
       ↓
0 tasks → showListState('empty') with "No matching tasks"
n tasks → renderTaskList(...) then showListState('list')
```

How the `ORDER BY` is chosen in `listTasks`:

```ts
const direction = query.order === 'asc' ? asc : desc;
let orderBy: SQL[];
switch (query.sort) {
  case 'priority':
    orderBy = [direction(priorityRank)];
    break;
  case 'dueDate':
    // Tasks without a due date always go last, whichever direction we sort.
    orderBy = [sql`${tasks.dueDate} IS NULL`, direction(tasks.dueDate)];
    break;
  case 'title':
    orderBy = [direction(sql`${tasks.title} COLLATE NOCASE`)];
    break;
  case 'updatedAt':
    orderBy = [direction(tasks.updatedAt)];
    break;
  default:
    orderBy = [direction(tasks.createdAt)];
}
// A final tie-breaker so the order is always predictable.
orderBy.push(desc(tasks.id));
```

- `priorityRank` is SQL that turns `high/medium/low` into `3/2/1`, because sorting the words alphabetically would give "high, low, medium".
- `due_date IS NULL` is `0` for tasks with a date and `1` for tasks without, so sorting by it first puts dateless tasks last.
- `COLLATE NOCASE` sorts titles ignoring upper/lower case.

Flow 5: Editing a task

```
Click "Edit" on a card → openEditTask(task): fills the form, editingTaskId = task.id
       ↓ user changes fields, clicks "Save changes"
submitTaskForm → api.updateTask(id, data) → PATCH /api/tasks/7  {…all form fields…}
       ↓
validate('param', idParamSchema) → id = 7
validate('json', updateTaskSchema) → every field optional, at least one required
       ↓
updateTask(db, viewer, 7, input):
  getAccessibleTask(db, viewer, 7) → 404 unless assigned to you (or you are admin)
  assigneeIds given by a non-admin → 403
  category CHANGED to one that is not yours → 400
  TRANSACTION: UPDATE tasks SET …, updated_at = now WHERE id = 7
               (admin + assigneeIds: replace the task_assignees rows)
  return getTaskById(db, viewer, 7)
       ↓
200 {"data":{…updated task…}} → toast "Task updated" → refresh
```

Flow 6: Completing and reopening

```
Tick the checkbox → toggleTask(task)
       ↓ task.status !== 'completed'          ↓ task.status === 'completed'
POST /api/tasks/7/complete              POST /api/tasks/7/reopen
       ↓                                       ↓
completeTask(db, viewer, 7)             reopenTask(db, viewer, 7)
= updateTask(db, viewer, 7,             = updateTask(db, viewer, 7,
    {status:'completed'})                   {status:'pending'})
       ↓
200 → toast → refresh() → the card is struck through / restored,
                          the dashboard numbers and progress bar change
```

Flow 7: Deleting a task

```
Click "Delete" → removeTask(task)
       ↓
confirmAction("Delete the task …?") opens the confirm <dialog>
and returns a Promise that resolves when the dialog closes
       ↓ Cancel → stop            ↓ Delete
                          DELETE /api/tasks/7
                                  ↓
                          deleteTask(db, viewer, 7): 404 if you cannot see it;
                          403 unless you created it or are an admin;
                          DELETE FROM tasks WHERE id = ?
                          (assignees + subtasks go too: ON DELETE CASCADE)
                                  ↓
                          204 No Content (empty body)
                                  ↓
                          api.js returns null → toast → refresh + loadCategories
```

Flow 8: Managing categories

```
"Categories" button → category dialog (list rendered by renderCategoryManager)

(every call below passes the logged-in user's ID: c.get('user').id)
Add:    POST   /api/categories {name,color} → createCategory(db, userId, …)
          ensureNameIsFree: user_id = ? AND lower(name) = lower(?) → 409 if taken
          → INSERT with user_id → 201
Save:   PATCH  /api/categories/:id {name,color} → updateCategory
          404 if missing or not yours; 409 if another of YOUR categories has the name
          → UPDATE → 200
Delete: DELETE /api/categories/:id → deleteCategory
          404 if missing or not yours → DELETE → SQLite sets tasks.category_id = NULL → 204

After each: loadCategories() refreshes the dropdowns + manager, loadTasks()
            refreshes the category chips on task cards
```

Flow 9: The dashboard

```
refresh() → api.getDashboard() → GET /api/dashboard?today=2026-11-01
       ↓
validate('query', dashboardQuerySchema)
       ↓
getDashboardStats(db, userId, today) — ONE SQL query over the tasks assigned to
the user (… WHERE id IN (SELECT task_id FROM task_assignees WHERE user_id = ?)):
  SELECT COUNT(*),
         SUM(CASE WHEN status='completed' THEN 1 ELSE 0 END),
         SUM(CASE WHEN status='pending' …), SUM(CASE WHEN status='in_progress' …),
         SUM(CASE WHEN due_date < ? AND status != 'completed' …),   -- overdue
         SUM(CASE WHEN due_date = ? AND status != 'completed' …)    -- due today
  FROM tasks
       ↓
completionPercentage = round(completed / total × 100)  (0 when there are no tasks)
       ↓
200 {"data":{total, completed, pending, inProgress, overdue, dueToday, completionPercentage, today}}
       ↓
renderDashboard(stats): fills every [data-stat] element and sets the progress-bar width
```

`SUM(CASE WHEN condition THEN 1 ELSE 0 END)` is a classic SQL trick: it adds 1 for each row matching the condition, which counts matching rows. `COALESCE(x, 0)` turns the `NULL` that `SUM` returns for an empty table into `0`.

Flow 10: Errors that are not your fault

Situation	What happens
Not logged in / session expired	`requireAuth` → `401 UNAUTHORIZED` → frontend shows the login screen
Normal user calls an admin route	`requireRole('admin')` → `403 FORBIDDEN`
Another website tries to send a form/body-less request with your cookie	`csrf` → `403`
Unknown URL, e.g. `/api/nothing`	`app.notFound` → `404 {"error":{"code":"NOT_FOUND",…}}`
Body is not valid JSON	Hono throws `HTTPException(400)` → error handler → `400 HTTP_ERROR`
Body bigger than 100 KB	`bodyLimit` → `AppError(413)` → `413 PAYLOAD_TOO_LARGE`
A bug throws a normal `Error`	error handler logs it with `console.error`, sends `500 INTERNAL_ERROR` without details
Server is stopped	`fetch` throws → `ApiError('Cannot reach the server. Is it running?')` → error state with **Try again**

Chapter 11 — Understanding Every Source File

This chapter walks through every source file. For each file: why it exists, what it imports and exports, what it receives and returns, and who depends on it.

src/index.ts — the entry point

Why it exists: something has to start everything. This is the file `pnpm start` runs.

```ts
import { serve } from '@hono/node-server';
import { createApp } from './app';
import { config } from './config';
import { closeDatabase, createDatabase, runMigrations } from './db';

// 1. Open the database file and bring its tables up to date.
const db = createDatabase(config.databasePath);
runMigrations(db);

// 2. Build the Hono application.
const app = createApp({ db, logRequests: config.logRequests, secureCookies: config.secureCookies });

// 3. Start a Node.js HTTP server that hands every request to app.fetch.
const server = serve({ fetch: app.fetch, port: config.port }, (info) => {
  console.log(`To-Do app running at http://localhost:${info.port}`);
  console.log(`Database file: ${config.databasePath}`);
});

// 4. When you press Ctrl+C, stop the server and close the database cleanly.
function shutdown() {
  console.log('\nShutting down...');
  server.close(() => {
    closeDatabase(db);
    process.exit(0);
  });
}
process.on('SIGINT', shutdown);
process.on('SIGTERM', shutdown);
```

- **Imports:** `serve` (Node adapter), `createApp`, `config`, database functions.
- **Exports:** nothing — it is the top of the tree.
- `serve({ fetch: app.fetch, port })` starts Node's HTTP server and hands every request to Hono.
- `SIGINT` is the signal sent when you press `Ctrl+C`; `SIGTERM` is what hosting platforms send to stop an app. We close the server and the database cleanly.
- **Depended on by:** nobody (it is run directly).

src/app.ts — builds the Hono application

Why it exists: one function that assembles the complete app, so both the real server and the tests use the identical setup.

- **Imports:** Hono, built-in middleware (`bodyLimit`, `csrf`, `secureHeaders`, `serveStatic`), our middleware (including `injectAuthSettings`), all five route groups, `AppError`, `LoginThrottle`, types.
- **Exports:** `createApp(options)` and the `CreateAppOptions` type.
- **Receives:** `{ db, logRequests?, serveFrontend?, secureCookies? }`. It also creates one `LoginThrottle` per app.
- **Returns:** a configured `Hono<AppEnv>` app.
- **Depended on by:** `src/index.ts`, `tests/helpers.ts`, `scripts/e2e-smoke.ts`.

The order of registration is the order of execution — see the table in Chapter 6. `serveFrontend` is `false` in most tests so they do not need the `public/` folder.

src/config.ts — settings from environment variables

Why it exists: settings such as the port should be changeable without editing code.

```ts
if (existsSync('.env')) {
  process.loadEnvFile('.env');
}

const envSchema = z.object({
  PORT: z.coerce.number().int().min(1).max(65535).default(3000),
  DATABASE_PATH: z.string().min(1).default('./data/todo.db'),
  LOG_REQUESTS: z
    .enum(['true', 'false'])
    .default('true')
    .transform((value) => value === 'true'),
  COOKIE_SECURE: z
    .enum(['true', 'false'])
    .default('false')
    .transform((value) => value === 'true'),
});

const parsed = envSchema.safeParse(process.env);
if (!parsed.success) {
  console.error('Invalid environment variables:', z.prettifyError(parsed.error));
  process.exit(1);
}
```

- `process.env` holds environment variables — named text values given to a program by the operating system.
- `process.loadEnvFile('.env')` (built into Node.js 22+) copies the lines of a `.env` file into `process.env`.

- We validate them with Zod, exactly like request data. `safeParse` returns `{ success, data }` or `{ success: false, error }` instead of throwing.
- `.transform(...)` converts the text `"true"` into the boolean `true`.
- `COOKIE_SECURE` decides whether the session cookie gets the `Secure` flag (HTTPS only). Keep `false` on `http://localhost`; set `true` in production.
- **Exports:** `config = { port, databasePath, logRequests, secureCookies }`.
- **Depended on by:** `src/index.ts`, `scripts/migrate.ts`, `scripts/seed.ts`.

src/types.ts — the shape of the Hono environment

```ts
export type AuthSettings = {
  /** Send the session cookie only over HTTPS. Must be true in production. */
  secureCookies: boolean;
  loginThrottle: LoginThrottle;
};

export type AppEnv = {
  Variables: {
    db: AppDatabase;
    auth: AuthSettings;
    /** The logged-in user. Only set on routes protected by requireAuth. */
    user: PublicUser;
  };
};
```

Tells TypeScript which values live on the context, so `c.get('db')`, `c.get('auth')` and `c.get('user')` are typed.

> **WARNING**
> TypeScript believes `c.get('user')` always exists, but it is only *set* by `requireAuth`. Only call `c.get('user')` in routes protected by `requireAuth` — on a public route it would be `undefined` at run time.

Depended on by: `app.ts`, all route files, `middleware/database.ts`, `middleware/auth.ts`.

src/db/schema.ts — the tables

Defines `users`, `sessions`, `categories` and `tasks`, the allowed `TASK_STATUSES`, `TASK_PRIORITIES` and `USER_ROLES`, and exports inferred types `User`, `Session`, `Task`, `NewTask`, `Category`, `NewCategory`, `TaskStatus`, `TaskPriority`, `UserRole`. Explained in Chapter 7 (tasks, categories) and Chapter 20 (users, sessions). **Depended on by:** `db/index.ts`, services, validators, `drizzle.config.ts`, `scripts/seed.ts`.

src/db/index.ts — connecting to the database

```ts
export type AppDatabase = BetterSQLite3Database<typeof schema> & { $client: Database.Database };

const MIGRATIONS_FOLDER = fileURLToPath(new URL('./migrations', import.meta.url));

export function createDatabase(filePath: string): AppDatabase {
  if (filePath !== ':memory:') {
    mkdirSync(dirname(filePath), { recursive: true });
  }
  const sqlite = new Database(filePath);
  sqlite.pragma('journal_mode = WAL');
  sqlite.pragma('foreign_keys = ON');
  return drizzle({ client: sqlite, schema });
}

export function runMigrations(db: AppDatabase): void {
  const sqlite = db.$client;
  sqlite.pragma('foreign_keys = OFF');
  try {
    migrate(db, { migrationsFolder: MIGRATIONS_FOLDER });
    const problems = sqlite.pragma('foreign_key_check') as unknown[];
    if (problems.length > 0) {
      throw new Error(`Migration left ${problems.length} broken foreign key reference(s).`);
    }
  } finally {
    sqlite.pragma('foreign_keys = ON');
  }
}

export function closeDatabase(db: AppDatabase): void {
  db.$client.close();
}
```

- `createDatabase(path)` makes sure the folder exists, opens the SQLite file (creating it if needed), turns on WAL and foreign keys, and wraps it with Drizzle. `':memory:'` creates a temporary database that lives only in memory — perfect for tests.
- `MIGRATIONS_FOLDER` is calculated from this file's own location (`import.meta.url`), so migrations are found no matter which folder you start the app from.
- `runMigrations(db)` applies pending migrations with foreign-key enforcement switched off (so rebuilding a table cannot cascade-delete other tables' rows), then checks every foreign key with `PRAGMA foreign_key_check` and switches enforcement back on. `finally` guarantees it is switched on again even if a migration fails. Chapter 7 tells the story of why this matters.
- `closeDatabase(db)` closes the raw connection (`$client`).
- **Depended on by:** `index.ts`, `app.ts` (type), services (type), tests, scripts.

src/lib/errors.ts — our error classes

Exports `AppError` (message, HTTP `status`, machine-readable `code`, optional `details`) and three specialised versions:

Class	Status	Code	Thrown when
`ValidationError`	400	`VALIDATION_ERROR`	Zod validation fails (by `validate.ts`)
`NotFoundError`	404	`NOT_FOUND`	A task/category ID does not exist
`ConflictError`	409	`CONFLICT`	A category name is already used
`AppError` directly	any	any	e.g. `400 INVALID_CATEGORY`, `401 UNAUTHORIZED`, `401 INVALID_CREDENTIALS`, `403 FORBIDDEN`, `413 PAYLOAD_TOO_LARGE`, `429 TOO_MANY_ATTEMPTS`

`ConflictError` is also thrown when someone registers with an email that already has an account.

Depended on by: services, `validate.ts`, `error-handler.ts`, `app.ts`.

src/lib/dates.ts — date helpers

- `nowIso()` → current moment as ISO text, used for `createdAt` / `updatedAt`.
- `todayLocal()` → today's date on the server as `YYYY-MM-DD`, the fallback when the browser does not send `today`.

`padStart(2, '0')` turns `"7"` into `"07"`. `getMonth()` counts months from 0, hence `+ 1`.

src/lib/password.ts and src/lib/login-throttle.ts

- `hashPassword(password)` → `Promise<string>` like `"scrypt:<salt>:<hash>"`; `verifyPassword(password, stored)` → `Promise<boolean>`. Used by `auth.service.ts`.
- `LoginThrottle` — a small class with `isBlocked(key)`, `recordFailure(key)` and `reset(key)`. `app.ts` creates one; `auth.routes.ts` uses it on login.

Both are explained line by line in Chapter 20.

src/middleware/auth.ts

Exports `SESSION_COOKIE` (`"session"`), `injectAuthSettings(settings)`, `requireAuth`, `requireRole(role)`, `setSessionCookie(c, token, expiresAt)` and `clearSessionCookie(c)`. Explained in Chapter 20. **Depended on by:** `app.ts` and all five route files.

src/middleware/request-logger.ts, database.ts, validate.ts, error-handler.ts

All four are explained line by line in Chapter 6.

File	Exports	Receives	Returns
`request-logger.ts`	`requestLogger(options)`	`{ enabled }`	middleware
`database.ts`	`injectDatabase(db)`	the database	middleware
`validate.ts`	`validate(target, schema)`	`'json'/'query'/'param'` + Zod schema	middleware
`error-handler.ts`	`errorHandler`, `notFoundHandler`	error + context / context	a JSON `Response`

src/validators/common.ts

```ts
export const dateString = z.iso.date({ message: 'Must be a date in the format YYYY-MM-DD' });

export const idParamSchema = z.object({
  id: z.coerce.number().int().positive({ message: 'ID must be a positive whole number' }),
});
```

`z.iso.date()` accepts only real calendar dates written `YYYY-MM-DD`. `z.coerce.number()` converts the URL text `"7"` to `7` before checking.

src/validators/task.validator.ts

Defines the rules for each field once and builds three schemas from them:

Schema	Used by	Rules
`createTaskSchema`	`POST /api/tasks`	`title` required (1–200 chars, trimmed); `description` ≤ 2000 (default `''`); `status` default `pending`; `priority` default `medium`; `categoryId` positive integer or `null`; `dueDate` `YYYY-MM-DD` or `null`
`updateTaskSchema`	`PATCH /api/tasks/:id`	same fields, all optional; `.refine(...)` requires at least one
`listTasksQuerySchema`	`GET /api/tasks`	`search`, `status`, `priority`, `categoryId`, `due` (`overdue/today/upcoming/none`), `kind` (`individual/group`), `dueFrom`, `dueTo`, `sort` (default `createdAt`), `order` (default `desc`), `today`

Schema	Used by	Rules
`adminListTasksQuerySchema`	`GET /api/admin/tasks`	the same, plus `assigneeId`

Both `createTaskSchema` and `updateTaskSchema` accept an optional `assigneeIds` (a list of 1–50 user IDs). Whether the viewer may use it (admins only) is a *business rule*, so it is checked in the service, not in the validator.

It also exports the inferred types `CreateTaskInput`, `UpdateTaskInput`, `ListTasksQuery`, used by the service functions. That link is important: *the service's input type is generated from the validator*, so they can never disagree.

src/validators/category.validator.ts and dashboard.validator.ts

- `createCategorySchema`: `name` 1–50 chars (trimmed), `color` must match `#RRGGBB` (default `#6366f1`).
- `updateCategorySchema`: both optional, at least one required.
- `dashboardQuerySchema`: optional `today` date.

src/validators/auth.validator.ts

- `registerSchema`: `name` 1–50 (trimmed); `email` trimmed, **lower-cased**, ≤ 254, valid email format; `password` 8–128 characters.
- `loginSchema`: same `email` rule; `password` just 1–128 characters (we never hint at password rules on the login form).
- Exports the inferred types `RegisterInput` and `LoginInput`.

src/services/auth.service.ts

Function	Receives	Returns	Errors
`registerUser(db, input)` *(async)*	name, email, password	`PublicUser`	409 email taken
`authenticate(db, email, password)` *(async)*	credentials	`PublicUser` or `null`	—
`createSession(db, userId)`	user ID	`{ token, expiresAt }`	—
`getUserBySessionToken(db, token)`	cookie value	`PublicUser` or `null`	—
`deleteSession(db, token)`	cookie value	nothing	—
`listUsers(db)`	—	users with `taskCount`	—

Also exports `SESSION_TTL_MS` (7 days) and the types `PublicUser` and `UserWithTaskCount`. Explained in Chapter 20. **Depended on by:** `middleware/auth.ts`, `routes/auth.routes.ts`, `routes/admin.routes.ts`, `scripts/seed.ts`, tests.

src/services/task.service.ts

Every function takes the **viewer** — the logged-in user `{ id, role }` — as its **second** parameter and applies the task **authorization** rules (Chapter 21): you see tasks assigned to you, admins see all.

Function	Receives	Returns	Errors
`listTasks(db, viewer, query, scope)`	filters; scope `'mine'` (assigned to viewer) or `'all'` (admin page)	`TaskView[]`	—
`getTaskById(db, viewer, id)`	id	`TaskView`	404 (missing or not visible)
`getAccessibleTask(db, viewer, id)`	id	the raw `Task` row	404 — shared with the subtask service
`createTask(db, viewer, input)`	`CreateTaskInput` (+ optional `assigneeIds`)	the new task	400 category/assignee, 403
`updateTask(db, viewer, id, input)`	id + `UpdateTaskInput`	the updated task	404, 400, 403 (assignees by non-admin)
`deleteTask(db, viewer, id)`	id	nothing	404, 403 (not creator/admin)
`completeTask(db, viewer, id)`	id	the updated task	404
`reopenTask(db, viewer, id)`	id	the updated task	404

A `TaskView` is the task's columns (without `createdById`) plus `category`, `assignees` (`[{id, name}]`), `isGroup`, `subtasks`, `canDelete` and — **for administrators only** — `createdBy` (`{id, name, email}`).

Private helpers: `selectTasksWithCategory` (the shared `SELECT … LEFT JOIN categories`), `hydrate` (loads assignees, subtasks and — for admins — creators of *all* listed tasks in a few queries and builds `TaskView`s), `taskIdsAssignedTo` (a subquery), `resolveAssignees`, `ensureCategoryExists`, and the `priorityRank` and `assigneeCount` SQL snippets.

A **LEFT JOIN** returns every task, plus its category's columns when it has one, or `null` when it does not. A plain (inner) join would hide tasks without a category.

Depended on by: `routes/task.routes.ts`, `routes/admin.routes.ts`, `services/subtask.service.ts`, `scripts/seed.ts`.

src/services/subtask.service.ts and src/validators/subtask.validator.ts

Function	Receives	Returns	Errors
`addSubtask(db, viewer, taskId, { title })`	parent task ID, title	the new subtask	404 (task not visible)
`updateSubtask(db, viewer, id, { title?, done? })`	subtask ID, changes	the updated subtask	404
`deleteSubtask(db, viewer, id)`	subtask ID	nothing	404

Whoever may see a task may change its subtasks (`getAccessibleTask` decides). Every change also moves the parent task's `updatedAt` forward. The validators: `createSubtaskSchema` (`title` 1–200, trimmed) and `updateSubtaskSchema` (`title` and/or `done` boolean, at least one).

src/services/category.service.ts

Function	Returns	Errors
`listCategories(db, userId)`	the user's categories A–Z, each with `taskCount`	—
`getCategoryById(db, userId, id)`	one category	404 (missing or not yours)
`createCategory(db, userId, input)`	the new category	409 duplicate
`updateCategory(db, userId, id, input)`	the updated category	404, 409
`deleteCategory(db, userId, id)`	nothing	404

`listCategories` uses `LEFT JOIN tasks … WHERE categories.user_id = ? GROUP BY categories.id` with `COUNT(tasks.id)` to count each category's tasks in one query. `ensureNameIsFree` compares `lower(name) = lower(?)` **among the user's own categories**, so "Work" and "WORK" count as the same name; when renaming, it ignores the category's own row so you can change "home" to "Home".

src/services/dashboard.service.ts

Exports `getDashboardStats(db, userId, today)` and the `DashboardStats` type. The query counts only tasks **assigned to** the user, and also returns `groupTasks` (tasks with more than one assignee). See Flow 9 in Chapter 10. **Definitions:** *overdue* = has a due date before today and is not completed; *due today* = due today and not completed.

src/routes/*.routes.ts

Each file creates a small `new Hono<AppEnv>()` and registers handlers. Every handler follows the same steps: **read validated input → call one service function (passing `c.get('user').id`) → return JSON**. The complete list of endpoints is in Chapter 12.

File	Mounted at	Protection
`auth.routes.ts`	`/api/auth`	public, except `/me` (`requireAuth`)
`task.routes.ts`	`/api/tasks`	`taskRoutes.use('*', requireAuth)`
`subtask.routes.ts`	`/api/subtasks`	`subtaskRoutes.use('*', requireAuth)`
`category.routes.ts`	`/api/categories`	`categoryRoutes.use('*', requireAuth)`
`dashboard.routes.ts`	`/api/dashboard`	`dashboardRoutes.use('*', requireAuth)`
`admin.routes.ts`	`/api/admin`	`adminRoutes.use('*', requireAuth, requireRole('admin'))`

public/index.html, public/css/styles.css, public/js/*.js

Explained in Chapter 8. Key functions in the frontend:

File	Function	Purpose
`api.js`	`request(path, options)`	The single place that calls `fetch`
`api.js`	`localToday()`	Browser's local date `YYYY-MM-DD`
`api.js`	`onUnauthorized(handler)`	Registers what to do when a request answers 401
`api.js`	`getMe`, `login`, `register`, `logout`, `getUsers`, `getAdminTasks`, `getTasks`, `createTask`, `updateTask`, `deleteTask`, `completeTask`, `reopenTask`, `addSubtask`, `updateSubtask`, `deleteSubtask`, `getCategories`, `createCategory`, `updateCategory`, `deleteCategory`, `getDashboard`	One function per endpoint
`api.js`	`localToday`, `toDateString`, `addDays`	Date helpers (`YYYY-MM-DD` in the user's time zone)

File	Function	Purpose
`ui.js`	`el`, `showToast`, `renderDashboard`, `resetDashboard`, `renderCompactList`, `renderCurrentUser`, `renderActiveTab`, `showListState`, `renderTaskList` (with subtasks and assignee badges), `CALENDAR_MODES`, `calendarRange`, `shiftCalendar`, `renderCalendar` (month, week, day, agenda), `renderAdminUsers`, `renderAdminTasks`, `renderAssigneeOptions`, `renderCategoryOptions`, `renderCategoryManager`, `showFormErrors`, `confirmAction`	Drawing on the screen
`app.js`	`start`, `showAuthScreen`, `showApp`, `submitLogin`, `submitRegister`, `logout`	Login, register, logout
`app.js`	`viewFromHash`, `showView`, `loadCurrentView`	Switching between the Dashboard, Tasks, Calendar and Admin views
`app.js`	`loadDashboard`, `loadTasks`, `loadCalendar`, `setCalendarMode`, `moveCalendar`, `loadAdmin`, `loadUsers`, `loadCategories`	Load data, change calendar view type
`app.js`	`openTaskForm`, `updateAssignControls`, `readTaskForm`, `submitTaskForm`, `toggleTask`, `removeTask`	Task actions (incl. assigning)
`app.js`	`addSubtask`, `toggleSubtask`, `removeSubtask`	Subtask actions
`app.js`	`submitCategoryForm`, `saveCategory`, `removeCategory`	Category actions
`app.js`	`setFilter`, `onSearchInput`, `resetFilterControls`, `clearFilters`, `bindEvents`	Filters and event wiring

Configuration files

File	Purpose
`tsconfig.json`	TypeScript settings: `strict` checking, modern JavaScript target, `noEmit` (we only *check* types; `tsx` runs the code). `moduleResolution: "Bundler"` lets us write imports without `.js` extensions.
`drizzle.config.ts`	Tells drizzle-kit where the schema is and where to write migrations.
`vitest.config.ts`	Tells Vitest to run `tests/**/*.test.ts` in a Node environment.

File	Purpose
`.env.example`	Documents the available environment variables. Copy it to `.env` to customise.
`.gitignore`	Keeps `node_modules/`, `data/`, database files and `.env` out of git.
`pnpm-workspace.yaml`	pnpm settings: which packages may run install scripts.

Scripts

File	Command	What it does
`scripts/migrate.ts`	`pnpm db:migrate`	Applies migrations without starting the server
`scripts/seed.ts`	`pnpm db:seed`	Only if the database has **no accounts**: creates the demo account `demo@example.com` / `demo-password-123` (it becomes the admin). If older ownerless tasks exist, the demo account claims them; otherwise it gets 3 categories and 6 example tasks
`scripts/e2e-smoke.ts`	`pnpm test:e2e`	Starts the app on a temporary database and clicks through it in a headless browser
`scripts/build-docs.ts`	`pnpm docs:build`	Regenerates this manual (Chapter 19)
`scripts/lib/browser.ts`	—	`findBrowser()` locates Chrome/Edge/Chromium (or `CHROME_PATH`)

The seed script reuses the real services (`registerUser`, `createCategory`, `createTask`), so example data follows exactly the same rules as data created through the API.

Chapter 12 — API Reference

Base URL: `http://localhost:3000` **Format:** JSON in, JSON out. Successful responses look like `{ "data": … }`. Errors look like `{ "error": { "code", "message", "details?" } }`. **Authentication:** every endpoint except `GET /api/health`, `POST /api/auth/register`, `POST /api/auth/login` and `POST /api/auth/logout` requires the `session` cookie (set by register/login). Without it the answer is `401 UNAUTHORIZED`. **Authorization:** you see the tasks **assigned to you** (administrators can open every task) and only **your own** categories. Any other task or category behaves exactly like one that does not exist (`404`). Only administrators may assign tasks to other people; only a task's creator or an administrator may delete it (`403 FORBIDDEN` otherwise). `/api/admin/*` requires the `admin` role. **CSRF:** POST/PATCH/DELETE requests without a JSON body must come from the same origin (browsers do this automatically; tools like `curl` need `-H "Origin: http://localhost:3000"`).

The Task object:

JSON

```
{
  "id": 1,
  "title": "Read the Hono learning manual",
  "description": "Start with chapters 1 and 2.",
  "status": "in_progress",
  "priority": "high",
  "categoryId": 3,
  "dueDate": "2026-11-01",
  "createdAt": "2026-11-01T09:36:23.408Z",
  "updatedAt": "2026-11-01T09:36:23.408Z",
  "category": { "id": 3, "name": "Learning", "color": "#d97706" },
  "assignees": [ { "id": 1, "name": "Minal" }, { "id": 2, "name": "Sita" } ],
  "isGroup": true,
  "subtasks": [ { "id": 4, "title": "Book venue", "done": true, "createdAt": "2026-11-
01T11:52:00.000Z" } ],
  "canDelete": true,
  "createdBy": { "id": 1, "name": "Minal", "email": "admin@gmail.com" }
}
```

- `assignees` — everyone the task is assigned to; `isGroup` is `true` when there are two or more.
- `subtasks` — the checklist inside the task, oldest first.
- `canDelete` — whether **you** may delete it (you created it, or you are an admin).
- `createdBy` — **only present in responses to administrators.** Normal users never receive it (not even the creator's ID).

The Subtask object: `{ "id": 4, "title": "Book venue", "done": true, "createdAt": "…" }`

The Category object (in lists it also has `taskCount`):

JSON

```
{ "id": 3, "name": "Learning", "color": "#d97706", "createdAt": "2026-11-01T09:36:23.401Z",
"taskCount": 2 }
```

The User object (never contains the password hash):

```
JSON
{ "id": 1, "name": "Minal", "email": "owner@example.com", "role": "admin", "createdAt": "2026-11-01T10:12:00.000Z" }
```

Error codes: `VALIDATION_ERROR` (400), `INVALID_CATEGORY` (400), `INVALID_ASSIGNEE` (400), `HTTP_ERROR` (e.g. 400 for invalid JSON, 403 for a blocked cross-site request), `UNAUTHORIZED` (401), `INVALID_CREDENTIALS` (401), `FORBIDDEN` (403), `NOT_FOUND` (404), `CONFLICT` (409), `PAYLOAD_TOO_LARGE` (413), `TOO_MANY_ATTEMPTS` (429), `INTERNAL_ERROR` (500).

GET /api/health

- **Purpose:** check that the server is running. **Public.**
- **Response:** `200 {"data":{"status":"ok"}}`
- **Implementation:** `src/app.ts`

POST /api/auth/register

- **Purpose:** create an account and log it in. **Public.** The first account ever becomes `admin`.
- **Body:** `{ "name": "Sam", "email": "sam@example.com", "password": "at-least-8-chars" }` — name 1–50, email valid (stored lower-case), password 8–128.
- **Response:** `201 {"data":User}` plus `Set-Cookie: session=…; HttpOnly; SameSite=Lax; Path=/; Expires=…`
- **Errors:** `400` validation, `409` email already registered.
- **Implementation:** `src/routes/auth.routes.ts` → `registerUser`, `createSession` in `src/services/auth.service.ts`

POST /api/auth/login

- **Purpose:** log in. **Public.**
- **Body:** `{ "email": "sam@example.com", "password": "…" }`
- **Response:** `200 {"data":User}` plus the session cookie.
- **Errors:** `400` validation; `401 INVALID_CREDENTIALS` (same message for unknown email and wrong password); `429 TOO_MANY_ATTEMPTS` after 5 failures for that email within 15 minutes.

POST /api/auth/logout

- **Purpose:** end the session. **Public** (works even if already logged out).
- **Response:** `204`; deletes the session row and tells the browser to delete the cookie.

GET /api/auth/me

- **Purpose:** who is logged in?
- **Response:** `200 {"data":User}`. **Errors:** `401` when not logged in.

GET /api/admin/users

- **Purpose:** list all accounts with their number of tasks. **Admins only.**
- **Response:** `200 {"data":[User + "taskCount", …]}` ordered by ID.
- **Errors:** `401` not logged in, `403` not an admin.
- **Implementation:** `src/routes/admin.routes.ts` → `listUsers` in `src/services/auth.service.ts`

GET /api/admin/tasks

- **Purpose: every** task in the system, with assignees and `createdBy`. **Admins only.**
- **Query:** all the filters of `GET /api/tasks`, plus `assigneeId` (only tasks assigned to that user).
- **Response:** `200 {"data":[Task, …]}`. **Errors:** `401`, `403`, `400` for invalid filters.
- **Implementation:** `src/routes/admin.routes.ts` → `listTasks(db, viewer, query, 'all')`

GET /api/tasks

- **Purpose:** list the tasks **assigned to you** (individual and group), with optional search, filters and sorting.
- **Query parameters (all optional):**

Name	Values	Meaning
`search`	text (≤ 200)	title or description contains this (case-insensitive)
`status`	`pending`, `in_progress`, `completed`	exact status
`priority`	`low`, `medium`, `high`	exact priority
`categoryId`	positive integer	tasks in this category
`due`	`overdue`, `today`, `upcoming`, `none`	due-date filter relative to `today`
`sort`	`createdAt` (default), `updatedAt`, `dueDate`, `priority`, `title`	sort field
`order`	`asc`, `desc` (default)	sort direction
`kind`	`individual`, `group`	one assignee / two or more assignees

Name	Values	Meaning
`dueFrom`, `dueTo`	`YYYY-MM-DD`	only tasks due within this range (inclusive) — used by the calendar
`today`	`YYYY-MM-DD`	the client's date (defaults to the server's date)

- **Response:** `200 {"data":[Task, …]}`
- **Errors:** `400` for an unknown value (e.g. `status=finished`).
- **Implementation:** `src/routes/task.routes.ts` → `listTasks` in `src/services/task.service.ts`

GET /api/tasks/:id

- **Purpose:** one task.
- **Response:** `200 {"data":Task}`
- **Errors:** `400` if `id` is not a positive integer; `404` if not found.

POST /api/tasks

- **Purpose:** create a task.
- **Body:**

Field	Type	Required	Default	Rules
`title`	string	yes	—	trimmed, 1–200 characters
`description`	string	no	`""`	≤ 2000 characters
`status`	string	no	`pending`	`pending`, `in_progress`, `completed`
`priority`	string	no	`medium`	`low`, `medium`, `high`
`categoryId`	integer or null	no	`null`	must be an existing category
`dueDate`	string or null	no	`null`	`YYYY-MM-DD`
`assigneeIds`	integer[]	no	`[you]`	1–50 existing user IDs. **Admins only** may include other people. One ID = individual, several = group task

- **Response:** `201 {"data":Task}`
- **More errors:** `400 INVALID_ASSIGNEE` (unknown user), `403` (non-admin assigning others).

- **Errors:** `400 VALIDATION_ERROR`, `400 INVALID_CATEGORY`, `400 HTTP_ERROR` (invalid JSON), `413`.
- **Implementation:** `createTask` in `src/services/task.service.ts`

PATCH /api/tasks/:id

- **Purpose:** change some fields of a task. `updatedAt` is refreshed.
- **Body:** any subset of the POST fields (at least one). Use `null` to clear `categoryId` or `dueDate`. Every assignee may edit. `assigneeIds` **replaces** the assignee list (admins only). A `categoryId` is only checked when it *changes* (so group members can keep the creator's category).
- **Response:** `200 {"data":Task}`
- **Errors:** `400` (validation / empty body / invalid category or assignee), `403` (non-admin changing assignees), `404`.

POST /api/tasks/:id/complete

- **Purpose:** set status to `completed`.
- **Body:** none. **Response:** `200 {"data":Task}`. **Errors:** `400`, `404`.

POST /api/tasks/:id/reopen

- **Purpose:** set status back to `pending`.
- **Body:** none. **Response:** `200 {"data":Task}`. **Errors:** `400`, `404`.

DELETE /api/tasks/:id

- **Purpose:** permanently delete a task, its assignments and its subtasks. Only its **creator** or an **admin**.
- **Response:** `204` with no body. **Errors:** `400`, `403` (not creator/admin), `404`.

POST /api/tasks/:id/subtasks

- **Purpose:** add a subtask (checklist item) inside a task you can see.
- **Body:** `{ "title": "Book flights" }` (1–200 characters, trimmed).
- **Response:** `201 {"data":Subtask}`. **Errors:** `400`, `404`.
- **Implementation:** `src/routes/task.routes.ts` → `addSubtask` in `src/services/subtask.service.ts`

PATCH /api/subtasks/:id

- **Purpose:** rename and/or tick a subtask.
- **Body:** `{ "title"?: "…", "done"?: true }` (at least one).
- **Response:** `200 {"data":Subtask}`. **Errors:** `400`, `404` (missing, or its task is not visible to you).

DELETE /api/subtasks/:id

- **Response:** `204`. **Errors:** `400`, `404`.

GET /api/categories

- **Purpose:** list all categories alphabetically with their task counts.
- **Response:** `200 {"data":[Category, …]}`
- **Implementation:** `listCategories` in `src/services/category.service.ts`

POST /api/categories

- **Body:** `{ "name": "Work", "color": "#4f46e5" }` — `name` required (1–50, trimmed), `color` optional (`#RRGGBB`, default `#6366f1`).
- **Response:** `201 {"data":Category}`
- **Errors:** `400` validation, `409` name already exists (ignoring letter case).

PATCH /api/categories/:id

- **Body:** `name` and/or `color`.
- **Response:** `200 {"data":Category}`. **Errors:** `400`, `404`, `409`.

DELETE /api/categories/:id

- **Purpose:** delete a category. Its tasks are kept and become uncategorised.
- **Response:** `204`. **Errors:** `400`, `404`.

GET /api/dashboard

- **Query:** optional `today=YYYY-MM-DD`.
- **Response:**

```json
{ "data": { "total": 6, "completed": 1, "pending": 4, "inProgress": 1, "overdue": 1, "dueToday": 1, "groupTasks": 2, "completionPercentage": 17, "today": "2026-11-01" } }
```

- **Errors:** `400` for an invalid date.
- **Implementation:** `src/routes/dashboard.routes.ts` → `getDashboardStats` in `src/services/dashboard.service.ts`

Chapter 13 — Testing

What is automated testing?

An **automated test** is code that checks other code. Instead of clicking through the app by hand after every change, you run one command and the computer checks dozens of behaviours in about two seconds.

Tests give you **confidence**: when you change something, the tests tell you immediately if you broke something else.

Unit tests and integration tests

Kind	Tests	Example in this project
Unit test	One small piece in isolation	Calling `getDashboardStats(db, userId, today)` directly
Integration test	Several pieces working together	Sending `POST /api/tasks` through middleware, validation, service and database
End-to-end (E2E) test	The whole system as a user sees it	`pnpm test:e2e` clicks buttons in a real browser

Most of our tests are integration tests, because they check what really matters — the behaviour of the API — while still being fast.

Vitest

Vitest is our testing framework. A test file looks like this (from `tests/tasks.test.ts`):

```ts
import { beforeEach, describe, expect, it } from 'vitest';
import { createTestApp } from './helpers';

let api: Awaited<ReturnType<typeof createTestApp>>;

beforeEach(async () => {
  api = await createTestApp();    // a brand-new app + empty database + logged-in user for EVERY test
});

describe('POST /api/tasks (create)', () => {
  it('creates a task with default values', async () => {
    const res = await api.request('POST', '/api/tasks', { title: '  Buy milk  ' });

    expect(res.status).toBe(201);
    expect(res.json.data).toMatchObject({
      id: 1,
      title: 'Buy milk', // whitespace trimmed by the validator
```

```
      status: 'pending',
      priority: 'medium',
    });
  });
});
```

Vitest word	Meaning
`describe('name', fn)`	A group of related tests
`it('does something', fn)`	One test
`beforeEach(fn)`	Run before every test (fresh setup)
`expect(value).toBe(x)`	Check that `value` is exactly `x`
`toEqual(x)`	Check that objects/arrays have the same contents
`toMatchObject(x)`	Check that an object contains at least these properties
`toContainEqual(x)`	Check that an array contains an item equal to `x`

The test helper

`tests/helpers.ts` creates an isolated app for each test. Since accounts exist, it also simulates **browsers with cookies** (shortened here):

TS

```ts
export async function createTestApp(options: { serveFrontend?: boolean } = {}) {
  const db = createDatabase(':memory:');
  runMigrations(db);
  const app = createApp({ db, logRequests: false, serveFrontend: options.serveFrontend ?? false });

  /** A pretend browser: remembers the session cookie between requests. */
  function client() {
    let cookie = '';
    async function request(method: string, path: string, body?: unknown) {
      const headers = { Origin: 'http://localhost', ...(cookie ? { Cookie: cookie } : {}), ... };
      const response = await app.request(path, { method, headers, body: ... });
      // Remember (or forget) the session cookie, exactly like a browser.
      for (const line of response.headers.getSetCookie()) { /* update cookie */ }
      ...
      return { status, headers, json, setCookie };
    }
    return { request, get cookie() { return cookie; }, setCookie(value) { cookie = value; } };
  }

  async function registerClient(details = {}) { /* new client + POST /api/auth/register */ }

  const defaultUser = await registerClient({ name: 'Default User', email: 'default@example.com' });
  return { app, db, request: defaultUser.request, user: defaultUser.user, client, registerClient };
}
```

- `api.request(...)` sends requests as a **default user that is already logged in** (it registered first, so it is an admin). That is why the older task/category tests did not need to change much.
- `api.client()` is a new, **logged-out** browser — used to check that protected routes answer `401`.
- `api.registerClient()` is a new browser that registers its **own account** — used to prove that Alice cannot see Bob's tasks.
- Every request sends `Origin: http://localhost`, like a real browser, so the CSRF middleware accepts it.

`app.request(path, options)` is a Hono feature: it sends a request **directly** to the app without any network. This is possible because we separated `createApp` from the server in `index.ts`.

What is tested

File	Tests	Covers
`tests/tasks.test.ts`	23	create (defaults, all fields, validation, unknown category, invalid JSON), get (found, 404, bad ID), edit (partial update, `updatedAt`, clearing fields, empty body, 404), complete/reopen, delete, list (default order, search, status/priority/category filters, all four due filters, sort by priority/due date/title, invalid filter)
`tests/categories.test.ts`	8	create, default colour, duplicate names (case-insensitive), invalid colour, list with counts and A–Z order, rename/recolour, renaming own case, delete keeps tasks, 404
`tests/dashboard.test.ts`	3	empty database, every statistic with mixed data (unit test), invalid date
`tests/app.test.ts`	6	health check without login, JSON 404, security headers, 413 body limit, CSRF: cross-site DELETE blocked with 403, frontend files served
`tests/auth.test.ts`	15	register (cookie flags, no hash in response, scrypt hash stored, duplicate email, validation, first user admin), login (success, identical 401 for wrong password/unknown email, lock after 5 failures → 429), `/me`, logout invalidates the old cookie, expired session, made-up token, only token *hashes* stored, ownerless data claimed by the first account and invisible meanwhile

File	Tests	Covers
`tests/assignment.test.ts`	14	own tasks are individual; normal users cannot assign others (403); admin assigns individual and group tasks; every group member sees and completes the shared task; non-members get 404; only admins change assignees; unknown/empty assignees rejected; group count on the dashboard; `kind` filter; `createdBy` only for admins (and no `createdById` for others); delete only by creator/admin (`canDelete`); admin all-tasks list with `assigneeId` filter and 403 for normal users; `dueFrom` / `dueTo` range; group members keep the creator's category
`tests/subtasks.test.ts`	8	add (trimmed) and list subtasks with the task; tick, rename, delete; parent `updatedAt` moves; validation; other users get 404; group members share the checklist; deleting a task deletes its subtasks; missing subtask 404
`tests/migrations.test.ts`	1	regression test: a database at the old version keeps every owner as creator + assignee after migrations 0002/0003, and foreign keys are switched back on
`tests/authorization.test.ts`	15	10 protected routes answer 401 when logged out; users cannot see, edit, complete, reopen or delete each other's tasks (404); separate dashboards; categories separate, same name allowed per user, cannot use another user's category; admin route 403 for normal users, user list (without hashes) for admins
`scripts/e2e-smoke.ts`	26 steps	real browser: login screen for visitors, registration validation, first account (admin) lands on the Dashboard view, Tasks tab empty state, category creation, server validation errors, create task with all fields, XSS-safe rendering, edit, add/tick/delete subtasks, complete/reopen, search + filters + "no matching" state, sort, delete with confirmation, Calendar shows tasks and "+" pre-fills the due date, Calendar switches Month/Week/Day/Agenda (‹ › navigation, date click opens Day view, view type remembered after reload), Dashboard counts and lists, reload keeps data and login, a second user (separate browser) sees nothing of the first and no Admin tab, admin assigns a group task from the Admin view, admin sees "Created by", the member sees the group badge but no "Created by" and no Delete, a shared subtask ticked by one member is seen by the other, logout, wrong password refused, log back in, zero console errors

How to run the tests

```bash
pnpm test           # run all Vitest tests once
pnpm test:watch     # re-run automatically when files change
pnpm test:e2e       # browser smoke test (needs Chrome or Edge installed)
pnpm typecheck      # TypeScript type check
```

A successful run ends like this:

```
 Test Files  9 passed (9)
      Tests  93 passed (93)
```

If a test fails, Vitest prints which test, the expected value, the actual value, and the line number.

Writing your own test

1. Decide on the behaviour: "searching is case-insensitive".
2. **Arrange** — create the data you need.
3. **Act** — make the request.
4. **Assert** — check the result with `expect`.

```ts
it('finds tasks regardless of letter case', async () => {
  await api.request('POST', '/api/tasks', { title: 'Buy MILK' });     // arrange
  const res = await api.request('GET', '/api/tasks?search=milk');     // act
  expect(res.json.data).toHaveLength(1);                              // assert
});
```

> **TIP**
> A good habit: before fixing a bug, write a test that fails *because of* the bug. Then fix the code and watch the test pass. The bug can never silently return.

Chapter 14 — Security

Security means making sure the application only does what it is supposed to do, even when someone deliberately sends unexpected input.

Input validation

Never trust data from the client. Anyone can send any request to your API with tools like `curl`, bypassing the frontend completely. So the **server** validates everything with Zod:

- types (`title` must be a string), lengths (≤ 200), formats (`YYYY-MM-DD`, `#RRGGBB`);
- allowed values only (`status` must be one of three words);
- IDs must be positive integers;
- referenced categories must exist.

The frontend's `maxlength` and `required` attributes are only for convenience — they are *not* security.

SQL injection

SQL injection happens when user text is glued directly into SQL. Imagine this dangerous code (which we do **not** write):

```ts
// ❌ NEVER DO THIS
db.run(`SELECT * FROM tasks WHERE title LIKE '%${search}%'`);
```

If someone searched for `'; DROP TABLE tasks; --`, the database would receive a command to delete the table.

Safe database queries

Drizzle always sends values as **parameters**, separate from the SQL text:

```ts
// ✅ What our code does (src/services/task.service.ts)
const pattern = `%${query.search}%`;
conditions.push(or(like(tasks.title, pattern), like(tasks.description, pattern))!);
```

The SQL becomes `… WHERE title LIKE ? OR description LIKE ?` and the search text is sent separately. The database treats it purely as text, never as commands. Even our hand-written `sql` templates (like `sql`lower(${categories.name}) = lower(${name})``) turn `${...}` values into parameters automatically.

XSS (Cross-Site Scripting)

XSS happens when user-provided text is inserted into a page as HTML, so a task titled `<img src=x onerror=alert(1)>` would *run code* in every visitor's browser.

Our defences:

1. **`textContent`, never `innerHTML`.** `ui.js` always puts user text into the page with `textContent` (via the `el()` helper), which displays text literally. The browser smoke test creates a task with exactly that malicious title and checks that no `<img>` element appears.
2. **Content Security Policy (CSP).** `secureHeaders` in `src/app.ts` sends a CSP header telling the browser to run scripts only from our own server (`script-src 'self'`). Even if malicious HTML somehow got in, inline scripts would be blocked.

```ts
contentSecurityPolicy: {
  defaultSrc: ["'self'"],
  scriptSrc: ["'self'"],
  styleSrc: ["'self'"],
  imgSrc: ["'self'", 'data:'],
  objectSrc: ["'none'"],
  frameAncestors: ["'none'"],
},
```

`frameAncestors: 'none'` stops other websites from embedding our app in an invisible frame (a trick called *clickjacking*). `secureHeaders` also adds `X-Content-Type-Options: nosniff` and other protective headers.

Error handling

The error handler sends **generic** messages for unexpected errors (`Something went wrong on the server.`) and logs the details only on the server. Stack traces and SQL errors can reveal how the system works and must never reach users.

Limiting request size

`bodyLimit` rejects bodies above 100 KB with `413`, so nobody can exhaust the server's memory with a huge request.

Environment variables

Settings and secrets (passwords, API keys) belong in **environment variables**, not in code:

- `.env` holds your local values and is listed in `.gitignore`, so it is never committed.

- `.env.example` is committed and shows *which* variables exist, without real secret values.
- `src/config.ts` validates them at startup.

Our app currently has no secrets, but you now know where they would go.

Dependency security

Every package you install is code written by someone else that runs with your permissions.

- Install only what you need (this project has seven runtime dependencies).
- `pnpm audit` checks installed packages against a database of known vulnerabilities.
- `pnpm outdated` shows which packages have newer versions; update regularly.
- The lockfile ensures you get exactly the versions you tested.
- pnpm does not run package install scripts unless allowed (`pnpm-workspace.yaml`), and it refuses brand-new package versions for a short period — both reduce supply-chain attack risk.

Data protection

- The database file `data/todo.db` contains all tasks and all accounts (with *hashed* passwords). It is excluded from git. Back it up by copying it while the server is stopped, and protect the copies.
- Every user sees only their own tasks and categories (authorization, Chapter 20).

Authentication and authorization (summary)

Chapter 20 explains each of these protections in depth:

Threat	Protection in this app
Someone uses the API without an account	`requireAuth` on every task, category, dashboard and admin route → `401`
A user reads or changes another user's data	Every task query only includes tasks **assigned to** the logged-in user (admins: all); categories are filtered by owner; foreign IDs look like `404`
A normal user assigns tasks to others or deletes someone else's task	Service rules: assigning is admin-only, deleting is creator/admin-only → `403`
Normal users learn who created a task	`createdBy` is only added to responses for administrators
A normal user opens admin features	`requireRole('admin')` → `403`
A stolen database reveals passwords	Passwords stored only as salted **scrypt** hashes
A stolen database reveals active logins	Only the SHA-256 **hash** of each session token is stored
XSS steals the login cookie	The cookie is `HttpOnly` — JavaScript cannot read it

Threat	Protection in this app
Another website makes your browser send requests (CSRF)	`SameSite=Lax` cookie + Hono's `csrf()` middleware
Password guessing	5 failed logins per email → locked for 15 minutes (`429`)
Finding out which emails have accounts	Identical message *and* similar timing for "unknown email" and "wrong password"
Cookie sent over unencrypted Wi-Fi	`COOKIE_SECURE=true` in production → cookie only sent over HTTPS

> **WARNING**
>
> In production, always serve the app over **HTTPS** and set `COOKIE_SECURE=true`. See Chapter 17.

Chapter 15 — Troubleshooting

How to troubleshoot anything

1. **Read the error message slowly**, all of it. It usually says exactly what is wrong and where (file and line number).
2. Check the **terminal** where the server runs (server errors, request log).
3. Check the browser's **DevTools → Console** (frontend errors) and **Network** tab (status codes and responses).
4. Ask: *what did I change last?* Undo it and see if the problem disappears.
5. Run `pnpm typecheck` and `pnpm test`.

Common problems

Symptom	Likely cause	Solution
`'pnpm' is not recognized` / `command not found: pnpm`	pnpm is not installed	`npm install -g pnpm` or `corepack enable pnpm`, then open a new terminal
`Error: listen EADDRINUSE: address already in use :::3000`	Another program (often another copy of this server) uses port 3000	Stop the other server (`Ctrl+C` in its terminal), or set `PORT=3001` in `.env`
`Cannot find module 'hono'`	Dependencies not installed	Run `pnpm install`
`ERR_PNPM_EXECUTOR_LIFECYCLE_SCRIPT_FAILED … better-sqlite3 … node-gyp`	pnpm tried to compile better-sqlite3 from source	Make sure `pnpm-workspace.yaml` contains `better-sqlite3: false` under `allowBuilds` (prebuilt binaries are included)
`Could not locate the bindings file` / `NODE_MODULE_VERSION mismatch` (better-sqlite3)	Node.js version changed after installing	Delete `node_modules` and run `pnpm install` again
Page shows **Cannot reach the server. Is it running?**	The server is stopped or crashed	Look at the terminal; restart with `pnpm start`
Page is blank, Console shows an error in `app.js`	A JavaScript mistake in `public/js/`	Read the Console message and line number; browsers do not type-check plain JS
Changes to `src/` have no effect	Server started with `pnpm start`	Restart it, or use `pnpm dev` which restarts automatically
`Route POST /api/auth/register was not found.` (or any `Route … was not found` for a route that exists in the code)	An **old server process** is still running code from before the route was added. Node.js loads the code once at start-up	Stop every running copy (`Ctrl+C` in its terminal; on Windows, `netstat -ano | findstr :3000` shows the process ID, then `taskkill /PID <id> /F`) and start again with `pnpm start`

Symptom	Likely cause	Solution
Changes to `public/` have no effect	Browser cache	Hard-reload with `Ctrl+Shift+R` (or `Cmd+Shift+R`)
`400 VALIDATION_ERROR`	The request data broke a rule	Read `error.details` — it lists each field and problem
`SqliteError: no such column` / `no such table`	Schema changed but no migration was generated/applied	`pnpm db:generate`, then restart the server
`FOREIGN KEY constraint failed`	Inserting a non-existent `category_id` outside the service	Use the service functions, which check first
`database is locked`	Another program (e.g. a DB viewer) holds a write lock	Close the other program or its open transaction
Overdue / due today counts look wrong	Time-zone mismatch	The browser sends `today`; check your computer's date and time zone
The app always shows the login screen / `401 UNAUTHORIZED`	No session cookie, or it expired (7 days), or you logged out	Log in again. With `COOKIE_SECURE=true` on plain `http://` the browser refuses to store the cookie — use `false` locally
`Incorrect email or password.`	Typo, or wrong account	Check Caps Lock; emails are not case-sensitive, passwords are
`429 Too many failed login attempts`	5 wrong passwords for that email within 15 minutes	Wait 15 minutes, or restart the server (the counter is kept in memory)
"What is the admin password?"	There is no default admin	The first account you register is the admin — its email and password are the ones you chose (Chapter 20, "What is the admin's email and password?")
`403 FORBIDDEN` on `/api/admin/users`	You are not an admin	Only the first account is an admin. To promote someone, change `role` to `admin` in the `users` table with a DB viewer
`403` from `curl -X DELETE …` or `POST …/complete`	CSRF protection: requests without a JSON body need an `Origin` header	Add `-H "Origin: http://localhost:3000"`
My old tasks disappeared after updating	Old tasks have no assignee yet	The **first** account that registers claims them. If someone else registered first, add rows to `task_assignees` (and set `categories.user_id`) with a DB viewer
As admin, a task I assigned to someone is not in my **Tasks** view	The Tasks view shows tasks assigned **to you**; you created it for someone else	Open the **Admin** tab: it lists every task, with creator and assignees
No **Delete** button on a group task	Only the task's creator or an admin may delete it	Ask the creator or an admin; you can still edit, complete and use subtasks
A task is missing from the **Calendar**	It has no due date, or its date is outside what the current view type shows (another month/week/day, or more than 30 days ahead in Agenda)	Give it a due date; use ‹ ›, **Today**, or switch to Month/Agenda

Symptom	Likely cause	Solution
The calendar always opens in Week (or Day/Agenda) view	The last chosen view type is remembered in the browser (`localStorage`)	Click **Month** — that choice is remembered instead
A migration deleted data / "broken foreign key reference(s)" on start	A table rebuild ran with foreign keys on, or a migration left invalid references	`runMigrations` prevents the first and detects the second. Restore from `data/backups/` and report the problem (Chapter 7)
Forgot my password	There is no "reset password" feature yet	Stop the server and delete your user row (your tasks are deleted too!), or add a reset feature (Chapter 16 exercise)
`Invalid environment variables` on start	A value in `.env` is wrong (e.g. `PORT=abc`)	Fix the value; the message names the variable
`pnpm test:e2e` or `pnpm docs:build`: `No Chrome, Edge or Chromium browser found`	No supported browser at the usual locations	Install Chrome/Edge or set `CHROME_PATH` to the browser's executable
TypeScript error `Property 'x' does not exist on type …`	Misspelt property, or a type that needs updating	Check the spelling; hover over the variable in VS Code to see its real type

Reading a stack trace

When an unexpected error happens, the terminal prints a **stack trace** — the list of function calls that led to the error, most recent first:

```
Unexpected error: TypeError: Cannot read properties of undefined (reading 'title')
    at createTask (src/services/task.service.ts:96:31)
    at <anonymous> (src/routes/task.routes.ts:29:28)
```

Read the first line (what went wrong), then the first line that points at *your* file (where). Here: line 96 of `task.service.ts`.

Starting fresh

To reset all data (including all accounts), stop the server and delete the `data/` folder. It is recreated (empty) on the next start. The next account you register becomes the admin. Run `pnpm db:seed` for a demo account with example data.

Chapter 16 — Modifying the Application

This chapter teaches the recipes for the most common changes. Each recipe lists the files to touch **in order**, following the flow of data from the database up to the screen.

The golden workflow

```
1. Understand: which layers does this change touch?
   Security: who may use it? (logged in? admin only? only own data → userId)
2. Database   → src/db/schema.ts, then pnpm db:generate
3. Validation → src/validators/*.ts
4. Service    → src/services/*.ts
5. Route      → src/routes/*.ts (and app.ts for a new route group)
6. Frontend   → public/index.html, public/js/api.js, ui.js, app.js
7. Tests      → tests/*.test.ts
8. Check      → pnpm typecheck && pnpm test && pnpm test:e2e
9. Document   → docs/DOCUMENTATION-SOURCE.md + CHANGELOG.md, pnpm docs:build
```

Recipe: add a new database field (example: `notes`)

1. **Schema** — in `src/db/schema.ts`, inside `tasks`, add:

   ```ts
   notes: text('notes').notNull().default(''),
   ```

2. **Migration** — run `pnpm db:generate --name add_task_notes`. Open the new SQL file and read it: it should contain `ALTER TABLE tasks ADD notes text DEFAULT '' NOT NULL`.
3. **Validation** — in `src/validators/task.validator.ts` add a rule `const notes = z.string().trim().max(5000);`, then add `notes: notes.default('')` to `createTaskSchema` and `notes: notes.optional()` to `updateTaskSchema`.
4. **Service** — nothing to do! `createTask` and `updateTask` spread the validated input (`...input`) into the query, and `Task` types update automatically from the schema. (They already add `userId` themselves, so the new field is automatically per-user.)
5. **Frontend** — add a `<textarea name="notes">` to the task form in `index.html`; read it in `readTaskForm()` and fill it in `openEditTask()` (`app.js`); show it in `createTaskItem()` (`ui.js`).
6. **Tests** — add a test that creates a task with notes and reads them back.
7. Restart the server: the migration runs automatically.

Recipe: add a new filter (example: "created this week")

1. Add the new value to `DUE_FILTERS` (or a new query field) in `task.validator.ts`.

2. Add a condition in `listTasks` (`task.service.ts`), for example `conditions.push(gte(tasks.createdAt, startOfWeekIso))`.
3. Add an `<option>` to the right `<select>` in `index.html`. `app.js` already sends dropdown values as query parameters.
4. Add a test in the "search, filter, sort" group.

Recipe: add a new API endpoint (example: delete all completed tasks)

1. **Service** (`task.service.ts`):

```ts
// ALWAYS limit it to the user: without that, one user would delete everybody's tasks!
// Here: completed tasks the viewer created (the same rule as single deletes).
export function deleteCompletedTasks(db: AppDatabase, viewer: Viewer): number {
  const result = db.delete(tasks).where(and(eq(tasks.createdById, viewer.id), eq(tasks.status,
'completed'))).run();
  return result.changes; // number of deleted rows
}
```

2. **Route** (`task.routes.ts`) — put it **before** routes with `/:id` if its path could be confused with an ID:

```ts
taskRoutes.delete('/completed', (c) => {
  const deleted = taskService.deleteCompletedTasks(c.get('db'), c.get('user'));
  return c.json({ data: { deleted } });
});
```

3. **Frontend** — add `export const deleteCompleted = () => request('/api/tasks/completed', { method: 'DELETE' });` to `api.js`, a button to `index.html`, and a click handler in `app.js` that confirms, calls it, shows a toast and calls `refresh()`.
4. **Test** it and **document** it in Chapter 12.

Recipe: add a new route group / new database table (example: comments on tasks)

Subtasks (Development Entry #6) were built exactly this way — use `subtask.*` files as your template.

1. **Schema** — a new table with foreign keys to `tasks` and `users`:

```ts
export const comments = sqliteTable('comments', {
  id: integer('id').primaryKey({ autoIncrement: true }),
  taskId: integer('task_id').notNull().references(() => tasks.id, { onDelete: 'cascade' }),
  authorId: integer('author_id').references(() => users.id, { onDelete: 'set null' }),
  body: text('body').notNull(),
```

```
  createdAt: text('created_at').notNull(),
});
```

`onDelete: 'cascade'` deletes comments when their task is deleted.

2. `pnpm db:generate --name add_comments`. Read the SQL; if it adds a foreign-key column to an *existing* table, check the `ON DELETE` rule is there (Chapter 7).
3. Create `src/validators/comment.validator.ts`, `src/services/comment.service.ts`, `src/routes/comment.routes.ts`, copying the subtask files. Start the routes file with `commentRoutes.use('*', requireAuth);`, and in the service **check that the viewer may see the parent task** by calling `getAccessibleTask(db, viewer, taskId)` first — it throws 404 otherwise.
4. Mount it in `src/app.ts`: `app.route('/api/comments', commentRoutes);`
5. Add `tests/comments.test.ts` — including "another user gets 404".

Recipe: add a new page

Our app is a **single-page application**: one HTML page whose content JavaScript changes. To add a separate page (for example an "About" page):

1. Create `public/about.html` (copy the `<head>` from `index.html` for the same styles).
2. Link to it: `<a href="/about.html">About</a>`. `serveStatic` serves it automatically.
3. If it needs data, create `public/js/about.js` and load it with `<script type="module" src="/js/about.js">`, reusing functions from `api.js`.

Recipe: add a new frontend interaction (example: a "Start" button that sets status to in_progress)

1. In `createTaskItem()` (`ui.js`), create a button shown only when `task.status === 'pending'` and call `handlers.onStart(task)` on click.
2. In `app.js`, write `async function startTask(task) { await api.updateTask(task.id, { status: 'in_progress' }); showToast('Task started'); await refresh(); }` with a `try/catch`, and pass `onStart: startTask` to `renderTaskList`.
3. No backend change needed: `PATCH /api/tasks/:id` already accepts `status`.

Ideas for bigger features

- **Change password** — `PATCH /api/auth/password` with `{ currentPassword, newPassword }`: verify the current one with `verifyPassword`, store `hashPassword(newPassword)`, and delete the user's *other* sessions.
- **Admin user management** — `PATCH /api/admin/users/:id` to change a role, `DELETE /api/admin/users/:id` (the `ON DELETE CASCADE` rules then remove that user's sessions, tasks and categories). Prevent an admin from removing the last admin.

- **Password reset by email** — needs an email-sending service and single-use, expiring reset tokens (store only their hashes, like sessions).
- **Pagination** — `limit` and `offset` query parameters for very long lists.
- **Tags** — a many-to-many relationship (a `tags` table plus a `task_tags` linking table).
- **Recurring tasks** — a `repeat` field and logic that creates the next task when one is completed.

Chapter 17 — Deployment

Deployment means running your application on a server that other people can reach, usually on the internet.

> **WARNING**
>
> The app has accounts and per-user data (Chapter 20), but anyone who can reach it can **register**. Before deploying publicly: serve it over **HTTPS**, set `COOKIE_SECURE=true`, register your own (admin) account first — so a stranger cannot become the admin — and delete or change the seeded demo account if you created one.

What production needs

Need	How it applies to this app
A computer that is always on	A VPS (virtual private server), or a platform such as Railway, Render or Fly.io
Node.js 22+ and pnpm	Install them on the server, or use a Docker image
Environment variables	Set `PORT`, `DATABASE_PATH`, `LOG_REQUESTS` and `COOKIE_SECURE=true` in the platform's settings
Persistent storage	The SQLite file must live on a disk that survives restarts and redeploys (a "volume")
HTTPS	Usually provided by the platform or a reverse proxy (Caddy, Nginx)
Restart on crash	The platform, `systemd`, or a process manager like `pm2`
Backups	Regularly copy the database file

Build process

Our app does not need a separate build step: `tsx` runs the TypeScript directly, which is why `tsx` is a normal dependency rather than a development dependency. The deployment steps are:

```bash
pnpm install --prod --frozen-lockfile   # install exactly the locked runtime packages
pnpm start                              # runs migrations, then serves on $PORT
```

`--frozen-lockfile` makes the install fail if `package.json` and `pnpm-lock.yaml` disagree, which guarantees production uses exactly the tested versions. `--prod` skips development tools such as Vitest.

> **NOTE**
> Larger projects often compile TypeScript to JavaScript ahead of time (`tsc` with output enabled) and run the result with plain `node`, which starts slightly faster. For this project the simplicity of `tsx` is the better trade-off; Chapter 18 lists this as a later learning step.

Database handling

- Migrations run automatically on every start (`runMigrations(db)` in `index.ts`), so deploying a new version with a new migration updates the database structure.
- Point `DATABASE_PATH` at a persistent volume, for example `DATABASE_PATH=/data/todo.db`.
- Run only **one** instance of the app with SQLite (one file, one writer). If you need several servers, move to a database server such as PostgreSQL (Drizzle supports it with a different driver and `dialect: 'postgresql'`).
- Back up by copying `todo.db` while the app is stopped, or use SQLite's online backup (`sqlite3 todo.db ".backup backup.db"`).

Server configuration example (Linux VPS)



A minimal `Caddyfile`:

```
todo.example.com {
    reverse_proxy localhost:3000
}
```

Your code on GitHub (Git basics)

Since Development Entry #8 the project lives in its own **Git repository**, published on **GitHub** at **https://github.com/minal-aetonet/aeto-to-do** (a **private** repository: only people you invite can see it; you can make it public in the repository's *Settings*).

- **Git** is a *version control system*: it records snapshots ("commits") of your files, so you can see what changed, when and why, and go back if something breaks.

- **GitHub** is a website that stores Git repositories online — a backup, a place to share code and the starting point of many deployment platforms.
- A **remote** is the online copy (named `origin`); **push** sends your commits there, **pull** fetches other people's commits, **clone** downloads a repository for the first time.

The everyday workflow:

```bash
git status                      # what changed?
git add .                       # choose all changes for the next snapshot
git commit -m "Add week view"   # take the snapshot, with a message saying WHY
git push                        # upload the commits to GitHub
git log --oneline               # list earlier snapshots
```

`.gitignore` decides what is **never** committed: `node_modules/` (recreated by `pnpm install`), `data/` (your database and backups — real user data must never be published), `.env` (secrets) and database files. Before the first push, the file list was checked to make sure none of these were included.

> **WARNING**
> Anything pushed to GitHub should be treated as published — even in a private repository, and even if you delete it later. Never commit passwords, tokens or `.env` files. If it happens, change the secret immediately.

Deployment checklist

- [ ] Your own admin account registered first; demo account removed
- [ ] `pnpm typecheck`, `pnpm test`, `pnpm test:e2e` all pass
- [ ] Environment variables set on the server, including `COOKIE_SECURE=true`
- [ ] `DATABASE_PATH` on persistent storage
- [ ] HTTPS working
- [ ] Automatic restart configured
- [ ] Backups scheduled
- [ ] `pnpm audit` shows no serious vulnerabilities

Chapter 18 — Learning Roadmap

This roadmap takes you from absolute beginner to building your own Hono applications. Do not rush; each stage builds on the previous one.

Stage 1 — Read and run (week 1–2)

- [ ] Run the app, create and complete tasks, open DevTools.
- [ ] Read Chapters 1–5. Do every "Try it yourself".
- [ ] Follow Flow 2 (creating a task) through the real files with your finger on the screen.
- **Goal:** explain in your own words what happens when you click "Create task".

Stage 2 — Small changes (week 3–4)

- [ ] Change colours and texts in the frontend.
- [ ] Add a `GET /api/hello` route (Chapter 2).
- [ ] Write your own middleware (Chapter 6).
- [ ] Change a validation rule (e.g. title max 100) and update the test that breaks.
- **Goal:** make a change, see a test fail, fix it.

Stage 3 — Full features (month 2)

- [ ] Add the `notes` field (Chapter 16 recipe) — touches every layer.
- [ ] Add the "Start" button interaction.
- [ ] Add the "delete all completed" endpoint with a test.
- [ ] Learn basic **git**: `git init`, `git add`, `git commit`, `git log`, branches.
- **Goal:** add a feature from database to screen, with tests, by yourself.

Stage 4 — Deeper understanding (month 3)

- [ ] Learn more SQL: joins, grouping, indexes. Try queries in a SQLite viewer.
- [ ] Read the Hono documentation (hono.dev): routing, middleware list, helpers.
- [ ] Read the Zod and Drizzle documentation.
- [ ] Add comments on tasks (a new table and route group, Chapter 16) — protected with `requireAuth` and checked with `getAccessibleTask`.
- [ ] Read Chapter 21 and trace how a group task reaches each member.
- [ ] Read Chapter 20 twice. Then add "change password" (Chapter 16 ideas) with tests.
- **Goal:** design a new table and its API without a recipe.

Stage 5 — Build your own app (month 4+)

- ☐ Start an empty folder: `pnpm init`, `pnpm add hono @hono/node-server`, create `src/index.ts` with a single route.
- ☐ Build something you care about: a reading list, a habit tracker, a recipe book.
- ☐ Copy the *structure* of this project (routes/services/validators/db), not the code.
- ☐ Add authentication the way Chapter 20 does it (users, sessions, `requireAuth`, `user_id` in every query), then deploy it (Chapter 17).
- **Goal:** a working app of your own on the internet.

Further topics to explore

Topic	Why
Hono RPC (`hc` client)	Type-safe API calls from the frontend
Hono JSX	Render HTML on the server with Hono
A frontend framework (React, Vue, Svelte)	For larger interfaces
PostgreSQL	Multi-user production databases
Docker	Package the app with its environment
OAuth / "Log in with Google"	Let users log in with existing accounts instead of passwords
The OWASP Top 10	The ten most common web security risks — a checklist for every app
Compiling TypeScript (`tsc` build)	Faster production start-up
Deploying to Cloudflare Workers	Hono's "runs everywhere" superpower

Chapter 19 — How This Manual Is Built

This manual is generated automatically so that the HTML and PDF versions can never drift apart.

The files

File	Role
`docs/DOCUMENTATION-SOURCE.md`	**The source of truth.** All chapters, written in Markdown. Edit this.
`docs/CHANGELOG.md`	Every change request, newest first. Becomes the final appendix.
`scripts/build-docs.ts`	The generator.
`docs/LEARN-HONO-COMPLETE.html`	Generated. Do not edit by hand.
`docs/LEARN-HONO-COMPLETE.pdf`	Generated. Do not edit by hand.

Markdown is a simple text format: `# Heading`, `**bold**`, `- list item`, `` `code` ``, tables with `|`, and code blocks fenced with three back-ticks.

The command

```bash
pnpm docs:build
```

What the generator does

```
DOCUMENTATION-SOURCE.md + CHANGELOG.md
            ↓  read both files; compute a "build id" (fingerprint of their contents)
marked (Markdown → HTML), with custom rules:
  • headings get ids, and chapters/sections are collected for the table of contents
  • code blocks are coloured by highlight.js at build time (no internet needed)
  • ```diagram blocks become framed text diagrams
  • "> [!NOTE]" / TIP / WARNING / IMPORTANT quotes become coloured callout boxes
  • tables are wrapped so they scroll on phones and fit on paper
            ↓
One standalone HTML file: cover page, sidebar contents with search,
embedded CSS + JavaScript, print styles  →  LEARN-HONO-COMPLETE.html
            ↓
puppeteer-core opens THAT file in headless Chrome/Edge and prints it:
  A4, margins, header, footer with "Page X of Y" + build id,
  PDF bookmarks from the headings                  →  LEARN-HONO-COMPLETE.pdf
            ↓
Verification: the HTML loaded with no script errors or dialogs; the PDF
starts with %PDF and has pages; the browser saw every chapter/section
heading; the build id in the loaded HTML matches the source.
Any problem → the command fails with an explanation.
```

Because the PDF is printed from the very HTML file that was just written, both always contain identical content. The **build id** printed on the cover of both formats (and in every PDF footer) lets you confirm they come from the same build.

Design decisions

- **Text diagrams instead of Mermaid.** Mermaid diagrams need a large JavaScript library at view time. Plain-text diagrams display identically in the Markdown source, the HTML (offline) and the PDF, and are easy to edit.
- **Local browser instead of a downloaded one.** `puppeteer-core` uses the Chrome or Edge already installed (`scripts/lib/browser.ts`), avoiding a ~150 MB download. Set `CHROME_PATH` to choose a specific browser.
- **Search** in the HTML sidebar filters chapters and sections by their full text, using an index embedded at build time. It works offline.

Updating the manual

1. Edit `docs/DOCUMENTATION-SOURCE.md` (and add a `CHANGELOG.md` entry).
2. Run `pnpm docs:build`.
3. Open the HTML in a browser and skim the PDF.

Chapter 20 — Authentication and Authorization

Two different questions

These two words sound alike but answer different questions:

	Question	Real-life example	In our app	Failure status
Authentication ("authN")	*Who are you?*	Showing your ID card at a hotel reception	Logging in with email + password; the session cookie	`401 Unauthorized`
Authorization ("authZ")	*What are you allowed to do?*	Your key card opens **your** room, not every room	You can only touch your own tasks; only admins see the user list	`403 Forbidden` (or `404` to hide data)

Authentication always comes first: you cannot decide what someone may do before you know who they are.

> **NOTE**
> The name "401 Unauthorized" is a historical mistake in the HTTP standard — it really means "unauthenticated". `403 Forbidden` is the real "not authorized".

What we built

Piece	File	Job
`users` table	`src/db/schema.ts`	One row per account (name, email, password **hash**, role)
`sessions` table	`src/db/schema.ts`	One row per logged-in browser (token **hash**, user, expiry)
Password hashing	`src/lib/password.ts`	`hashPassword`, `verifyPassword` with scrypt
Login throttle	`src/lib/login-throttle.ts`	Blocks password guessing
Auth validators	`src/validators/auth.validator.ts`	Rules for register/login input
Auth service	`src/services/auth.service.ts`	Register, check password, create/find/delete sessions, list users
Auth middleware	`src/middleware/auth.ts`	`requireAuth`, `requireRole`, cookie helpers
Auth routes	`src/routes/auth.routes.ts`	`/api/auth/register`, `/login`, `/logout`, `/me`
Admin routes	`src/routes/admin.routes.ts`	`/api/admin/users` (admins only)

Piece	File	Job
Ownership	all services	`user_id = <you>` in every task/category/dashboard query
Frontend	`public/index.html`, `public/js/*.js`	Login/register screen, logout, admin-only Admin tab

The users and sessions tables

```ts
export const USER_ROLES = ['user', 'admin'] as const;

export const users = sqliteTable('users', {
  id: integer('id').primaryKey({ autoIncrement: true }),
  name: text('name').notNull(),
  // Always stored in lower case, so "Sam@Mail.com" and "sam@mail.com" are the same account.
  email: text('email').notNull().unique(),
  // NEVER the password itself: a scrypt hash like "scrypt:<salt>:<hash>".
  passwordHash: text('password_hash').notNull(),
  role: text('role', { enum: USER_ROLES }).notNull().default('user'),
  createdAt: text('created_at').notNull(),
});

export const sessions = sqliteTable(
  'sessions',
  {
    // The SHA-256 hash of the random token stored in the browser's cookie.
    id: text('id').primaryKey(),
    userId: integer('user_id')
      .notNull()
      .references(() => users.id, { onDelete: 'cascade' }),
    expiresAt: text('expires_at').notNull(),
    createdAt: text('created_at').notNull(),
  },
  (table) => [index('sessions_user_idx').on(table.userId)],
);
```

A user can have **many sessions** — one per browser or device where they are logged in.

Passwords: never store them

If the database stored passwords as plain text, anyone who got a copy of the file (a thief, a careless backup, a curious developer) would know every user's password — and, because people reuse passwords, could log into their email and bank too.

Instead we store a **hash**. A hash function turns input into a fixed-looking jumble, and it only works in one direction:

```
"correct-horse"  ──hash──►  "a91f…7c2e"      easy
"a91f…7c2e"      ──?─────►  "correct-horse"  impossible (except by guessing)
```

To check a login, we hash the password the user typed *the same way* and compare the two hashes.

Two extra ingredients make hashes safe for passwords:

- **Salt** — 16 random bytes created for *each* password and stored next to the hash. Two users with the same password get completely different hashes, and attackers cannot use pre-computed tables of common passwords.
- **Slowness** — normal hash functions (like SHA-256) are designed to be fast: billions per second. For passwords that is bad, because an attacker can try billions of guesses. **scrypt** is deliberately slow and uses lots of memory, so each guess is expensive. (Registering takes about 70 ms in our logs — mostly scrypt.)

`src/lib/password.ts`:

TS

```ts
const scryptAsync = promisify(scrypt) as (password: string, salt: Buffer, keyLength: number) =>
Promise<Buffer>;

const KEY_LENGTH = 64;

export async function hashPassword(password: string): Promise<string> {
  const salt = randomBytes(16);
  const hash = await scryptAsync(password, salt, KEY_LENGTH);
  return `scrypt:${salt.toString('hex')}:${hash.toString('hex')}`;
}

export async function verifyPassword(password: string, stored: string): Promise<boolean> {
  const [algorithm, saltHex, hashHex] = stored.split(':');
  if (algorithm !== 'scrypt' || !saltHex || !hashHex) return false;

  const expected = Buffer.from(hashHex, 'hex');
  const actual = await scryptAsync(password, Buffer.from(saltHex, 'hex'), expected.length);
  return timingSafeEqual(actual, expected);
}
```

Line by line:

1. `scrypt` comes from Node's built-in `node:crypto` module — no extra package needed. It reports its result through a *callback*; `promisify` turns it into a function that returns a **Promise**, so we can `await` it.
2. `randomBytes(16)` — a fresh random salt.
3. The stored string has three parts separated by `:` — the algorithm name, the salt and the hash, written in **hex** (digits 0–9 and letters a–f). Storing the algorithm name lets us switch algorithms in the future.
4. `verifyPassword` splits the stored string, hashes the typed password with the **same salt**, and compares.

5. `timingSafeEqual` compares two byte sequences in **constant time**. A normal comparison stops at the first different byte, so it finishes slightly faster when the first bytes differ — an attacker measuring response times could learn something. Constant-time comparison removes that clue.

> **IMPORTANT**
> Never invent your own password hashing. Use a well-known slow algorithm (scrypt, bcrypt, Argon2) with a random salt, exactly as here.

Sessions and cookies: staying logged in

HTTP has no memory: every request stands alone. After you log in, how does the server know the *next* request is from you? With a **session**:

```
LOGIN
1. Server creates a random token:   "kX9…QpE"  (32 random bytes)
2. Server stores sha256(token) in the sessions table, with user_id and expiry
3. Server answers:  Set-Cookie: session=kX9…QpE; HttpOnly; SameSite=Lax; Path=/; Expires=…
4. Browser stores the cookie

EVERY LATER REQUEST
5. Browser sends:   Cookie: session=kX9…QpE        (automatically!)
6. requireAuth: sha256(token) → find session → not expired? → load the user
7. c.set('user', user) → the route knows who you are
```

`src/services/auth.service.ts`:

TS

```ts
function hashToken(token: string): string {
  return createHash('sha256').update(token).digest('hex');
}

export function createSession(db: AppDatabase, userId: number): { token: string; expiresAt: Date } {
  const token = randomBytes(32).toString('base64url');
  const expiresAt = new Date(Date.now() + SESSION_TTL_MS);

  // Housekeeping: remove sessions that have already expired.
  db.delete(sessions).where(lte(sessions.expiresAt, nowIso())).run();

  db.insert(sessions)
    .values({ id: hashToken(token), userId, expiresAt: expiresAt.toISOString(), createdAt: nowIso() })
    .run();
  return { token, expiresAt };
}

export function getUserBySessionToken(db: AppDatabase, token: string): PublicUser | null {
  const row = db
    .select({ user: users })
    .from(sessions)
    .innerJoin(users, eq(sessions.userId, users.id))
    .where(and(eq(sessions.id, hashToken(token)), gt(sessions.expiresAt, nowIso())))
    .get();
```

```
  return row ? toPublicUser(row.user) : null;
}
```

- **32 random bytes** is a number so large that guessing a valid token is impossible in practice. `base64url` writes it with letters, digits, `-` and `_`, safe for cookies.
- **Why store only a hash of the token?** If someone stole the database, they would see only hashes and still could not log in as anyone. (Here a fast hash, SHA-256, is fine: the token is already completely random, so there is nothing to guess — unlike human-chosen passwords.)
- `SESSION_TTL_MS` is 7 days. Expiry is checked in the query: `expires_at > now`. ISO timestamps compare correctly as text.
- `innerJoin` returns a row only if both the session and its user exist.
- `toPublicUser` copies only safe fields — the password hash never leaves the service.

The cookie and its flags

`src/middleware/auth.ts`:

TS

```ts
export function setSessionCookie(c: Context<AppEnv>, token: string, expiresAt: Date): void {
  setCookie(c, SESSION_COOKIE, token, {
    httpOnly: true,
    sameSite: 'Lax',
    secure: c.get('auth').secureCookies,
    path: '/',
    expires: expiresAt,
  });
}
```

Flag	Meaning	Protects against
`HttpOnly`	Page JavaScript (`document.cookie`) cannot read the cookie	XSS stealing the login
`SameSite=Lax`	The browser does not send the cookie on requests *started by other websites* (except simple link clicks)	CSRF
`Secure`	Only sent over HTTPS (when `COOKIE_SECURE=true`)	Eavesdropping on public Wi-Fi
`Path=/`	Sent for every path on our site	—
`Expires`	The browser deletes it after 7 days	Forgotten logins lasting forever

`setCookie`, `getCookie` and `deleteCookie` come from Hono's cookie helper, `hono/cookie`.

Why sessions and not JWT?

You will often read about **JWT** (JSON Web Tokens): signed tokens that contain the user's data, so the server does not need a sessions table.

	Database sessions (our choice)	JWT
Logout	Delete the row — takes effect **immediately**	Token stays valid until it expires (needs extra "deny lists")
Server storage	One small row per login	None
Simplicity for a single server	Very simple	More concepts (signing keys, refresh tokens)
Best for	Classic websites like ours	Many separate services that cannot share a database

For one server with a database, sessions are simpler and safer. Hono also has JWT helpers (`hono/jwt`) if you need them later.

The authentication middleware

```ts
export const requireAuth = createMiddleware<AppEnv>(async (c, next) => {
  const token = getCookie(c, SESSION_COOKIE);
  const user = token ? getUserBySessionToken(c.get('db'), token) : null;
  if (!user) {
    throw new AppError('You must be logged in to do this.', 401, 'UNAUTHORIZED');
  }
  c.set('user', user);
  await next();
});

export function requireRole(role: UserRole) {
  return createMiddleware<AppEnv>(async (c, next) => {
    if (c.get('user').role !== role) {
      throw new AppError('You do not have permission to do this.', 403, 'FORBIDDEN');
    }
    await next();
  });
}
```

- `requireAuth` is a ready-made middleware (a constant). No cookie, unknown token, or expired session → `401`. Otherwise the user is stored on the context for every later middleware and handler.
- `requireRole('admin')` is a factory: it *returns* a middleware for the given role. It must run after `requireAuth`.
- Both **throw** instead of returning a response; the central error handler (Chapter 6) turns the error into JSON.

Where they are used:

```ts
// src/routes/task.routes.ts  (same in category.routes.ts and dashboard.routes.ts)
taskRoutes.use('*', requireAuth);

// src/routes/admin.routes.ts
adminRoutes.use('*', requireAuth, requireRole('admin'));
```

```
// src/routes/auth.routes.ts — only /me is protected
authRoutes.get('/me', requireAuth, (c) => {
  return c.json({ data: c.get('user') });
});
```

Flow A: Registering

```
Register form → submitRegister() → api.register(name, email, password)
       ↓
POST /api/auth/register  {"name":"Sam","email":"Sam@Example.com","password":"…"}
       ↓
validate('json', registerSchema): trim, lower-case email, password length 8–128
       ↓
registerUser(db, input):
  email already used?  → 409 CONFLICT
  passwordHash = await hashPassword(password)          (slow, ~70 ms)
  TRANSACTION:
    is this the first user?  → role 'admin', else 'user'
    INSERT INTO users …
    first user? → assign every task that has no assignee to the new user,
                  make them creator of tasks without a creator,
                  and owner of categories without an owner
       ↓
createSession(db, user.id) → token
setSessionCookie(c, token, expiresAt)
       ↓
201 {"data":{id,name,email,role,createdAt}}  +  Set-Cookie: session=…
       ↓
showApp(user): name in the top bar, tasks loaded
```

The registration code:

TS

```
export async function registerUser(db: AppDatabase, input: RegisterInput): Promise<PublicUser> {
  const existing = db.select({ id: users.id }).from(users).where(eq(users.email, input.email)).get();
  if (existing) {
    throw new ConflictError('An account with this email already exists.');
  }

  // Hashing is slow on purpose, so we do it before opening the transaction.
  const passwordHash = await hashPassword(input.password);

  // A transaction: either ALL of these changes are saved, or none of them.
  return db.transaction((tx) => {
    const isFirstUser = tx.select({ value: count() }).from(users).get()!.value === 0;
    const user = tx
      .insert(users)
      .values({ name: input.name, email: input.email, passwordHash, role: isFirstUser ? 'admin' :
'user', createdAt: nowIso() })
      .returning()
      .get();

    if (isFirstUser) {
      // Tasks with nobody assigned (from before accounts existed) go to the first user.
```

```
      const ownerless = tx
        .select({ id: tasks.id })
        .from(tasks)
        .where(sql`${tasks.id} NOT IN (SELECT ${taskAssignees.taskId} FROM ${taskAssignees})`)
        .all();
      if (ownerless.length > 0) {
        const assignedAt = nowIso();
        tx.insert(taskAssignees)
          .values(ownerless.map((task) => ({ taskId: task.id, userId: user.id, assignedAt })))
          .run();
      }
      tx.update(tasks).set({ createdById: user.id }).where(isNull(tasks.createdById)).run();
      tx.update(categories).set({ userId: user.id }).where(isNull(categories.userId)).run();
    }
    return toPublicUser(user);
  });
}
```

A **transaction** groups several database changes so they succeed or fail *together*. If anything inside throws, SQLite undoes all of it — we can never end up with a user who is half-created, or tasks claimed by a user who does not exist.

Flow B: Logging in

TS

```
authRoutes.post('/login', validate('json', loginSchema), async (c) => {
  const db = c.get('db');
  const { email, password } = c.req.valid('json');
  const throttle = c.get('auth').loginThrottle;

  if (throttle.isBlocked(email)) {
    throw new AppError('Too many failed login attempts. Please try again in 15 minutes.', 429,
'TOO_MANY_ATTEMPTS');
  }

  const user = await authService.authenticate(db, email, password);
  if (!user) {
    throttle.recordFailure(email);
    // Same message for "no such email" and "wrong password" (see authenticate()).
    throw new AppError('Incorrect email or password.', 401, 'INVALID_CREDENTIALS');
  }

  throttle.reset(email);
  const session = authService.createSession(db, user.id);
  setSessionCookie(c, session.token, session.expiresAt);
  return c.json({ data: user });
});
```

And `authenticate` in the service:

TS

```
export async function authenticate(db: AppDatabase, email: string, password: string):
Promise<PublicUser | null> {
  const user = db.select().from(users).where(eq(users.email, email)).get();
```

```
  if (!user) {
    // Do the same slow work as a real check, so the response time does not
    // reveal that this email has no account.
    await hashPassword(password);
    return null;
  }
  const valid = await verifyPassword(password, user.passwordHash);
  return valid ? toPublicUser(user) : null;
}
```

Two deliberate details:

- **The same message** for an unknown email and a wrong password. A message like "no account with this email" would let attackers test which emails are registered ("user enumeration").
- **The same amount of work** in both cases. Without the dummy `hashPassword` call, unknown emails would answer ~70 ms faster, which also leaks the information.

> **NOTE**
> Honest limitation: the *registration* endpoint still reveals whether an email is taken (`409`), because users need to know that. Big sites solve this by sending a confirmation email instead of answering directly.

Slowing down password guessing

`src/lib/login-throttle.ts` counts failures per email in memory:

TS

```
isBlocked(key: string): boolean {
  const entry = this.failures.get(key);
  if (!entry) return false;
  if (Date.now() - entry.firstFailureAt > this.windowMs) {
    this.failures.delete(key); // the window has passed: forget the old failures
    return false;
  }
  return entry.count >= this.maxFailures;
}
```

- A `Map` stores `{ count, firstFailureAt }` per email.
- After **5** failures within **15 minutes**, even the correct password is refused (`429`), so an attacker cannot simply keep trying.
- A successful login calls `reset(email)` .
- It lives in memory: a server restart clears it, and with several servers each has its own counter (a shared store such as Redis would fix that). `app.ts` creates one `LoginThrottle` per app and puts it on the context with `injectAuthSettings` .

Flow C: Every protected request

```
GET /api/tasks   Cookie: session=kX9…QpE
      ↓
csrf → bodyLimit → injectDatabase → injectAuthSettings
      ↓
requireAuth:
  no cookie / unknown token / expired?  → 401 UNAUTHORIZED ─→ frontend: login screen
  otherwise                             → c.set('user', {id: 3, role: 'user', …})
      ↓
handler: taskService.listTasks(db, c.get('user'), query, 'mine')
      ↓
SELECT … FROM tasks … WHERE id IN
  (SELECT task_id FROM task_assignees WHERE user_id = 3) AND …   ← authorization
      ↓
200 { data: [only tasks assigned to user 3] }
```

Flow D: Logging out, and Flow E: an expired session

```ts
authRoutes.post('/logout', (c) => {
  const token = getCookie(c, SESSION_COOKIE);
  if (token) authService.deleteSession(c.get('db'), token);
  clearSessionCookie(c);
  return c.body(null, 204);
});
```

Logout deletes the session **row** (so the token is dead even if someone copied it — a test proves this) and tells the browser to delete the cookie.

When a session expires while the page is open, the next API call answers `401`. `api.js` notices (`onUnauthorized`), and `app.js` shows "Your session has ended. Please log in again." and the login screen.

Authorization: only your own data

Authentication tells us *who* is asking. Authorization makes sure they only touch rows they are entitled to. We do it in the **service layer**, by giving every function the **viewer** (the logged-in user's `id` and `role`) and checking access in **every** query. Since Development Entry #6 a task can be assigned to several people, so "your task" means "a task assigned to you" (Chapter 21 explains the `task_assignees` table):

```ts
// src/services/task.service.ts
export function getAccessibleTask(db: AppDatabase, viewer: Viewer, id: number): Task {
  const conditions: SQL[] = [eq(tasks.id, id)];
  if (viewer.role !== 'admin') {
    conditions.push(inArray(tasks.id, taskIdsAssignedTo(db, viewer.id)));
  }
  const task = db.select().from(tasks).where(and(...conditions)).get();
```

```
  if (!task) {
    throw new NotFoundError(`Task ${id} was not found.`);
  }
  return task;
}

export function deleteTask(db: AppDatabase, viewer: Viewer, id: number): void {
  const task = getAccessibleTask(db, viewer, id); // throws 404 if the viewer may not see it
  if (viewer.role !== 'admin' && task.createdById !== viewer.id) {
    throw new AppError('Only the person who created this task or an administrator can delete it.',
403, 'FORBIDDEN');
  }
  db.delete(tasks).where(eq(tasks.id, id)).run();
}
```

Two levels of protection are visible here: **seeing** a task (otherwise `404`) and, among those who can see it, **deleting** it (otherwise `403` — the user already knows the task exists, so `403` reveals nothing new).

Why in the services, not in each route? Because then it is impossible to forget: a service function *cannot be called* without a viewer — TypeScript would complain. If the check lived in routes, one forgotten line in one new route would leak data.

Why `404` and not `403` for someone else's task? A `403` would confirm "task 7 exists, but it isn't yours". A `404` reveals nothing. Attacks that try IDs one by one to reach other people's records are called **IDOR** (Insecure Direct Object Reference) — this app's tests prove they fail.

Categories too. A user may only put tasks into *their own* categories:

TS

```
function ensureCategoryExists(db: AppDatabase, userId: number, categoryId: number | null | undefined):
void {
  if (categoryId === null || categoryId === undefined) return;
  const found = db
    .select({ id: categories.id })
    .from(categories)
    .where(and(eq(categories.id, categoryId), eq(categories.userId, userId)))
    .get();
  if (!found) {
    throw new AppError(`Category ${categoryId} does not exist.`, 400, 'INVALID_CATEGORY');
  }
}
```

The dashboard counts only tasks assigned to you, and category names are unique *per user*.

Roles: user and admin

Every user has a `role`: `user` or `admin`. The **first account** ever registered becomes `admin` (see the transaction above); all later accounts are `user`.

Role-based authorization is done in two places: `requireRole('admin')` protects whole routes (`GET /api/admin/users` — accounts with task counts, without password hashes — and `GET`

`/api/admin/tasks`), and the task service checks `viewer.role` for finer rules (assigning, deleting, seeing every task, seeing who created a task — Chapter 21). In the frontend, the **Admin** tab is only shown to admins, but remember: **hiding a button is not security** — the server check is what counts. Our tests call the admin URLs directly as a normal user and expect `403`.

What is the admin's email and password?

There is no built-in admin account and no default password. A fixed password written in the code would be known to everyone who can read the code — a classic security hole. Instead:

1. Start the app (`pnpm start`) and open http://localhost:3000.
2. Click **Create one** and register with **your own** name, email and password.
3. If no account exists yet, that account becomes the **admin**. Your email and the password you just chose *are* the admin login.

Only for learning, `pnpm db:seed` can create a demo account — but only when no account exists yet, and it then becomes the admin:

Email	Password	Role
`demo@example.com`	`demo-password-123`	admin (only if it was the first account)

> **WARNING**
> Whichever account is created first becomes the admin **and** takes over all tasks that existed before accounts. If you want your old tasks in *your* account, register yourself **before** running `pnpm db:seed`.

Forgot the admin password? Passwords are stored only as hashes, so nobody — not even the developer — can read them back. Options: register a new account and promote it by changing its `role` to `admin` in the `users` table with a SQLite viewer, or build a "reset password" feature (Chapter 16).

Which account is the admin? Log in: admins see an **ADMIN** badge next to their name and an **Admin** tab. Or look at the `role` column of the `users` table.

	Visitor (not logged in)	`user`	`admin`
Register / log in	✓	—	—
Tasks assigned to me, own categories, dashboard, calendar	✗ 401	✓	✓
Edit / complete / subtasks on tasks assigned to me (incl. group tasks)	✗ 401	✓	✓
Delete a task	✗ 401	only tasks I created	✓ any
Assign tasks to other people (individual or group)	✗ 401	✗ 403	✓

	Visitor (not logged in)	`user`	`admin`
Tasks not assigned to me	✗ 401	✗ 404	✓ (Admin tab)
See who created a task	✗	✗ (never sent)	✓
`GET /api/admin/users`, `/api/admin/tasks`	✗ 401	✗ 403	✓

(Since Development Entry #6 — before that, admins could see only task counts of other users. Chapter 21 explains the change.)

CSRF: requests your browser sends for someone else

CSRF (Cross-Site Request Forgery): while you are logged in to our app, you visit an evil website. It contains a hidden form that automatically submits a POST to `http://localhost:3000/api/tasks/1/complete` (or a script that sends `fetch(..., { method: 'DELETE' })`). Your browser would normally attach your cookie — and the server would think *you* asked.

Our two layers of defence:

1. `SameSite=Lax` **cookie** — modern browsers do not attach the cookie to such cross-site form submissions or `fetch` calls.
2. **Hono's** `csrf()` **middleware** on `/api/*` — for POST/PATCH/DELETE requests whose body is a form, plain text, or missing, it checks the `Sec-Fetch-Site` and `Origin` headers that browsers add, and answers `403` unless the request comes from our own site. (Requests with a JSON body cannot be sent cross-site without the server's explicit permission, called CORS, which we never give.)

A test in `tests/app.test.ts` sends a DELETE with `Origin: https://evil.example` and the victim's cookie, and expects `403` — and the task still exists afterwards.

Upgrading an existing database

Tasks created before accounts existed have `user_id = NULL`. They are invisible to everyone (every query filters by `user_id`), until the **first** account registers and claims them all. That is what happened in testing with the 6 seeded tasks from Entry #1: the first registered account saw all 6.

Limitations and next steps

- No "forgot password" or "change password" yet (Chapter 16 has ideas).
- No email verification: anyone can register with any address.
- Admins can list users but not change roles or delete accounts through the UI.
- The login throttle is in memory (resets on restart; per server).

- Sessions last a fixed 7 days; they are not extended while you use the app.

Try it yourself

1. Log in, open DevTools → **Application** → **Cookies** → `http://localhost:3000`. Find the `session` cookie and notice the **HttpOnly** tick. Then type `document.cookie` in the Console — the session cookie is not there!
2. Open a private/incognito window, register a second account, and confirm you see none of the first account's tasks.
3. Open `data/todo.db` in a SQLite viewer. Look at `users.password_hash` and `sessions.id`. Can you find your password or your cookie value anywhere? (You should not.)
4. Log in with a wrong password six times and read the message you get.
5. Exercise: add `PATCH /api/auth/password` (change password). Write a test first.

Chapter 21 — Teamwork: Assigning Tasks, Group Tasks, Subtasks and Views

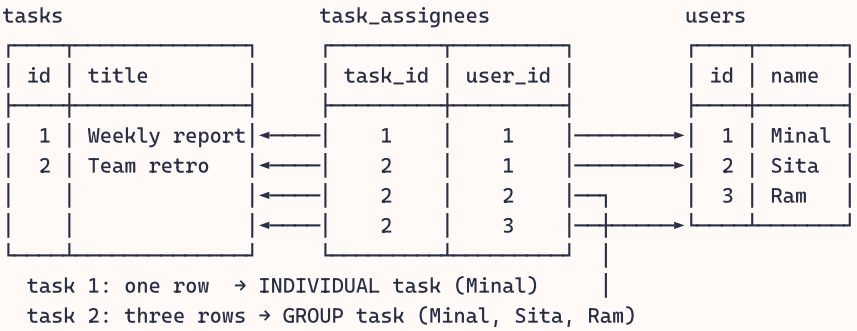
Development Entry #6 turned the personal to-do list into a small team tool. This chapter explains the five new features and the ideas behind them.

Feature	What it means	Who
Assigning	When creating or editing a task, choose who it is for	Admins
Individual / group tasks	One assignee = individual; several = group. Every member sees, edits and completes the same task	Admins create group tasks; members work on them
Subtasks	A checklist inside a task ("Book flights", "Book hotel")	Everyone who can see the task
Views	Separate **Dashboard**, **Tasks**, **Calendar** and **Admin** tabs	Admin tab: admins only
Created by	Who created a task	Shown to admins only

From "owner" to "assignees": a many-to-many relationship

Before, each task had one `user_id` column: one task → one owner. A group task needs **several** people, and a person has **many** tasks. That is a **many-to-many** relationship, and a single column cannot hold it (a database cell holds one value — storing "3,5,8" in one cell would make searching and checking impossible).

The standard solution is a **link table** with one row per connection:



The pair `(task_id, user_id)` is the table's **composite primary key**, so nobody can be assigned twice to the same task. `ON DELETE CASCADE` on both foreign keys removes the rows automatically when a task or a user is deleted.

"Is this a group task?" is never stored — it is **calculated** from the number of rows, so it can never be out of date:

```ts
// SQL that counts how many people a task is assigned to.
const assigneeCount = sql`(SELECT COUNT(*) FROM ${taskAssignees} WHERE ${taskAssignees.taskId} =
${tasks.id})`;
// ...
if (query.kind === 'group') conditions.push(sql`${assigneeCount} > 1`);
if (query.kind === 'individual') conditions.push(sql`${assigneeCount} <= 1`);
```

A query inside another query, like `(SELECT COUNT(*) …)`, is called a **subquery**.

Who may do what with a task

```ts
// AUTHORIZATION RULES FOR TASKS  (top of src/services/task.service.ts)
//  - A task is visible to the users it is ASSIGNED to (table task_assignees).
//    One assignee = individual task, several assignees = group task.
//  - Administrators can see and edit every task, and choose the assignees.
//  - Every assignee can edit, complete and reopen the task and its subtasks.
//  - Only the task's creator or an administrator can delete it.
//  - Only administrators are told WHO created a task.
// Tasks you may not see behave exactly like tasks that do not exist (404).
```

The heart of "visible to its assignees" is a subquery that lists the IDs of a user's tasks:

```ts
/** A sub-query: the IDs of all tasks assigned to one user. */
function taskIdsAssignedTo(db: AppDatabase, userId: number) {
  return db.select({ id: taskAssignees.taskId }).from(taskAssignees).where(eq(taskAssignees.userId,
userId));
}

// in listTasks, scope 'mine':
conditions.push(inArray(tasks.id, taskIdsAssignedTo(db, viewer.id)));
```

`inArray(column, subquery)` becomes `tasks.id IN (SELECT task_id FROM task_assignees WHERE user_id = ?)`.

Services now receive the whole **viewer** (`{ id, role }`) instead of only the user's ID, because the rules depend on the role: `getAccessibleTask` skips the assignee check for admins.

Assigning: who decides?

```ts
function resolveAssignees(db: AppDatabase, viewer: Viewer, assigneeIds: number[] | undefined):
number[] {
  if (assigneeIds === undefined) return [viewer.id];
  const unique = [...new Set(assigneeIds)];
  if (viewer.role !== 'admin' && unique.some((id) => id !== viewer.id)) {
    throw new AppError('Only administrators can assign tasks to other people.', 403, 'FORBIDDEN');
```

```
  }
  const found = db.select({ id: users.id }).from(users).where(inArray(users.id, unique)).all();
  if (found.length !== unique.length) {
    const missing = unique.filter((id) => !found.some((f) => f.id === id));
    throw new AppError(`User ${missing.join(', ')} does not exist.`, 400, 'INVALID_ASSIGNEE');
  }
  return unique;
}
```

1. No `assigneeIds` → the task is for its creator (normal users' tasks always work like this).
2. `new Set(...)` removes duplicates (`[2, 2, 3]` → `[2, 3]`).
3. Only admins may name other people (`403`).
4. Every ID must belong to a real user (`400 INVALID_ASSIGNEE`).

Creating the task and its assignee rows happens in one **transaction** (see Chapter 10, Flow 2). Changing assignees on `PATCH` (admins only) deletes the old rows and inserts the new ones, also in one transaction.

Categories on shared tasks

Categories are personal. When an admin assigns a task with the admin's "Work" category, members see the chip, but "Work" is not in *their* category list. Two small rules keep this working:

- The server only checks a `categoryId` when it **changes** — so a member's edit that keeps the creator's category is accepted, but a member cannot pick someone else's category.
- The edit form adds the creator's category as an extra option labelled "(set by creator)" so saving does not accidentally remove it.

"Created by" — only for admins

The `tasks.created_by` column records who created each task. The service decides what goes into the response:

TS

```
const { createdById, ...taskFields } = task;      // never send the raw ID
// ...
if (isAdmin) {
  view.createdBy = creators.find((c) => c.id === createdById) ?? null;
}
```

Normal users never receive `createdBy` — not even `createdById`. They do receive `canDelete` (true/false) so the page can hide the Delete button; that only tells them whether *they* created it, which they already know. A test checks that a normal user's response has neither property.

Loading related data efficiently (avoiding "N+1 queries")

Each task in a list needs its assignees and subtasks. Querying them **per task** would mean 1 query for the list + 2 per task — for 100 tasks, 201 queries. This classic performance trap is called the **N+1 query problem**. `hydrate()` instead loads everything for *all* listed tasks at once:

```ts
const ids = rows.map((row) => row.task.id);
const assigneeRows = db.select({ taskId: taskAssignees.taskId, id: users.id, name: users.name })
  .from(taskAssignees).innerJoin(users, eq(users.id, taskAssignees.userId))
  .where(inArray(taskAssignees.taskId, ids)).orderBy(asc(users.name)).all();
const subtaskRows = db.select().from(subtasks).where(inArray(subtasks.taskId,
ids)).orderBy(asc(subtasks.id)).all();
```

Then JavaScript sorts the rows into the right tasks with `filter`. Three or four queries in total, however many tasks there are.

Subtasks: tasks inside a task

A subtask is a simple checklist item stored in the `subtasks` table (`task_id`, `title`, `done`). The rules come for free: `subtask.service.ts` calls `getAccessibleTask` on the parent task, so **whoever can see a task can manage its checklist** — including every member of a group task. A subtask of an invisible task answers `404` with the message "Subtask N was not found", never revealing the parent.

```ts
export function addSubtask(db: AppDatabase, viewer: Viewer, taskId: number, input:
CreateSubtaskInput): SubtaskView {
  getAccessibleTask(db, viewer, taskId); // 404 if the viewer may not see the task
  const subtask = db
    .insert(subtasks)
    .values({ taskId, title: input.title, done: false, createdAt: nowIso() })
    .returning()
    .get();
  touchTask(db, taskId);
  return toView(subtask);
}
```

`touchTask` moves the parent's `updatedAt` forward, because changing a checklist is changing the task. `done` is a **boolean**: SQLite stores it as `0` / `1`, and Drizzle's `integer('done', { mode: 'boolean' })` converts it to `false` / `true` for us.

Why a checklist and not "full tasks inside tasks"? A subtask with its own status, priority, assignees and due date would make every rule twice as complicated (can a group member see a subtask assigned to someone else?). A checklist covers the common need simply. Chapter 16 shows how a similar child table (comments) is added.

The four views

The page now has tabs. Each is just a `<section>` that is shown or hidden; the address bar's **hash** (`#/calendar`) remembers which one (Chapter 8, "hash routing").

View	Shows	Data it loads
Dashboard (`#/dashboard`, default)	7 number cards (incl. **Group tasks**), completion bar, "Overdue" and "Due in the next 7 days" lists	`GET /api/dashboard`; `GET /api/tasks?due=overdue`; `GET /api/tasks?dueFrom=today&dueTo=today+6`
Tasks (`#/tasks`)	Filters (incl. Individual/Group), task cards with assignee badge, subtasks and — for admins — "Created by"	`GET /api/tasks?…filters`
Calendar (`#/calendar`)	Four view types — **Month**, **Week**, **Day**, **Agenda** — with tasks on their due dates; ‹ › to move; **+** on a day creates a task with that due date; clicking a date opens the Day view	`GET /api/tasks?dueFrom=…&dueTo=…` (the range depends on the view type)
Admin (`#/admin`, admins only)	Users table; **All tasks** table with Type, Assigned to, **Created by**; filter by person; **Assign a task** button	`GET /api/admin/users`; `GET /api/admin/tasks`

The calendar's four view types (Development Entry #7)

Buttons above the calendar switch between four **view types** (called *modes* in the code):

View type	What it shows	‹ › moves by	Best for
Month	A 6-week grid; one short label per task	1 month	The big picture
Week	7 tall columns (Monday–Sunday); each task shows its status and subtask progress	7 days	Planning this week
Day	One day as a list of full task rows (status, priority, group members, subtasks, "Overdue") and **+ Add task on this day**	1 day	Focusing on today
Agenda	A list of the next 30 days, grouped by date, **only days with tasks**; today is marked	30 days	Seeing what's coming

Clicking a **date** (the number in Month/Week, the heading in Agenda) opens that day in the Day view; **Today** jumps back to today. On a phone, the week's 7 columns become a vertical list.

One idea for all four: the "anchor" date

Every view is described by two things kept in `state`: the **mode** and an **anchor** date (`"YYYY-MM-DD"`), the day the view is built around. Two small, pure functions in `ui.js` do all the date maths:

```JS
export function calendarRange(mode, anchor) {
  if (mode === 'day') return { start: anchor, end: anchor };
  if (mode === 'week') {
    const start = mondayOf(anchor);
    return { start, end: addDays(start, 6) };
  }
  if (mode === 'agenda') return { start: anchor, end: addDays(anchor, AGENDA_DAYS - 1) };
  // month: whole weeks around the month — 6 weeks × 7 days = 42 cells
  const date = parseDate(anchor);
  const start = mondayOf(toDateString(new Date(date.getFullYear(), date.getMonth(), 1)));
  return { start, end: addDays(start, 41) };
}

export function shiftCalendar(mode, anchor, direction) {
  if (mode === 'month') {
    const date = parseDate(anchor);
    return toDateString(new Date(date.getFullYear(), date.getMonth() + direction, 1));
  }
  if (mode === 'week') return addDays(anchor, 7 * direction);
  if (mode === 'day') return addDays(anchor, direction);
  return addDays(anchor, AGENDA_DAYS * direction); // agenda
}
```

- `calendarRange` answers "which dates does this view show?". `app.js` sends exactly that range to the API (`dueFrom` / `dueTo`), so each view loads only the tasks it needs — no server change was required for the new view types.
- `shiftCalendar` answers "where do ‹ and › go?" (`direction` is −1 or +1).
- A **pure function** returns a result based only on its inputs and changes nothing else. Pure functions are easy to understand and to test.

`mondayOf` finds the Monday of a week:

```JS
function mondayOf(dateString) {
  const mondayOffset = (parseDate(dateString).getDay() + 6) % 7; // Monday=0 … Sunday=6
  return addDays(dateString, -mondayOffset);
}
```

`getDay()` numbers days Sunday = 0 … Saturday = 6. `(day + 6) % 7` re-numbers them Monday = 0 … Sunday = 6 — exactly the number of days to go back to reach Monday. `% 7` ("modulo") is the remainder after dividing by 7. JavaScript's `new Date(year, month, day)` happily handles a day such as "−5" by moving into the previous month, which is why `addDays` can simply add or subtract.

Drawing each view type

`renderCalendar(mode, anchor, tasks, handlers)` sets the heading, marks the active button (`aria-pressed="true"`), sets the class `calendar mode-<mode>` on the container (CSS switches between the grid and list layouts with it), and then builds:

- **Month / Week:** one `calendarCell` per date, each with a clickable date, a + button and a `calendarChip` per task. Week chips are *detailed* (a second line with status and subtask progress).

- **Day:** a header with the number of tasks and **+ Add task on this day**, then one `calendarRow` per task (or "No tasks due on this day.").
- **Agenda:** loops over the 30 dates, skips dates without tasks, and adds a date heading plus `calendarRow`s; if nothing is due, "Nothing due in these 30 days.".

Every task chip or row calls `handlers.onTaskClick(task)` (opens the edit form); dates call `handlers.onDayClick(date)`; + calls `handlers.onAddForDate(date)`.

Remembering the chosen view type: localStorage

```JS
calendarMode: CALENDAR_MODES.includes(localStorage.getItem('calendarMode')) ?
localStorage.getItem('calendarMode') : 'month',
// …
function setCalendarMode(mode, anchor = state.calendarAnchor) {
  state.calendarMode = mode;
  state.calendarAnchor = anchor;
  localStorage.setItem('calendarMode', mode);
  loadCalendar();
}
```

`localStorage` is a small key–value store inside the browser that survives page reloads and restarts. It is perfect for harmless *preferences* such as a view type. The value is checked against `CALENDAR_MODES` before use, because anything stored in the browser can be edited by the user. (It would be the wrong place for secrets: any JavaScript on the page can read it — which is why the login token lives in an `HttpOnly` cookie instead, Chapter 20.)

The assignee picker

Admins see an extra **Assign to** section in the task form: **Individual** shows a dropdown of users; **Group** shows a checklist. `readTaskForm()` turns it into `assigneeIds` and refuses a "group" with fewer than two people before anything is sent to the server.

Upgrading existing data

Moving from `tasks.user_id` to `task_assignees` needed a data migration and uncovered a dangerous SQLite behaviour (a table rebuild silently cascade-deleting the new assignee rows). Chapter 7, "Migrations 0002 and 0003", tells the whole story and shows the fix in `runMigrations`. Your real database was backed up to `data/backups/todo-before-assignees-2026-11-01.db` before the upgrade; afterwards all 7 tasks still had their creator and assignee.

Try it yourself

1. Register a second account in a private window. As admin, open **Admin → Assign a task**, choose **Group** and tick both accounts. Check that the other account sees the task with a "Group" badge but

no "Created by" and no Delete button.

2. In the other account, add a subtask to that group task and tick it. Reload the admin's page: the subtask is ticked there too.
3. Click + on a day in the **Calendar** and create a task. Find it in the **Dashboard** list "Due in the next 7 days".
4. Open `data/todo.db` in a SQLite viewer and look at `task_assignees`. Find the rows of your group task.
5. Exercise: add a "Remove me from this group task" button for members (a `DELETE /api/tasks/:id/assignees/me` route). Who should be allowed? What if the last member leaves?

Development Log

This part of the manual is a permanent learning record. Every development request gets its own entry, in the order the requests were made. Entries are never deleted, so you can see how the application grew and why each decision was made.

Development Entry: #1 — Build the Hono To-Do Application and Its Learning Manual

Date: 2026-11-01

A. What I Asked For

Build a complete, working To-Do application with Hono, TypeScript, Node.js, SQLite, Drizzle ORM, Zod, plain HTML/CSS/JavaScript and Vitest — including a dashboard, full task management, categories, search, filters, sorting, and a professional interface with notifications, loading, error and empty states. At the same time, create a beginner-friendly learning manual in HTML and PDF that explains every concept and file, generated from one Markdown source with a repeatable command, plus a changelog — and keep all of it updated after every future request.

B. What We Are Building

- A Hono backend with a JSON REST API for tasks, categories and dashboard statistics.
- A SQLite database (one file) managed with Drizzle ORM and migrations.
- Input validation with Zod on every endpoint.
- A single-page frontend in vanilla JavaScript that uses the API.
- Automated tests (39 Vitest tests) and a browser smoke test (11 steps).
- A documentation generator (`pnpm docs:build`) producing this manual as HTML and PDF.

C. Why We Need It

The application is the practical example; the manual turns it into a course. Building both together means every explanation is tied to real, running, tested code — which is the fastest way to move from copying code to understanding it.

D. Technical Concepts

All concepts introduced in this entry are explained in Chapters 1–19: frontend/backend (Ch. 1), Hono apps, context, routes, handlers (Ch. 2), Node.js and pnpm (Ch. 3), TypeScript (Ch. 4), HTTP and REST (Ch. 5), middleware and the onion model (Ch. 6), SQL, tables, keys, ORM and migrations (Ch. 7), DOM, events and fetch (Ch. 8), layered architecture (Ch. 9), automated testing (Ch. 13) and web security (Ch. 14). Concepts that first appear in this entry and deserve special attention:

- **Dependency injection** — the database is *given* to the app (`createApp({ db })`) and to each request (`c.set('db', db)`), so tests can use a temporary database.
- **Schema-driven types** — Drizzle infers `Task` from the table definition; Zod infers `CreateTaskInput` from the validator. Each shape is described once.
- **Central error handling** — services throw typed errors (`NotFoundError`), and one handler converts all errors into consistent JSON.
- **Parameterised queries** — values are sent separately from SQL, preventing SQL injection.

E. Files Changed

All files were **created** in this entry.

File	What it does	Connects to
`package.json`	Project info, scripts, dependencies	everything
`pnpm-lock.yaml`, `pnpm-workspace.yaml`	Exact versions; pnpm build-script settings	pnpm
`tsconfig.json`	TypeScript settings	`pnpm typecheck`
`drizzle.config.ts`	drizzle-kit settings	`src/db/schema.ts` → `src/db/migrations/`
`vitest.config.ts`	Test settings	`tests/`
`.env.example`, `.gitignore`	Example settings; files git ignores	`src/config.ts`
`README.md`	Short overview and commands	—
`src/index.ts`	Starts the server	`app.ts`, `config.ts`, `db/index.ts`
`src/app.ts`	Builds the Hono app	middleware, routes
`src/config.ts`	Validated environment variables	`index.ts`, scripts
`src/types.ts`	`AppEnv` context types	app, routes, middleware
`src/db/schema.ts`	`tasks` and `categories` tables	services, validators
`src/db/index.ts`	Open DB, migrate, close	index, tests, scripts
`src/db/migrations/0000_init.sql` (+ `meta/`)	Creates the tables and indexes	`runMigrations`
`src/lib/errors.ts`	Error classes	services, middleware
`src/lib/dates.ts`	`nowIso`, `todayLocal`	services, routes, seed

File	What it does	Connects to
`src/middleware/request-logger.ts`	Request logging	`app.ts`
`src/middleware/database.ts`	`c.set('db', db)`	`app.ts`
`src/middleware/validate.ts`	Zod validation → `ValidationError`	routes
`src/middleware/error-handler.ts`	`onError` / `notFound`	`app.ts`
`src/validators/common.ts`	`dateString`, `idParamSchema`	other validators, routes
`src/validators/task.validator.ts`	Task schemas + types	task routes, task service
`src/validators/category.validator.ts`	Category schemas + types	category routes/service
`src/validators/dashboard.validator.ts`	Dashboard query schema	dashboard route
`src/services/task.service.ts`	Task CRUD, search, filters, sorting	task routes, seed
`src/services/category.service.ts`	Category CRUD, task counts	category routes, seed
`src/services/dashboard.service.ts`	Statistics	dashboard route, tests
`src/routes/task.routes.ts`	`/api/tasks` endpoints	`app.ts`
`src/routes/category.routes.ts`	`/api/categories` endpoints	`app.ts`
`src/routes/dashboard.routes.ts`	`/api/dashboard` endpoint	`app.ts`
`public/index.html`	Page structure	CSS, JS
`public/css/styles.css`	Styling, responsive layout	`index.html`
`public/js/api.js`	All `fetch` calls	`app.js`, `ui.js`
`public/js/ui.js`	Rendering helpers	`app.js`
`public/js/app.js`	State, events, actions	`api.js`, `ui.js`
`public/favicon.svg`	Browser tab icon	`index.html`
`tests/helpers.ts`	`createTestApp()`	all tests
`tests/tasks.test.ts`, `categories.test.ts`, `dashboard.test.ts`, `app.test.ts`	39 automated tests	`src/`
`scripts/migrate.ts`, `scripts/seed.ts`	DB commands	`src/db`, services
`scripts/e2e-smoke.ts`	Browser smoke test	`src/app.ts`, `public/`
`scripts/build-docs.ts`, `scripts/lib/browser.ts`	Documentation generator; browser finder	`docs/`

File	What it does	Connects to
`docs/DOCUMENTATION-SOURCE.md`, `docs/CHANGELOG.md`	Manual source and changelog	`build-docs.ts`
`docs/LEARN-HONO-COMPLETE.html`, `.pdf`	Generated manual	—

F. Code Explanation

The important code is explained line by line in the chapters: the server start-up (Ch. 11, `index.ts`), the app assembly and middleware order (Ch. 6), the validation wrapper and error handler (Ch. 6), the schema and migration (Ch. 7), the task service's filtering and sorting (Ch. 10, Flow 4), the dashboard query (Ch. 10, Flow 9), the frontend's `request()` wrapper and rendering (Ch. 8), and the test helper (Ch. 13).

One piece worth highlighting here is how the task list query is assembled from optional filters:

```ts
const conditions: SQL[] = [];
if (query.search) { /* … LIKE … */ }
if (query.status) conditions.push(eq(tasks.status, query.status));
if (query.priority) conditions.push(eq(tasks.priority, query.priority));
if (query.categoryId) conditions.push(eq(tasks.categoryId, query.categoryId));
// … due-date conditions …
return
selectTasksWithCategory(db).where(and(...conditions)).orderBy(...orderBy).all().map(toTaskWithCategory
);
```

Each filter that is present adds one condition to a list; `and(...conditions)` combines them. With no filters the list is empty and `and()` produces no `WHERE` clause, returning every task.

G. Complete Code Flow

```
User action (click / type)
       ↓
Frontend JavaScript (public/js/app.js → api.js)
       ↓
HTTP request (fetch, JSON body, ?today=…)
       ↓
Hono app.fetch → middleware: logger → secure headers → body limit → inject db
       ↓
Route (src/routes/*.routes.ts)
       ↓
Validation (validate(...) + Zod schema) ── invalid ──→ ValidationError ─┐
       ↓ valid                                                          │
Service (src/services/*.service.ts) ── missing/conflict ──→ AppError ──┤
       ↓                                                                │
Database (Drizzle → better-sqlite3 → data/todo.db)                      │
       ↓                                                                ▼
HTTP response  c.json({ data }, 200/201) or 204      errorHandler → { error } 4xx/5xx
       ↓
Frontend update (toast, refresh list + dashboard, or show errors)
```

Each feature's detailed flow is in Chapter 10.

H. Testing

What	How	Expected	Actual	Result
Type safety	`pnpm typecheck`	no errors	no errors (after fixing one: the `AppDatabase` type was missing `$client`)	Passed
API behaviour	`pnpm test` (Vitest, in-memory DB)	39 passing	39 passed, 4 files	Passed
Browser behaviour	`pnpm test:e2e` (headless Chrome, temporary DB)	11 steps, no console errors	11 passed, no console errors	Passed
Real start-up	`pnpm db:seed`, `pnpm start`, requests to `/api/health`, `/api/dashboard`, `/api/tasks`, invalid `POST`	200s with seeded data; 400 with details; one log line per request	As expected	Passed
Visual check	Screenshots at 1280 px and 390 px wide	Clean layout on desktop and phone	As expected	Passed
Documentation	`pnpm docs:build`	HTML + PDF generated, verification OK	See the CHANGELOG entry for the numbers	Passed

Problems found and fixed while testing:

- The first browser test failed on the "Clear filters" click: in the default 800×600 test window the sticky top bar covered the button after scrolling. The test now uses a realistic 1280×1000 window.
- The browser requested `/favicon.ico` automatically, causing a 404. A `favicon.svg` was added and linked from `index.html`.
- **A real XSS bug in the documentation generator.** The first `pnpm docs:build` timed out. The cause: the HTML manual embeds a search index (the plain text of every section) inside a `<script>` tag. That text contained `</script>` and `<img src=x onerror=alert(1)>` (examples quoted in Chapters 8 and 14). The browser treated `</script>` as the end of the script, then created a real `<img>` element whose `onerror` ran `alert(1)` — and the alert dialog blocked the page forever. This is exactly the XSS attack described in Chapter 14, caused by inserting text into HTML without escaping. **Fix:** every `<` in the embedded JSON is now written as `\u003c`, which JavaScript reads as `<` but the HTML parser ignores. The generator also now fails loudly if the page shows a dialog or throws a script error. **Lesson:** whenever text goes into HTML — or into a `<script>` inside HTML — it must be escaped for that exact context.
- In the HTML manual, the code font joined `===` into `≡` and `=>` into `⇒` (typographic ligatures). Ligatures were switched off so code looks exactly as you type it.

I. What I Learned

- How a Hono app is assembled from middleware and route groups, and why it is separated from the server start-up.
- How data flows through the layers: route → validation → service → database → response.
- How Zod and Drizzle let you describe data once and get TypeScript types for free.
- How SQLite stores data in one file, and how migrations evolve its structure.
- How the frontend uses `fetch`, handles errors, and keeps the screen in sync with the server.
- How automated tests give confidence, and how to test an app without a network.

J. What I Can Try Next

1. Add a `notes` field to tasks by following the recipe in Chapter 16 — it touches every layer.
2. Add a "Start" button on pending tasks that sets their status to `in_progress`.
3. Write a test proving that tasks without a due date never appear in the `overdue` filter.

K. Important Notes

- **No authentication**: do not expose the app to the internet as it is (Chapter 14, 17).
- **"Overdue"** means due before today and not completed; **"due today"** excludes completed tasks. The browser's local date is used.
- **Search** uses SQL `LIKE`; the characters `%` and `_` typed into the search box act as wildcards. This is harmless but can surprise.
- **SQLite** suits a single server instance; move to PostgreSQL for multiple instances.
- Deleting a category keeps its tasks (they become uncategorised).

Development Entry: #2 — Install the Project With pnpm Instead of npm

Date: 2026-11-01

A. What I Asked For

While the project was being set up, you asked: *"install it through pnpm"* — use the pnpm package manager instead of npm to install and manage the project's packages.

B. What We Are Building

The same application, but managed by pnpm: dependencies are installed with `pnpm install` / `pnpm add`, scripts run with `pnpm <script>`, the exact versions are recorded in `pnpm-lock.yaml`, and pnpm-specific settings live in `pnpm-workspace.yaml`.

C. Why We Need It

pnpm is faster, uses much less disk space (each package version is stored once on your computer and linked into projects), is stricter about undeclared dependencies, and does not run packages' install scripts unless you allow them — a useful security default.

D. Technical Concepts

- **Package manager** — a tool that downloads packages and records their versions (Chapter 3).
- **Lockfile** — `pnpm-lock.yaml` replaces npm's `package-lock.json`; it pins every exact version.
- **Content-addressable store** — pnpm keeps one copy of each package version in a global store (on this computer `C:\.pnpm-store`) and hard-links it into `node_modules/.pnpm`.
- **Install / build scripts** — some packages run a program when installed (e.g. to download or compile a native binary). pnpm only runs them for packages approved in `allowBuilds`.
- **Native addon and prebuilt binaries** — `better-sqlite3` contains C++ code. It ships ready-compiled versions ("prebuilds") for common systems, so no compiler is needed.
- **node-gyp** — the tool that compiles C++ addons from source. It needs Visual Studio C++ build tools on Windows, which this computer does not have.
- `packageManager` **field** — records which package manager and version the project expects.

E. Files Changed

File	Change	Why
`package-lock.json`	**Deleted**	npm's lockfile; replaced by pnpm's
`node_modules/`	**Deleted and reinstalled** by pnpm	pnpm uses a different, linked layout
`pnpm-lock.yaml`	**Created**	pnpm's lockfile
`pnpm-workspace.yaml`	**Created**	`allowBuilds` settings (and pnpm's own `minimumReleaseAgeExclude`)
`package.json`	**Modified**	Removed npm's `allowScripts` block; added `"packageManager": "pnpm@12.4.1"`; moved `tsx` to `dependencies` (needed to start the app); removed unused `build` / `start:prod` scripts; raised `engines.node` to `>=22`; removed the `dotenv` package (Node's built-in `process.loadEnvFile` is used instead)
`docs/*`, `README.md`	Written with pnpm commands throughout	—

F. Code Explanation

`pnpm-workspace.yaml`:

```yaml
# pnpm does not run package install (build) scripts unless we decide per package.
# - better-sqlite3 ships ready-made native binaries in its "prebuilds" folder,
#   so compiling it from source (node-gyp) is unnecessary -> false.
# - esbuild (used by tsx and vitest) needs its script to install its binary -> true.
allowBuilds:
  better-sqlite3: false
  esbuild: true
minimumReleaseAgeExclude:
  - hono@4.13.16
```

- `allowBuilds` is a map of package name → allowed (`true`) or not (`false`).
- `minimumReleaseAgeExclude` was added automatically by pnpm: pnpm refuses package versions published very recently (to give the community time to spot malicious releases); this allows that specific Hono version.

G. Complete Code Flow

```
pnpm install
    ↓ reads package.json + pnpm-lock.yaml (+ pnpm-workspace.yaml settings)
    ↓ verifies the lockfile against supply-chain policies
    ↓ downloads missing packages into the global store (C:\.pnpm-store)
    ↓ hard-links them into node_modules/.pnpm and links direct deps into node_modules/
    ↓ runs install scripts ONLY for packages with allowBuilds: true (esbuild)
node_modules ready → pnpm start / pnpm test / pnpm docs:build
```

H. Testing

What	How	Expected	Actual	Result
First attempt	`pnpm add …` with `better-sqlite3: true`	install succeeds	**Failed**: pnpm ran `node-gyp rebuild`, which needs C++ build tools	Failed → investigated
Investigation	Inspected `node_modules/.pnpm/better-sqlite3@13.0.3/…`	—	No install script of its own; `prebuilds/win32-x64.node` present; pnpm runs node-gyp automatically because `binding.gyp` exists	—
Fix	Set `better-sqlite3: false` and re-ran `pnpm add`	install succeeds	88 packages installed, lockfile passes supply-chain checks	Passed
SQLite works	`new Database(':memory:')` → `select sqlite_version()`	a version number	`3.53.4`	Passed

What	How	Expected	Actual	Result
Tools work	`pnpm exec tsx --version`, `pnpm exec tsc --version`	versions printed	`tsx 4.23.15`, `TypeScript 7.0.2`	Passed
Everything else	All tests from Entry #1 were run with pnpm	pass	pass	Passed

I. What I Learned

- npm and pnpm use the same package registry; they differ in how they install and store packages.
- Lockfiles guarantee identical installs.
- Some packages contain native (C++) code; they either ship prebuilt binaries or compile during installation.
- When an installation fails, read the error, inspect the package, and understand *why* before changing settings.

J. What I Can Try Next

1. Run `pnpm why better-sqlite3` to see which of your dependencies require it.
2. Run `pnpm outdated` to see whether newer versions of the packages exist.
3. Delete `node_modules` and run `pnpm install` — notice how fast it is the second time, because the packages come from the store.

K. Important Notes

- Always use pnpm in this project. Mixing in `npm install` would create a `package-lock.json` and a differently structured `node_modules`.
- If you upgrade `better-sqlite3` and a future version stops shipping a prebuilt binary for your system, you would need either C++ build tools (and `allowBuilds: better-sqlite3: true`) or a different SQLite driver.

Development Entry: #3 — Add Authentication and Authorization

Date: 2026-11-01

A. What I Asked For

"add authorization and authentication and add it in doc too" — people should have to log in to use the app (authentication), the app should control what each person may do (authorization), and the manual should explain it all.

B. What We Are Building

- **Accounts:** register (name, email, password), log in, log out, "who am I?".
- **Sessions:** a random token in an `HttpOnly` cookie; only its hash is stored in a new `sessions` table; valid for 7 days.
- **Protected API:** every task, category, dashboard and admin route requires a login (`401` otherwise).
- **Ownership:** tasks and categories now belong to a user; everyone sees and changes only their own.
- **Roles:** `user` and `admin`; the first account is the admin and can list all users (`403` for others).
- **Security extras:** scrypt password hashing, login throttling (`429`), CSRF protection, identical errors for unknown email and wrong password.
- **Frontend:** login/register screen, the signed-in user's name, Log out, an admin-only Users dialog, automatic return to the login screen when a session ends.
- **Upgrade path:** existing tasks are claimed by the first account that registers.

C. Why We Need It

Before this change anyone who could reach the server could read, change and delete every task. With accounts, the app can be shared by several people — each with private data — and it is a big step towards being safe to put on the internet. It also teaches the two most important security concepts of web development.

D. Technical Concepts

All explained in **Chapter 20**: authentication vs authorization; password hashing, salts and slow hash functions (scrypt); constant-time comparison; sessions vs JWT; cookies and their `HttpOnly`, `SameSite`, `Secure`, `Path`, `Expires` flags; database transactions; role-based access control; ownership checks and IDOR; why `404` instead of `403`; CSRF; brute-force throttling; user enumeration. Also new: the HTTP status codes `401`, `403`, `429` (Chapter 5), `innerJoin` and `db.transaction` in Drizzle (Chapter 7), and Hono's `hono/cookie` and `hono/csrf` (Chapters 6, 20).

E. Files Changed

Created

File	What it does	Connects to
`src/db/migrations/0001_add_users_and_sessions.sql` (+ `meta/0001_snapshot.json`)	Creates `users`, `sessions`; adds `user_id` to tasks and categories	`runMigrations`
`src/lib/password.ts`	scrypt hashing and verification	`auth.service.ts`
`src/lib/login-throttle.ts`	Blocks repeated failed logins	`app.ts`, `auth.routes.ts`
`src/validators/auth.validator.ts`	Register/login rules	`auth.routes.ts`, `auth.service.ts`

File	What it does	Connects to
`src/services/auth.service.ts`	Accounts, sessions, user list	middleware, routes, seed
`src/middleware/auth.ts`	`requireAuth`, `requireRole`, cookie helpers	all route files, `app.ts`
`src/routes/auth.routes.ts`	`/api/auth/*`	`app.ts`
`src/routes/admin.routes.ts`	`/api/admin/users`	`app.ts`
`tests/auth.test.ts`, `tests/authorization.test.ts`	30 new tests	`src/`

Modified

File	Change
`src/db/schema.ts`	New `users`, `sessions` tables; `user_id` on tasks/categories; per-user unique category names; new types
`src/types.ts`	`AppEnv` now also has `auth` and `user`; new `AuthSettings` type
`src/config.ts`, `.env.example`	New `COOKIE_SECURE` setting
`src/app.ts`	`csrf()`, `injectAuthSettings`, a `LoginThrottle`, auth and admin route groups, `secureCookies` option
`src/index.ts`	Passes `secureCookies` to `createApp`
`src/services/task.service.ts`, `category.service.ts`, `dashboard.service.ts`	Every function takes `userId`; every query filters by it
`src/routes/task.routes.ts`, `category.routes.ts`, `dashboard.routes.ts`	`use('*', requireAuth)`; pass `c.get('user').id` to services
`scripts/seed.ts`	Creates a demo account (`demo@example.com` / `demo-password-123`)
`scripts/e2e-smoke.ts`	7 new browser steps (18 in total)
`public/index.html`, `public/css/styles.css`	Login/register screen, user chip, Log out, Users dialog
`public/js/api.js`, `ui.js`, `app.js`	Auth API calls, `onUnauthorized`, auth screens, admin list
`tests/helpers.ts`, `tests/*.test.ts`	Cookie-aware test clients; existing tests run as a logged-in user
`README.md`, `docs/*`	Documentation

Deleted: none.

F. Code Explanation

The important code is explained line by line in Chapter 20: password hashing (`password.ts`), session creation and lookup (`auth.service.ts`), the cookie flags (`setSessionCookie`), `requireAuth` and `requireRole`, the register transaction, the login route with throttling, logout, and the ownership checks in the services. The frontend part is in Chapter 8 ("Logging in from the frontend").

The single most important pattern to remember — authorization in every query:

```ts
.where(and(eq(tasks.id, id), eq(tasks.userId, userId)))
```

G. Complete Code Flow

```
User types email + password → submitLogin() (app.js)
       ↓
api.login() → POST /api/auth/login  (JSON)
       ↓
middleware: logger → secure headers → csrf → body limit → inject db + auth settings
       ↓
validate('json', loginSchema)  → 400 if invalid
       ↓
throttle.isBlocked(email)?     → 429
       ↓
authenticate(): find user → verifyPassword (scrypt) → 401 if wrong
       ↓
createSession(): random token → INSERT sha256(token) → Set-Cookie (HttpOnly, Lax)
       ↓
200 { data: user } → showApp(user) → GET /api/tasks with Cookie
       ↓
requireAuth → c.set('user') → listTasks(db, user.id, …) → WHERE user_id = ?
       ↓
the user's own tasks on screen
```

Registration, logout and session expiry flows are in Chapter 20 (Flows A–E).

H. Testing

What	How	Expected	Actual	Result
Types	`pnpm typecheck`	no errors	no errors	Passed
Unit + integration tests	`pnpm test`	all pass	**70 passed** in 6 files (was 39; 30 new auth/authorization tests + 1 CSRF test)	Passed

What	How	Expected	Actual	Result
Browser smoke test	`pnpm test:e2e`	all steps, no console errors	**18/18** passed, no console errors	Passed
Upgrade of an existing database	Copied the real `data/todo.db` (6 tasks from Entry #1), started the server on the copy, registered the first account in a real browser	Migration runs; first account sees the 6 old tasks; API without cookie → 401	Migration applied; 6 tasks visible; `401 UNAUTHORIZED` without cookie	Passed
Visual check	Screenshots of the login screen and the logged-in app	Clean layout, name + admin badge + Users + Log out	As expected	Passed
Documentation	`pnpm docs:build`	HTML + PDF regenerated and verified	See the CHANGELOG entry	Passed

Issues found and handled during this work:

- drizzle-kit generated `ALTER TABLE … ADD user_id … REFERENCES users(id)` **without** `ON DELETE cascade`. The not-yet-applied migration was corrected by hand (explained in Chapter 7).
- Hono's `csrf()` treats requests with **no** `Content-Type` like form submissions. Body-less requests (complete, reopen, delete, logout) therefore need an `Origin` header — browsers always send it; the test helper now does too, and Chapter 5's `curl` examples were updated.
- All test files switched to `await createTestApp()` because registering the default test user is asynchronous (password hashing).

I. What I Learned

- Authentication answers *who are you?*; authorization answers *what may you do?*
- Passwords are stored only as salted, deliberately slow hashes.
- A session is a random token in a cookie; the server keeps (a hash of) it in a table; logout deletes it.
- Cookie flags (`HttpOnly`, `SameSite`, `Secure`) each stop a specific attack.
- Authorization belongs in a place where it cannot be forgotten — here, every service query.
- Returning `404` for other people's data hides that it exists.
- Hiding a button in the UI is never a security measure; the server must check.

J. What I Can Try Next

1. Add `PATCH /api/auth/password` to change your password (verify the old one, hash the new one, delete your other sessions), with tests.
2. Add `PATCH /api/admin/users/:id` so an admin can promote another user to admin — protected by `requireRole('admin')`.
3. Write a test proving that a deleted user's sessions stop working (hint: delete the row from `users` and rely on `ON DELETE CASCADE`).

K. Important Notes

- **Register your own account first** on any new installation: the first account becomes the admin.
- Set `COOKIE_SECURE=true` whenever the site runs on HTTPS (always in production). On plain `http://localhost` keep it `false`, or the browser will refuse to store the cookie.
- The seeded demo account (`demo@example.com` / `demo-password-123`) is for learning only — delete it before going live.
- No password reset yet. The login throttle is in memory.
- Older ownerless tasks are invisible until the first account registers and claims them.

Development Entry: #4 — Question: What Is the Admin's ID and Password?

Date: 2026-11-01

A. What I Asked For

"is the id password of admin" — which email (ID) and password log in as the administrator?

B. What We Are Building

No code change: an answer, a checked fact about the real database, and a new section in Chapter 20 ("What is the admin's email and password?") plus a Chapter 15 troubleshooting row.

C. Why We Need It

After Entry #3 the app requires a login, so you need to know how to get the first (admin) login.

D. Technical Concepts

- **No default credentials:** a password written in the source code is a security hole, because everyone with the code knows it.
- **First account = admin:** set inside the registration transaction (Chapter 20).
- **Hashes cannot be reversed:** a forgotten password cannot be "looked up", only replaced.

E. Files Changed

File	Change
`docs/DOCUMENTATION-SOURCE.md`	New Chapter 20 subsection, new Chapter 15 row, this entry
`docs/CHANGELOG.md`	Entry #4
`docs/LEARN-HONO-COMPLETE.html`, `.pdf`	Regenerated

F. Code Explanation

The rule lives in `registerUser` (`src/services/auth.service.ts`):

```ts
const isFirstUser = tx.select({ value: count() }).from(users).get()!.value === 0;
// ...
role: isFirstUser ? 'admin' : 'user',
```

It counts the users inside the transaction; zero means this new account is the first one and gets the `admin` role.

G. Complete Code Flow

```
You register (first ever) → POST /api/auth/register → registerUser()
  → COUNT(users) = 0 → INSERT … role = 'admin' → claim ownerless tasks
  → session cookie → app shows "Signed in as <you> ADMIN" + Users button
```

H. Testing

What	How	Result
Does the real database already have an admin?	Read `data/todo.db` (read-only)	No `users` table yet — the accounts migration runs on the next `pnpm start`; 6 tasks waiting to be claimed
First account becomes admin	Existing tests `makes only the first account an admin` and `is claimed by the first account that registers` (`tests/auth.test.ts`)	Passing (part of the 70)

I. What I Learned

There is no "master password". The admin is simply whoever registers first, with the password they chose — and nobody can read that password back from the database.

J. What I Can Try Next

Register your account, open the **Users** dialog, then register a second account in a private window and check it has no **Users** button.

K. Important Notes

Register **your own account before** running `pnpm db:seed`, otherwise the demo account becomes the admin and receives your existing tasks.

Development Entry: #5 — Bug Report: "Route POST /api/auth/register was not found."

Date: 2026-11-01

A. What I Asked For

You reported that registering failed with the message *"Route POST /api/auth/register was not found."*

B. What We Are Building

No code change was needed. We found out *why* a route that exists in the code answered "not found", fixed the running environment, and added a troubleshooting entry so you can recognise this problem yourself.

C. Why We Need It

"The code is right but the app behaves as if it isn't" is one of the most common — and most confusing — problems in programming. Knowing the cause saves hours.

D. Technical Concepts

- **A running program is a snapshot of the code.** `pnpm start` reads the source files once. Editing files afterwards does not change the program that is already running; it must be restarted. (`pnpm dev` restarts automatically on every save.)
- **Process:** a running program. Each has a **process ID (PID)**. Only one process can listen on a port such as 3000.
- **The message came from our own** `notFoundHandler` (`src/middleware/error-handler.ts`): *"Route ${method} ${path} was not found."* So the request *did* reach a Hono app — just one without the new route.

E. Files Changed

File	Change
`docs/DOCUMENTATION-SOURCE.md`	Chapter 15 troubleshooting row, this entry
`docs/CHANGELOG.md`	Entry #5
`docs/LEARN-HONO-COMPLETE.html`, `.pdf`	Regenerated

F. Code Explanation

The route exists — it is registered in `src/app.ts` and `src/routes/auth.routes.ts`:

```ts
app.route('/api/auth', authRoutes);                                    // src/app.ts
authRoutes.post('/register', validate('json', registerSchema), async (c) => { … });  // auth.routes.ts
```

A Hono app only knows the routes that existed **when the process started**. The process on port 3000 had started before these lines were written.

G. Complete Code Flow

```
BEFORE (old process, started 3:21 PM, before accounts existed at 4:18 PM)
Browser → POST /api/auth/register → old app: no matching route
        → app.notFound → 404 "Route POST /api/auth/register was not found."

AFTER (old process stopped, new one started with the current code)
Browser → POST /api/auth/register → requireAuth? no (public) → validate → registerUser
        → 201 + session cookie   (database migrated automatically on start-up)
```

H. Testing

Step	How	Result
Find what serves port 3000	`netstat -ano` + process details	One `tsx src/index.ts` process, started **3:21 PM** — before `auth.routes.ts` was created (**4:18 PM**)
Cause	That process was a server started by Claude during the initial build for screenshots; an earlier stop command did not end it	Confirmed
Fix	Stopped the old processes (PIDs 24356, 37224) and started `pnpm start` with the current code	Server running
Route exists now	`POST /api/auth/register` with an empty body (so no account is created)	`400 VALIDATION_ERROR` instead of `404` — the route exists
Database upgraded	Read `data/todo.db`	`users` and `sessions` tables created by migration 0001
Your registration	Server log + database	`POST /api/auth/register -> 201`; account *Minal Poudel* (`admin@gmail.com`) is **admin** and owns all 6 earlier tasks; `GET /api/admin/users -> 200`

I. What I Learned

If the app behaves like an older version of the code, suspect an **old process** before suspecting the code. Check *what* is running and *when* it started, and restart.

J. What I Can Try Next

Run `pnpm dev` instead of `pnpm start` while changing code. Edit the text of a message in `src/routes/auth.routes.ts`, save, and watch the server restart by itself in the terminal.

K. Important Notes

Only one program can use port 3000. If a fresh `pnpm start` fails with `EADDRINUSE`, an old copy is still running — see Chapter 15.

Development Entry: #6 — Admin Assigns Tasks, Group & Individual Tasks, Subtasks, Separate Dashboard and Calendar Views, "Created By" for Admins

Date: 2026-11-01

A. What I Asked For

"add a feature in admin to give a task to user and separate a dashboard and calendar like view too. add group task and individual task too, add a option to add task inside task, add a view to only admin on who created a task" — five features:

1. Administrators can give (assign) tasks to users.
2. A separate dashboard view and a calendar view.
3. Group tasks and individual tasks.
4. Tasks inside tasks.
5. A view, for administrators only, showing who created each task.

B. What We Are Building

- **Assignments:** a `task_assignees` link table; admins pick one person (individual) or several (group) when creating or editing a task.
- **Group tasks:** every member sees the same task, can edit, complete/reopen it and use its subtasks; only the creator or an admin can delete it; only admins change members.
- **Subtasks:** a checklist inside every task (`subtasks` table, three new endpoints).
- **Views:** Dashboard, Tasks, Calendar (month grid) and Admin tabs, switched with the URL hash.
- **Created by:** `tasks.created_by`, sent to administrators only; shown on task cards and in the Admin view's All-tasks table.
- Decisions made where the request left room (explained in Chapter 21): normal users keep creating individual tasks for themselves; subtasks are checklist items rather than full tasks; categories stay personal.

C. Why We Need It

The app becomes useful for a small team: an administrator can hand out and track work, people can share tasks, big tasks can be broken into steps, and deadlines are easy to see on a calendar.

D. Technical Concepts

All explained in **Chapter 21** (and Chapter 7 for the migration): many-to-many relationships and link tables; composite primary keys; subqueries and `IN (SELECT …)`; calculated vs stored values (`isGroup`); the N+1 query problem; booleans in SQLite; data migrations (`INSERT INTO … SELECT`); table rebuilds and why `DROP TABLE` can cascade; `PRAGMA foreign_keys` inside transactions; `PRAGMA foreign_key_check`; regression tests; hash routing; modulo arithmetic for calendars.

E. Files Changed

Created

File	What it does
`src/db/migrations/0002_assignees_and_subtasks.sql` (+ snapshot)	New tables, `created_by` column, hand-written data copy
`src/db/migrations/0003_drop_task_owner_column.sql` (+ snapshot)	Removes `tasks.user_id` (table rebuild)
`src/services/subtask.service.ts`	Add / update / delete subtasks with access checks
`src/validators/subtask.validator.ts`	Subtask rules
`src/routes/subtask.routes.ts`	`PATCH` / `DELETE /api/subtasks/:id`
`tests/assignment.test.ts` (14 tests), `tests/subtasks.test.ts` (8), `tests/migrations.test.ts` (1)	New automated tests

Modified

File	Change
`src/db/schema.ts`	`tasks.created_by` replaces `tasks.user_id`; new `task_assignees` and `subtasks` tables; `Subtask` type
`src/db/index.ts`	`runMigrations` disables foreign keys around migrations and runs `foreign_key_check` (**data-loss fix**)
`src/services/task.service.ts`	Rewritten around assignees: viewer-based access, `TaskView` (assignees, isGroup, subtasks, canDelete, createdBy for admins), `kind` / `dueFrom` / `dueTo` filters, admin `'all'` scope, assignment rules
`src/services/dashboard.service.ts`	Counts tasks assigned to the user; new `groupTasks`

File	Change
`src/services/auth.service.ts`	First user claims unassigned tasks via `task_assignees`; user list counts assignments
`src/validators/task.validator.ts`	`assigneeIds`, `kind`, `dueFrom`, `dueTo`, `adminListTasksQuerySchema`
`src/routes/task.routes.ts`	Passes the viewer; `POST /api/tasks/:id/subtasks`
`src/routes/admin.routes.ts`	`GET /api/admin/tasks`
`src/app.ts`	Mounts `/api/subtasks`
`scripts/seed.ts`	Passes the demo user as viewer
`scripts/e2e-smoke.ts`	Rewritten for tabs; 25 steps
`public/index.html`, `public/css/styles.css`	Tabs, four views, calendar, admin tables, assignee picker, subtask styles
`public/js/api.js`, `ui.js`, `app.js`	New API calls and date helpers; calendar, subtasks, admin, assignee rendering; hash router
`tests/auth.test.ts`, `tests/dashboard.test.ts`	Adjusted to the new tables / `groupTasks`
`README.md`, `docs/*`	Documentation

Deleted: none (the `tasks.user_id` *column* was removed by migration 0003; the old "Users" dialog in `index.html` was replaced by the Admin view).

Data: your database was backed up to `data/backups/todo-before-assignees-2026-11-01.db`, then migrated.

F. Code Explanation

Chapter 21 walks through the important code: the link table, `taskIdsAssignedTo` and `inArray`, `resolveAssignees`, the `createdBy` filtering, `hydrate()` (avoiding N+1 queries), `addSubtask`, `calendarRange`, and the hash router. Chapter 7 shows the data migration SQL and the `runMigrations` fix. Chapter 10 (Flow 2) shows the new `createTask` with its transaction.

G. Complete Code Flow

```
Admin: Admin tab → "Assign a task" → Group → ticks Sita + Ram → Create task
       ↓
readTaskForm() → { title, …, assigneeIds: [2, 3] }  (refuses < 2 people for a group)
       ↓
POST /api/tasks → requireAuth → validate(createTaskSchema)
       ↓
createTask(db, viewer=admin, input)
  resolveAssignees: admin? yes · users exist? yes
  TRANSACTION: INSERT tasks (created_by = admin) + INSERT 2 rows into task_assignees
       ↓
201 { data: { …, assignees: [Ram, Sita], isGroup: true, createdBy: {Minal} } }
       ↓
Sita opens Tasks → GET /api/tasks → WHERE id IN (SELECT task_id … user_id = Sita)
       ↓
{ …, assignees, isGroup: true, canDelete: false }   (no createdBy for Sita)
       ↓
Sita adds a subtask → POST /api/tasks/:id/subtasks → getAccessibleTask ✓ → 201
Ram ticks it        → PATCH /api/subtasks/:id {done:true} → getAccessibleTask ✓ → 200
```

H. Testing

What	How	Expected	Actual	Result
Types	`pnpm typecheck`	no errors	no errors	Passed
Automated tests	`pnpm test`	all pass	**93 passed** in 9 files (23 new)	Passed
Regression test really detects the bug	Temporarily removed the `runMigrations` fix and ran `tests/migrations.test.ts`	fails	failed (`expected [] to equal [{ task_id: 1, user_id: 1 }]`); passed again with the fix	Passed
Browser smoke test	`pnpm test:e2e`	all steps, no console errors	**25/25** passed, no console errors	Passed
Migration on a copy of the real DB (first try)	Copied `data/todo.db`, ran migrations	7 assignments kept	**0 assignments — data loss found**	Failed → fixed
Migration on a copy (after the fix)	Same, with the new `runMigrations`	7 kept	7 tasks, 7 assignments, 7 creators	Passed
Real database upgrade	Backup, then restart the server with the new code	migrations 0002–0003 applied, nothing lost	4 migrations applied; 7 tasks / 7 assignments / 7 creators; 0 unassigned; `user_id` column gone; `/api/health` ok	Passed

What	How	Expected	Actual	Result
Visual check	Screenshots of Dashboard, Tasks, Calendar, Admin, the assign form, and a normal user's Tasks view (temporary demo database)	All views correct; no "Created by"/Delete for the normal user	As expected; the Tasks toolbar's Sort box stretched too wide and was fixed	Passed

Other issues found and fixed during the work:

- drizzle-kit cannot ask its "is this a rename?" question in a non-interactive shell, so the change was split into two migrations (Chapter 7).
- drizzle-kit again omitted the `ON DELETE` rule on an `ALTER TABLE … ADD` foreign key; added by hand.
- The new server could not start at first because a leftover server process from earlier still held its log file; the leftover processes were stopped.
- One smoke-test step clicked a tab before the login request had finished; the test now waits for the app to appear.

I. What I Learned

- A many-to-many relationship needs a link table; "is this a group?" is best *calculated* from it.
- Authorization can depend on the role (`viewer.role`) as well as on ownership, and different actions (see / edit / delete / assign) can have different rules.
- Sending a field only to some users (`createdBy`) is another form of authorization.
- Load related data for many rows at once to avoid the N+1 problem.
- Migrations that move data need extra care: test them on a copy of real data, back up first, and add a regression test.

J. What I Can Try Next

1. Add the "Remove me from this group task" feature from Chapter 21's exercise.
2. Show subtask progress (e.g. "2/5") in the Calendar chips and the Admin table.
3. Add a week view to the calendar (7 columns, one row) — reuse `addDays` .

K. Important Notes

- Admins' **Tasks** view shows tasks assigned to *them*; use the **Admin** tab to see everything.
- Categories are personal; group members keep but cannot change to the creator's categories.
- Tasks without a due date do not appear in the calendar.
- Removing a person from all of a task's assignees is not possible (at least one is required); deleting a user leaves tasks they created in place with an empty "Created by".
- The backup `data/backups/todo-before-assignees-2026-11-01.db` can be deleted once you are happy with the upgrade.

Development Entry: #7 — Different View Types in the Calendar

Date: 2026-11-01

A. What I Asked For

"add different view type in calendar" — besides the month grid, other ways of looking at the calendar.

B. What We Are Building

A **Month / Week / Day / Agenda** switcher above the calendar (details in Chapter 21, "The calendar's four view types"). ‹ › move by the size of the current view, **Today** jumps back, clicking a date opens it in the Day view, and the chosen view type is remembered in the browser.

C. Why We Need It

Different questions need different views: the month for the big picture, the week for planning, the day for focus, and the agenda for "what's coming next?" without empty days.

D. Technical Concepts

- **View state as data:** a *mode* plus an *anchor* date describe any calendar view.
- **Pure functions** (`calendarRange`, `shiftCalendar`): output depends only on input — easy to reason about.
- **Date arithmetic** with `getDay()`, modulo (`% 7`) and JavaScript's automatic month roll-over.
- **localStorage** for remembering a harmless preference, and why it is unsuitable for secrets.
- `aria-pressed` on toggle buttons, so screen readers announce which view is active.
- **CSS modifier classes** (`mode-week`, `mode-day`, …) switching a grid layout to a list layout, and a media query turning the week into a vertical list on phones.

E. Files Changed

File	Change
`public/index.html`	The Month/Week/Day/Agenda button group; updated calendar note
`public/css/styles.css`	Switcher, week/day/agenda layouts, two-line chips, clickable dates, mobile week list
`public/js/ui.js`	`CALENDAR_MODES`, `AGENDA_DAYS`, `calendarRange(mode, anchor)`, `shiftCalendar`, `renderCalendar(mode, anchor, …)` with helpers `parseDate`, `mondayOf`, `calendarTitle`, `calendarChip`, `calendarCell`, `calendarRow`
`public/js/app.js`	`state.calendarMode` (from localStorage) and `state.calendarAnchor` replace `calendarMonth`; `setCalendarMode`; `moveCalendar(direction)`; switcher buttons wired; `firstOfMonth` removed

File	Change
`scripts/e2e-smoke.ts`	New step testing all four view types (26 steps)
`docs/*`	Chapter 21, 11, 13, 15 updates, this entry, changelog

No backend files changed: the existing `dueFrom` / `dueTo` filters already return exactly the tasks each view needs.

F. Code Explanation

The important code — `calendarRange`, `shiftCalendar`, `mondayOf`, how `renderCalendar` builds each view type, and `setCalendarMode` with `localStorage` — is explained step by step in Chapter 21.

G. Complete Code Flow

```
Click "Week"  →  setCalendarMode('week')  →  localStorage.calendarMode = 'week'
       ↓
loadCalendar(): calendarRange('week', anchor) → { start: Monday, end: Sunday }
       ↓
GET /api/tasks?dueFrom=Monday&dueTo=Sunday&sort=dueDate&order=asc
       ↓  (requireAuth → validate → listTasks: tasks assigned to me in that range)
200 { data: [...] }
       ↓
renderCalendar('week', anchor, tasks): 7 calendarCells with detailed chips
       ↓
Click "›"  →  shiftCalendar('week', anchor, +1) = anchor + 7 days  →  loadCalendar()
Click "Sun 8" → setCalendarMode('day', '…-08') → the Day view of that date
```

H. Testing

What	How	Expected	Actual	Result
Types / unit tests	`pnpm typecheck`, `pnpm test`	unchanged, all pass	no errors; 93/93 passed	Passed
Browser	`pnpm test:e2e`, new step: Week shows 7 days and the task; Day shows the task, › moves to an empty next day, ‹ comes back; Agenda groups the task under today; after reload Agenda is still selected; clicking the agenda date opens the Day view	all pass, no console errors	**26/26** passed, no console errors	Passed
Visual	Screenshots of Week, Day and Agenda (desktop) and Week on a 390 px phone screen, with demo data	readable layouts; week becomes a vertical list on phones	As expected	Passed

I. What I Learned

One small piece of state (mode + anchor) and two pure functions can drive four different views. Preferences can be remembered with `localStorage`, but anything from the browser must still be checked before use.

J. What I Can Try Next

1. Add a "This year" view showing 12 small month grids.
2. Let the user choose whether weeks start on Monday or Sunday (another `localStorage` preference) — only `mondayOf` and `WEEKDAYS` need to change.
3. Drag a task to another day to change its due date (HTML drag-and-drop events + `api.updateTask`).

K. Important Notes

- The view type is stored per browser, not per account.
- Agenda covers 30 days from its anchor (`AGENDA_DAYS` in `ui.js`).
- Tasks without a due date still appear only in the Tasks view.

Development Entry: #8 — Rename to "Aeto To-Do", Logo Theme, and Publish on GitHub

Date: 2026-11-01

A. What I Asked For

"name the project to aeto to do and create a new repo in minal-aetonet and push it in my github", followed by *"change the theme to match the logo too"* (with the Aeto logo attached in a dark-on-light and a light-on-dark version).

B. What We Are Building

- The app is renamed **Aeto To-Do** (page title, logo, `package.json` name `aeto-to-do`, README, manual).
- A **theme** based on the logo: charcoal `#383745`, orange gradient `#f58c5f → #ef5a35`, cream `#f7f3ef`; a simplified swoosh mark and a new favicon; the manual uses the same colours.
- The project gets its **own Git repository** and is pushed to a new **private** GitHub repository `minal-aetonet/aeto-to-do`.

C. Why We Need It

A recognisable name and brand make the app feel like a real product; GitHub gives you an online backup, a history of every change, and the basis for sharing and deploying the app.

D. Technical Concepts

Git, commits, repositories, remotes, push/clone, `.gitignore`, private vs public repositories (Chapter 17, "Your code on GitHub"); CSS design tokens and colour contrast (Chapter 8, "The Aeto theme"); SVG gradients.

E. Files Changed

File	Change
`package.json`	`name: "aeto-to-do"`, new description
`public/index.html`	Title "Aeto To-Do"; logo text and swoosh mark in the top bar and on the login card
`public/img/aeto-mark.svg`	**New** — the orange swoosh
`public/favicon.svg`	Swoosh on charcoal
`public/css/styles.css`	Brand tokens; warm neutrals; charcoal top bar and login screen; orange accents; old indigo tints replaced by tokens
`scripts/build-docs.ts`	Manual title texts and colours
`tests/app.test.ts`	Expects `<title>Aeto To-Do</title>`
`README.md`, `docs/*`	New name, clone instructions, Git section, this entry
`.git/` (not a file in the project)	New local repository with branch `main`, remote `origin` on GitHub

The local folder is still called `hono-todo` (the running server uses it); a fresh `git clone` creates a folder named `aeto-to-do`.

F. Code Explanation

The theme block and the contrast reasoning are explained in Chapter 8 ("The Aeto theme"). The swoosh is an SVG `<path>` drawn with cubic Bézier curves (`C x1 y1 x2 y2 x y`) and painted with a `<linearGradient>` stroke. The top bar's orange line uses `border-image: var(--brand-gradient) 1`, which paints a border with a gradient instead of a single colour.

G. Complete Code Flow

```
Rename + theme (files)  →  pnpm typecheck · pnpm test · pnpm test:e2e · pnpm docs:build
       ↓
git init -b main   →  check the file list (no data/, .env, node_modules)
       ↓
git add . → git commit -m "…"           (snapshot on your computer)
       ↓
gh auth login (you approve in the browser)
       ↓
gh repo create minal-aetonet/aeto-to-do --private --source . --push
       ↓
https://github.com/minal-aetonet/aeto-to-do
```

H. Testing

What	How	Result
Types, tests	`pnpm typecheck`, `pnpm test`	See the CHANGELOG entry
Browser	`pnpm test:e2e`	See the CHANGELOG entry
Visual	Screenshots of the login screen, Dashboard and Tasks with demo data	Charcoal top bar and login, orange swoosh, orange buttons, readable text
Nothing private is published	`git status --untracked-files=all` and `git check-ignore` before the first commit	75 project files; `data/` (database + backup), `.env`, `node_modules` ignored
Repository	`gh repo view` after pushing	See the CHANGELOG entry

I. What I Learned

How Git and GitHub keep a history and an online copy of a project; why `.gitignore` matters before the first push; how a whole theme can be changed through a few CSS variables; why accessible contrast sometimes needs a slightly different shade than the brand colour.

J. What I Can Try Next

1. Make a small change, then `git status`, `git add .`, `git commit -m "…"`, `git push`, and find the commit on GitHub.
2. Replace `public/img/aeto-mark.svg` with the original logo file and adjust `.logo-mark` in the CSS.
3. Add a dark mode: a second set of the same CSS variables inside `@media (prefers-color-scheme: dark) { :root { … } }`.

K. Important Notes

- The repository is **private**. Change it in GitHub → *Settings* → *Danger zone* → *Change visibility* if you want it public.

- Your database and its backup are **not** on GitHub (they are in `data/`, which is ignored). Back them up separately.
- The swoosh is a simplified re-drawing of the logo, not the original artwork.

Glossary

Term	Meaning
Admin	A user with the `admin` role; may use `/api/admin/*`
API	Application Programming Interface — URLs a program can call to read or change data
Arrow function	Short function syntax: `(x) => x * 2`
Assignee	A person a task is assigned to (a row in `task_assignees`)
async / await	Keywords for writing code that waits for Promises step by step
Authentication	Checking *who* someone is (login); failure → `401`
Authorization	Checking *what* someone may do (own data, roles); failure → `403` or `404`
Brute-force attack	Trying many passwords until one works; slowed down by the login throttle
Backend	The part of an application that runs on the server
Body (HTTP)	The data part of a request or response, e.g. JSON
Build id	A fingerprint printed on both manual formats to show they come from the same source
Callback	A function passed to another function to be called later
Clone	Download a Git repository (with its history) for the first time
Column	One named piece of information in a database table
Commit	A saved snapshot of the project in Git, with a message explaining the change
Contrast ratio	How different text and background colours are in brightness; at least 4.5 : 1 for normal text (WCAG AA)
Composite primary key	A primary key made of several columns together, e.g. (`task_id`, `user_id`)
Constant-time comparison	Comparing secrets in a way that always takes the same time (`timingSafeEqual`)
Context (`c`)	Hono's per-request object with the request, response helpers and stored variables
Cookie	A small named value the browser stores for a site and sends back with every request to it
CRUD	Create, Read, Update, Delete
CSP	Content Security Policy — a header limiting where a page may load scripts and styles from
CSRF	Cross-Site Request Forgery — another website making your browser send requests to our app with your cookie
CSS	The language that styles web pages
Debounce	Waiting until a burst of events stops before reacting (used for search)

Term	Meaning
Dependency	A package your project needs
Dependency injection	Giving code the things it needs (like the database) instead of letting it fetch them itself
DOM	Document Object Model — the browser's in-memory tree of the page
Drizzle ORM	The TypeScript library we use to query the database
Endpoint	One method + URL combination of an API, e.g. `POST /api/tasks`
Environment variable	A named setting given to a program by the operating system, e.g. `PORT`
Event listener	A function that runs when an event (click, submit…) happens
fetch	The browser function for sending HTTP requests
Foreign key	A column that refers to the primary key of another table
Frontend	The part of an application that runs in the browser
Git	A version control system that records the history of a project's files
GitHub	A website that hosts Git repositories online
Generic	A type with a placeholder filled in later, e.g. `Array<string>`
Handler	The function that runs when a route matches
Group task	A task assigned to two or more people, who share it
Hash	A one-way "fingerprint" of data; used for passwords (scrypt) and session tokens (SHA-256)
Hash (URL) / hash routing	The part of an address after `#`; changing it switches views without reloading the page
Header (HTTP)	A `Name: value` line with extra information about a request or response
Hono	The web framework this project uses
HttpOnly	A cookie flag that hides the cookie from page JavaScript
IDOR	Insecure Direct Object Reference — reaching other people's records by changing an ID in the URL
Individual task	A task assigned to exactly one person
HTML	The language describing the structure of web pages
HTTP	The protocol browsers and servers use to communicate
Index (database)	A structure that makes searching a column faster
Integration test	A test that checks several parts working together
JSON	JavaScript Object Notation — a text format for data
JWT	JSON Web Token — a signed token containing user data; an alternative to database sessions

Term	Meaning
LEFT JOIN	SQL that combines rows from two tables, keeping rows without a match
localStorage	A small key–value store in the browser that survives reloads; good for preferences, never for secrets
Link table (join table)	A table whose rows connect rows of two other tables, creating a many-to-many relationship
Many-to-many	A relationship where each side can have many of the other (tasks ↔ users)
Lockfile	A file recording exact package versions (`pnpm-lock.yaml`)
Middleware	Code that runs before/after route handlers
Migration	A versioned file of SQL that changes the database structure
Module	A file that shares code with `export` and uses code with `import`
N+1 query problem	Running one query for a list and then one more per item; avoided by loading related rows for all items at once
Node.js	A program that runs JavaScript outside the browser
npm	The public package registry, and Node's default package manager
NULL	"No value" in a database
Parameterised query	SQL where values are sent separately from the SQL text — prevents SQL injection
pnpm	The fast, disk-efficient package manager this project uses
Port	A numbered "door" on a computer where a server listens (ours: 3000)
Primary key	A column that uniquely identifies each row
Pure function	A function whose result depends only on its inputs and that changes nothing else
Push / remote	Upload commits to the online copy of a repository (the "remote", usually named `origin`)
Regression test	A test written for a bug, so the bug can never silently come back
Repository (repo)	A project folder whose history is tracked by Git
Promise	An object representing a value that will be available later
Query parameter	`key=value` data after `?` in a URL
REST	A style of API design using resources (URLs) and HTTP methods
Route	A method + path connected to a handler
Role	A user's permission level: `user` or `admin`
Row	One record in a database table
Salt	Random bytes mixed into a password before hashing, so equal passwords get different hashes
SameSite	A cookie flag controlling whether the cookie is sent with requests started by other websites

Term	Meaning
scrypt	A deliberately slow, memory-hard password-hashing function built into Node.js
Secure (cookie)	A cookie flag: only send the cookie over HTTPS
Session	A server-side record that a browser is logged in, identified by a random token in a cookie
Schema (database)	The description of all tables and columns
Schema (Zod)	A description of what valid data looks like
Service	A function in `src/services` containing business logic and database access
SQL	The language for talking to relational databases
SQL injection	An attack that sneaks SQL commands into user input
SQLite	A database stored in a single file
Stack trace	The list of function calls leading to an error
Static file	A file sent to the browser unchanged (HTML, CSS, JS, images)
Subquery	A query inside another query, e.g. `id IN (SELECT task_id FROM …)`
Subtask	A checklist item inside a task
Status code	A three-digit number summarising an HTTP response (200, 404…)
tsx	A tool that runs TypeScript files directly in Node.js
Transaction	A group of database changes that are saved together or not at all
Type	A description of what kind of value something is
Type inference	TypeScript working out a type automatically
TypeScript	JavaScript with types
Unit test	A test that checks one small piece in isolation
User enumeration	Finding out which emails have accounts from differences in error messages or timing
Validation	Checking that input data follows the rules
Viewer	In our services: the logged-in user (`id` and `role`) on whose behalf data is read or changed
Vitest	The testing framework this project uses
WAL	Write-Ahead Log — a SQLite mode that writes changes to a side file first
XSS	Cross-Site Scripting — an attack that injects code into a web page
Zod	The library we use to validate data

Appendix — Change History

Every development request and the resulting changes, **newest first**. Entries are never removed. The detailed learning explanation for each entry is in the **Development Log** part of the manual.

2026-11-01 — Entry #8: Rename to "Aeto To-Do", logo theme, publish on GitHub

Original requests: "name the projext to aeto to do and create a new repo in minal-aetonet and push it in my git hub" · "change the theme to match the logo too" (Aeto logo attached)

Summary:

- **Rename:** the app is now **Aeto To-Do** (title, logo text, `package.json` name `aeto-to-do`, README, manual).
- **Theme from the logo:**
  - Colours: charcoal `#383745`, the orange gradient `#f58c5f → #ef5a35`, and cream `#f7f3ef`.
  - A charcoal top bar with an orange gradient line, and a charcoal login screen.
  - Orange buttons use `#cf4520` so white text keeps WCAG AA contrast.
  - A simplified orange swoosh mark (`public/img/aeto-mark.svg`) and a new favicon.
  - The manual uses the same colours.
- **Git:** the project got its own Git repository (branch `main`). The parent repository at `C:\` is not used, so nothing outside the project can be published.
  - Before the first commit, the file list was checked: 75 files, with `data/` (database and backup), `.env` and `node_modules` ignored.
- **GitHub:** the GitHub CLI was installed with winget. The private repository `minal-aetonet/aeto-to-do` is created and pushed after the browser login is approved (see the next entry or the commit history for the result).

Files created: `public/img/aeto-mark.svg` **Files modified:** `package.json`, `public/index.html`, `public/favicon.svg`, `public/css/styles.css`, `scripts/build-docs.ts`, `tests/app.test.ts`, `README.md`, `docs/DOCUMENTATION-SOURCE.md`, `docs/CHANGELOG.md` (HTML and PDF regenerated) **Files deleted:** none

Tests performed:

- `pnpm typecheck`: no errors.
- `pnpm test`: 93/93 passed.
- `pnpm test:e2e`: 26/26 passed, with no console errors.
- Screenshots of the login screen, Dashboard and Tasks.
- Git ignore check before the first commit.
- `pnpm docs:build`: manual regenerated and verified.

Documentation sections updated: Preface (clone instructions), Chapter 3 (`package.json` name), Chapter 8 (new "The Aeto theme" section), Chapter 9 (folder name), Chapter 17 (new "Your code on GitHub" section; deployment path), Development Log Entry #8, Glossary (8 new Git and colour terms).

2026-11-01 — Entry #7: Different view types in the calendar

Original request: "add differnt view type in calander"

Summary:

- **New:** a Month / Week / Day / Agenda switcher on the Calendar view.
  - **Week:** 7 tall day columns; each chip also shows the status and subtask progress.
  - **Day:** a detailed list of one day, with "+ Add task on this day".
  - **Agenda:** the next 30 days, grouped by date, showing only days with tasks.
- **Navigation:** ‹ › move by the size of the current view, **Today** jumps back, and clicking a date opens it in the Day view.
- **Remembered:** the chosen view type is stored in `localStorage`.
- **On phones:** the week becomes a vertical list.
- **Frontend-only:** the existing `dueFrom` / `dueTo` API filters supply each view's date range.

Files created: none **Files modified:** `public/index.html`, `public/css/styles.css`, `public/js/ui.js`, `public/js/app.js`, `scripts/e2e-smoke.ts`, `docs/DOCUMENTATION-SOURCE.md`, `docs/CHANGELOG.md` (HTML and PDF regenerated) **Files deleted:** none

Tests performed:

- `pnpm typecheck`: no errors.
- `pnpm test`: 93/93 passed.
- `pnpm test:e2e`: 26/26 passed. A new step covers all four view types, ‹ › in Day view, the remembered view type after a reload, and clicking a date.
- Screenshots of Week, Day and Agenda on desktop, plus Week on a phone-sized screen.
- `pnpm docs:build`: manual regenerated and verified.

Documentation sections updated: Chapter 21 (a new section on the calendar's four view types: anchor date, `calendarRange`, `shiftCalendar`, rendering, `localStorage`), Chapter 11 (frontend functions), Chapter 13 (e2e steps), Chapter 15 (2 troubleshooting rows), Development Log Entry #7, Glossary (localStorage, pure function).

2026-11-01 — Entry #6: Admin assigns tasks, group and individual tasks, subtasks, separate views, "created by" for admins

Original request: "add a feature in admin to give a task to user and sepraste a dash board and calander like view too / add group task and inviuwal task too add a option to add task inside task add a view to only aadmin on who created a task"

Summary:

- **Assignments:** a new `task_assignees` link table (many-to-many) replaces the single owner column `tasks.user_id`.
  - Admins assign a task to one person (individual) or several people (group).
  - Normal users keep creating individual tasks for themselves.
- **Group tasks:** every member sees, edits and completes the same task and shares its checklist.
  - Only the creator or an admin can delete a group task.
  - Only admins change who it's assigned to.
- **Subtasks:** a checklist inside every task, via a new `subtasks` table and the endpoints `POST /api/tasks/:id/subtasks`, `PATCH /api/subtasks/:id` and `DELETE /api/subtasks/:id`.
- **Views:** Dashboard (7 cards including Group tasks, plus Overdue and Next-7-days lists), Tasks (with an Individual/Group filter), Calendar (month grid; "+" adds a task on that day) and Admin (users, plus all tasks with Type, Assigned to and Created by). They are switched with URL hash routing.
- **"Created by":** a new `tasks.created_by` column. The creator is included in API responses for admins only; normal users get `canDelete` instead.
- **New API features:** `GET /api/admin/tasks`, the `assigneeIds` field on create and update, the `kind`, `dueFrom` and `dueTo` filters, and `groupTasks` on the dashboard.
- **Data-loss bug found and fixed before it reached real data:**
  - The table rebuild in migration 0003 cascade-deleted all assignee rows, because Drizzle runs migrations inside a transaction, where SQLite ignores `PRAGMA foreign_keys=OFF`.
  - `runMigrations` now turns foreign keys off outside the transaction, runs `PRAGMA foreign_key_check`, and turns them back on. A regression test was added for this.
- **Real database:** backed up to `data/backups/todo-before-assignees-2026-11-01.db`, then migrated. All 7 tasks kept their creator and assignee.

Files created:

- Migrations: `src/db/migrations/0002_assignees_and_subtasks.sql`, `src/db/migrations/0003_drop_task_owner_column.sql`, `src/db/migrations/meta/0002_snapshot.json`, `meta/0003_snapshot.json`
- Subtask code: `src/services/subtask.service.ts`, `src/validators/subtask.validator.ts`, `src/routes/subtask.routes.ts`
- Tests: `tests/assignment.test.ts`, `tests/subtasks.test.ts`, `tests/migrations.test.ts`

Files modified:

- Database: `src/db/schema.ts`, `src/db/index.ts`, `src/db/migrations/meta/_journal.json`
- Services: `src/services/task.service.ts`, `dashboard.service.ts`, `auth.service.ts`
- Validators and routes: `src/validators/task.validator.ts`, `src/routes/task.routes.ts`, `admin.routes.ts`, `src/app.ts`
- Scripts: `scripts/seed.ts`, `scripts/e2e-smoke.ts`
- Frontend: `public/index.html`, `public/css/styles.css`, `public/js/api.js`, `ui.js`, `app.js`
- Tests: `tests/auth.test.ts`, `tests/dashboard.test.ts`
- Docs: `README.md`, `docs/DOCUMENTATION-SOURCE.md`, `docs/CHANGELOG.md`, and the regenerated HTML and PDF

Files deleted: none. The `tasks.user_id` column was removed by migration 0003.

Tests performed:

- `pnpm typecheck`: no errors.
- `pnpm test`: 93/93 passed across 9 files, 23 of them new.
- The regression test was confirmed to fail without the `runMigrations` fix and to pass with it.
- `pnpm test:e2e`: 25/25 browser steps passed with no console errors.
- Migration tests on a copy of the real database: the first run lost all 7 assignments; after the fix, all 7 were kept.
- Real database upgrade: 4 migrations applied, 7 tasks with 7 assignments and 7 creators, the `user_id` column removed, and the health check `ok`.
- Screenshots of all views and the assign form, plus a normal user's view, all checked against a temporary demo database.

Documentation sections updated:

- Preface (book layout), Chapter 1 (features), Chapter 2 (handler example)
- Chapter 7 (six tables, schema, composite key, new Drizzle helpers, migrations 0002/0003 and the data-loss fix)
- Chapter 8 (views and hash routing)
- Chapter 9 (file tree, decisions)
- Chapter 10 (flows using the viewer and assignees)
- Chapter 11 (task service, the new subtask service, validators, routes, frontend functions, `runMigrations`)
- Chapter 12 (Task object fields, new parameters and endpoints, `INVALID_ASSIGNEE`)
- Chapter 13 (test table: 93 tests, 25 e2e steps)
- Chapter 14 (security rows)
- Chapter 15 (5 troubleshooting rows)
- Chapter 16 (recipes: `deleteCompletedTasks` now uses the creator; the subtasks example became a comments example)
- Chapter 18 (roadmap)
- Chapter 20 (authorization code, registration code, roles table, Flow C)
- **New Chapter 21 — Teamwork: Assigning Tasks, Group Tasks, Subtasks and Views**
- Development Log Entry #6, Glossary (12 new terms)

2026-11-01 — Entry #5: Bug report "Route POST /api/auth/register was not found."

Original request: "Route POST /api/auth/register was not found."

Summary: No code change. The server on port 3000 was a stale process that Claude had started at 3:21 PM, during the initial build, for screenshots. An earlier stop command did not end it, so it was still running code from before the authentication routes existed (added 4:18 PM). Claude stopped the stale processes and started the server with the current code, which also ran the accounts migration on `data/todo.db`.

Registration now works: the first account, *Minal Poudel* (`admin@gmail.com`), is the admin and owns the 6 earlier tasks.

Files created: none **Files modified:** `docs/DOCUMENTATION-SOURCE.md`, `docs/CHANGELOG.md` (HTML and PDF regenerated) **Files deleted:** none

Tests performed:

- Port and process inspection: the process started at 3:21 PM, before `auth.routes.ts` was created at 4:18 PM.
- Probes after the restart: `GET /api/auth/me` returned `401`, and `POST /api/auth/register` with an empty body returned `400` instead of `404`, without creating an account.
- Database check: the `users` and `sessions` tables exist, there is 1 admin account, and 0 ownerless tasks remain.
- Server log: the user's registration returned `201`, and `/api/admin/users` returned `200`.
- `pnpm docs:build`: manual regenerated and verified.

Documentation sections updated: Chapter 15 (new troubleshooting row for "Route … was not found" caused by an old process), Development Log Entry #5.

2026-11-01 — Entry #4: Question about the admin ID and password

Original request: "is the id password of admin"

Summary: No code change. There is no built-in admin account or default password. The first account registered becomes the admin, with the email and password chosen at registration. The optional `pnpm db:seed` demo account (`demo@example.com` / `demo-password-123`) is created only when no accounts exist. A read-only check of the real `data/todo.db` showed no accounts yet: the accounts migration will run on the next `pnpm start`, and 6 tasks are waiting for the first account.

Files created: none **Files modified:** `docs/DOCUMENTATION-SOURCE.md`, `docs/CHANGELOG.md` (HTML and PDF regenerated) **Files deleted:** none

Tests performed: a read-only inspection of `data/todo.db`. The existing tests covering "first account is admin" and "claims ownerless data" are part of the 70 passing tests. `pnpm docs:build` regenerated and verified the manual.

Documentation sections updated: Chapter 20 (new subsection "What is the admin's email and password?"), Chapter 15 (troubleshooting row), Development Log Entry #4.

2026-11-01 — Entry #3: Authentication and authorization

Original request: "add ahotrizastion and authetication and add it in doc too" (add authorization and authentication, and document them)

Summary:

- **Accounts:** register, log in, log out and `/api/auth/me`.
  - Passwords are stored as salted scrypt hashes (Node's built-in crypto, so no new dependencies).
- **Sessions:**
  - A 32-byte random token is kept in an `HttpOnly`, `SameSite=Lax` cookie (with `Secure` when `COOKIE_SECURE=true`).
  - Only the token's SHA-256 hash is stored, in a new `sessions` table.
  - Sessions last 7 days, and logout deletes them.
- **Authentication:** `requireAuth` protects every task, category, dashboard and admin route (`401` without a valid session).
- **Authorization:**
  - Tasks and categories have an owner (`user_id`), and every service query filters by the logged-in user. Other users' records answer `404`.
  - Category names are unique per user.
  - `requireRole('admin')` protects `GET /api/admin/users` (`403` for other users).
  - The first account becomes admin and claims all existing ownerless data.
- **Hardening:**
  - Hono `csrf()` middleware.
  - Login throttling: 5 failures per email within 15 minutes gives `429`.
  - The same error message, and similar timing, for an unknown email and a wrong password.
- **Frontend:**
  - Login/register screen, plus the signed-in user's name, an admin badge and a Log out button.
  - An admin-only Users dialog.
  - Automatic return to the login screen after a 401.
- **Seed script:** now creates the demo account `demo@example.com` / `demo-password-123`.

Files created:

- Migration: `src/db/migrations/0001_add_users_and_sessions.sql`, `src/db/migrations/meta/0001_snapshot.json`
- Helpers: `src/lib/password.ts`, `src/lib/login-throttle.ts`
- Auth code: `src/validators/auth.validator.ts`, `src/services/auth.service.ts`, `src/middleware/auth.ts`
- Routes: `src/routes/auth.routes.ts`, `src/routes/admin.routes.ts`
- Tests: `tests/auth.test.ts`, `tests/authorization.test.ts`

Files modified:

- Database and config: `src/db/schema.ts`, `src/db/migrations/meta/_journal.json`, `src/types.ts`, `src/config.ts`, `src/app.ts`, `src/index.ts`, `.env.example`
- Services: `src/services/task.service.ts`, `category.service.ts`, `dashboard.service.ts`
- Routes: `src/routes/task.routes.ts`, `category.routes.ts`, `dashboard.routes.ts`
- Scripts: `scripts/seed.ts`, `scripts/e2e-smoke.ts`
- Frontend: `public/index.html`, `public/css/styles.css`, `public/js/api.js`, `ui.js`, `app.js`
- Tests: `tests/helpers.ts`, `tasks.test.ts`, `categories.test.ts`, `dashboard.test.ts`, `app.test.ts`

- Docs: `README.md`, `docs/DOCUMENTATION-SOURCE.md`, `docs/CHANGELOG.md`, and the regenerated HTML and PDF

Files deleted: none

Tests performed:

- `pnpm typecheck`: no errors.
- `pnpm test`: 70/70 passed across 6 files. That is 30 new authentication and authorization tests, 1 new CSRF test, and the existing tests updated to run as a logged-in user.
- `pnpm test:e2e`: 18/18 browser steps passed with no console errors. The new steps cover:
  - the login screen;
  - registration validation and registering the first account (admin);
  - the admin Users list;
  - a second user in a separate browser who sees none of the first user's tasks;
  - logout, a refused wrong password, and logging back in.
- Upgrade test on a **copy** of the real `data/todo.db`:
  - the migration applied;
  - the first registered account received the 6 existing tasks;
  - `GET /api/tasks` without a cookie returned `401`.
- Screenshots of the login screen and the logged-in app.
- Issues handled:
  - drizzle-kit left out `ON DELETE cascade` on the new `user_id` columns. It was added by hand to the not-yet-applied migration.
  - Body-less requests need an `Origin` header to pass the CSRF check. The test helper now sends one, and the `curl` examples were updated.
- `pnpm docs:build`: HTML and PDF regenerated and verified.

Documentation sections updated:

- Preface (quick start, book layout)
- Chapters 1 (features, page load), 2 (route groups, handler example, `createApp`) and 4 (async note)
- Chapter 5 (401/403/429, cookies, logged-in "Try it" commands)
- Chapter 6 (onion diagram, middleware order, auth middleware, `csrf`)
- Chapter 7 (four tables, schema, migration 0001)
- Chapter 8 (two screens, frontend login)
- Chapter 9 (architecture, file tree, decisions)
- Chapter 10 (all flows now include auth and `userId`)
- Chapter 11 (new files, changed signatures, config, types)
- Chapter 12 (auth and admin endpoints, User object, error codes)
- Chapter 13 (test helper, test counts)
- Chapter 14 (security summary table)
- Chapters 15 (auth troubleshooting), 16 (recipes with `userId`, new ideas), 17 (`COOKIE_SECURE`, deployment checklist) and 18 (roadmap)
- **New Chapter 20 — Authentication and Authorization**
- Development Log Entry #3, Glossary (19 new terms)

2026-11-01 — Entry #2: Install the project with pnpm

Original request: "install it through pnpm"

Summary: Switched the project from npm to pnpm. Removed npm's lockfile and `node_modules`, reinstalled every package with pnpm, and added `pnpm-workspace.yaml`. The first install failed because pnpm tried to compile `better-sqlite3` from C++ source with node-gyp, and this computer has no C++ build tools. The package already ships prebuilt binaries, so it is now set to `allowBuilds: false`; `esbuild` stays allowed. `package.json` was cleaned up: removed npm's `allowScripts`, added `packageManager`, moved `tsx` to `dependencies`, removed the unused `build` / `start:prod` scripts and the `dotenv` package (Node's built-in `process.loadEnvFile` is used instead), and set `engines.node` to `>=22`.

Files created: `pnpm-lock.yaml`, `pnpm-workspace.yaml`

Files modified: `package.json`

Files deleted: `package-lock.json` (and the npm-created `node_modules/`, reinstalled by pnpm)

Tests performed:

- `pnpm add …` with `better-sqlite3: true` failed (node-gyp has no C++ build tools); diagnosed and fixed with `false`.
- Install succeeded and the lockfile passed pnpm's supply-chain checks.
- SQLite loads (`sqlite_version()` = 3.53.4); `tsx` 4.23.15 and TypeScript 7.0.2 run through pnpm.
- Every test listed in Entry #1 was run with pnpm and passed.

Documentation sections updated: Preface (quick start), Chapter 3 (npm vs pnpm, lockfile, `pnpm-workspace.yaml`), Chapter 15 (pnpm troubleshooting), Chapter 17 (pnpm deployment commands), Development Log Entry #2, Glossary (pnpm, lockfile).

2026-11-01 — Entry #1: Initial build of the Hono To-Do application and learning manual

Original request: Build a fully functional Hono To-Do application (Hono, TypeScript, Node.js, SQLite, Drizzle ORM, Zod, HTML/CSS/vanilla JavaScript, Vitest). It needs a dashboard, full task management, categories, search, filters and sorting, plus a professional interface with notifications and loading, error and empty states. Also create a complete beginner-friendly learning manual in HTML and PDF, generated from a Markdown source with `docs:build`, and keep a changelog.

Summary:

- **Backend:**
  - Hono app built in layers: routes → Zod validation → services → Drizzle/SQLite.
  - Central error handling.
  - Request-logging, security-header, body-limit and database-injection middleware.

- **API:** 13 endpoints covering health, tasks (list, get, create, edit, complete, reopen, delete), categories (list, create, edit, delete) and the dashboard.
- **Database:** `tasks` and `categories` tables with a foreign key (`ON DELETE SET NULL`), indexes, and a generated migration `0000_init.sql`.
- **Frontend:** a responsive single page with a dashboard, filters, a task list, create/edit and category dialogs, a confirmation dialog, toasts, and loading/error/empty states. It is XSS-safe because it renders with `textContent`.
- **Tests:** 39 Vitest tests and an 11-step browser smoke test.
- **Scripts:** migrate, seed, e2e smoke test, documentation generator.
- **Manual:** 19 chapters, a Development Log, a Glossary, and this change history.

Files created:

- Root: `package.json`, `tsconfig.json`, `drizzle.config.ts`, `vitest.config.ts`, `.env.example`, `.gitignore`, `README.md`
- Backend core: `src/index.ts`, `src/app.ts`, `src/config.ts`, `src/types.ts`
- Database: `src/db/schema.ts`, `src/db/index.ts`, `src/db/migrations/0000_init.sql`, `src/db/migrations/meta/*`
- Helpers: `src/lib/errors.ts`, `src/lib/dates.ts`
- Middleware: `src/middleware/request-logger.ts`, `database.ts`, `validate.ts`, `error-handler.ts`
- Validators: `src/validators/common.ts`, `task.validator.ts`, `category.validator.ts`, `dashboard.validator.ts`
- Services: `src/services/task.service.ts`, `category.service.ts`, `dashboard.service.ts`
- Routes: `src/routes/task.routes.ts`, `category.routes.ts`, `dashboard.routes.ts`
- Frontend: `public/index.html`, `public/favicon.svg`, `public/css/styles.css`, `public/js/api.js`, `ui.js`, `app.js`
- Tests: `tests/helpers.ts`, `tasks.test.ts`, `categories.test.ts`, `dashboard.test.ts`, `app.test.ts`
- Scripts: `scripts/migrate.ts`, `seed.ts`, `e2e-smoke.ts`, `build-docs.ts`, `lib/browser.ts`
- Docs: `docs/DOCUMENTATION-SOURCE.md`, `docs/CHANGELOG.md`, `docs/LEARN-HONO-COMPLETE.html`, `docs/LEARN-HONO-COMPLETE.pdf`

Files modified: none (new project)

Files deleted: none

Tests performed:

- `pnpm typecheck`: no errors. One error was found and fixed: the `AppDatabase` type was missing `$client`.
- `pnpm test`: 39/39 passed across 4 test files.
- `pnpm test:e2e`: 11/11 steps passed with no browser console errors. Two test-environment issues were found and fixed: the test viewport was too small, and there was no favicon.
- Manual run:
  - `pnpm db:seed` added 3 categories and 6 tasks.
  - `pnpm start` served `/api/health`, `/api/dashboard` and `/api/tasks` with the correct data.

    - An invalid `POST` returned a 400 with field details.
    - The request log printed correctly.
- Visual check: screenshots at desktop (1280 px) and phone (390 px) widths.
- `pnpm docs:build`: HTML and PDF generated, and the verification passed.
    - The first build hung. Text in the embedded search index (`</script>`, `<img onerror=…>`) broke out of its `<script>` tag and ran `alert(1)`, which blocked the page. Fixed by escaping `<` as `<`.
    - The generator now fails on any script error or dialog in the manual.
    - Code-font ligatures were disabled.

Documentation sections updated: All (initial version): Preface, Chapters 1–19, Development Log Entry #1, Glossary, Appendix.